SPARX: An Open-Source, Automated, Programmatically Generated, Frequency-Scalable Six-Port Receiver in 130-nm CMOS

Open in Colab

Authors: David Kellerer-Pirklbauer, Simon Dorrer, and Harald Pretl

Institute: Institute for Integrated Circuits and Quantum Computing, Johannes Kepler University (JKU), Linz, Austria

Email: {david.kellerer-pirklbauer, simon.dorrer, harald.pretl}@jku.at

Publication: S. Dorrer, D. Kellerer-Pirklbauer, G. Zachl, and H. Pretl, "SPARX: An Open-Source, Automated, Programmatically Generated, Frequency-Scalable Six-Port Receiver in 130-nm CMOS," accepted for publication in the 2026 IEEE Nordic Circuits and Systems Conference (NorCAS), 2026.

Documentation: https://iic-jku.github.io/SG13CMOS_SPARX/ describes the design, the flow, and every simulation result, and the source is in the SPARX repository.

Index Terms: Branch-line coupler, CMOS-only, frequency-scalable layout, GDSFactory, hairpin coupled-line bandpass filter, IHP-Open-PDK, large-signal noise, mmWave, open-source EDA, power detector, programmatic layout, Schottky barrier diode, six-port receiver, Wilkinson power divider.

Project Description

In this notebook, a tapeout-ready frequency-scalable mmWave six-port receiver is programmatically generated in Python using GDSFactory and the 130-nm IHP-Open-PDK with CMOS-only devices. **SPARX** stands for **S**ix-**P**ort **A**utomated Receiver (**RX**).

The goal of this project was to integrate the IHP-Open-PDK and self-made RF devices into GDSFactory to test its limits with an actual tapeout. During development, we pushed the design toward full frequency scalability, so changing the target frequency in the notebook automatically resizes the full RF layout.

The receiver consists of three branch-line couplers, a hairpin coupled-line bandpass filter, a Wilkinson power divider, and a power detector utilizing Schottky barrier diodes (SBDs).

Structure of this notebook:

1. Installation of the IHP-Open-PDK and its GDSFactory add-on.

2. Layout generation: the power detector, the six-port network of branch-line couplers, Wilkinson power divider, and bandpass filter, the probe pads and bondpads, the sealring, the metal fill, and the GDS export, all derived from the design frequency `f`.
3. Design rule check of the complete chip with the KLayout DRC decks of the IHP-Open-PDK.
4. Circuit simulation with VACASK: the EM result of the six-port core is converted into a lumped-element netlist with snp2le, which carries the thermal noise of the core, the power detector is characterized, the receiver is simulated with its post-layout power detectors, and the noise figure of the receiver is simulated through the core model.
5. Conclusion.

Block Diagram

The following block diagram visualizes the mmWave six-port receiver design.

The six-port receiver performs complex frequency down-conversion by additive mixing. The LO and RF signals are superimposed in the linear, passive six-port core under four relative phases of 0, $\pi/2$, π, and $3\pi/2$. The four resulting waveforms drive four power detectors, whose nonlinearity produces two differential in-phase and quadrature pairs at the outputs, V_{I+}, V_{I-}, V_{Q+}, and V_{Q-}. The baseband signal is recovered by subtraction, $V_I = V_{I+} - V_{I-}$ and $V_Q = V_{Q+} - V_{Q-}$, apart from a calibration of the static imbalances of the network. With $f_{\mathrm{RF}} = f_{\mathrm{LO}}$ the receiver converts directly to baseband (homodyne), and with an LO offset it becomes a heterodyne receiver with a low IF, as in the circuit simulation at the end of this notebook, where the LO at 159 GHz and the RF at 161 GHz give a 2 GHz IF.

In [1]:
```
import os
import shutil
import time
import urllib.request

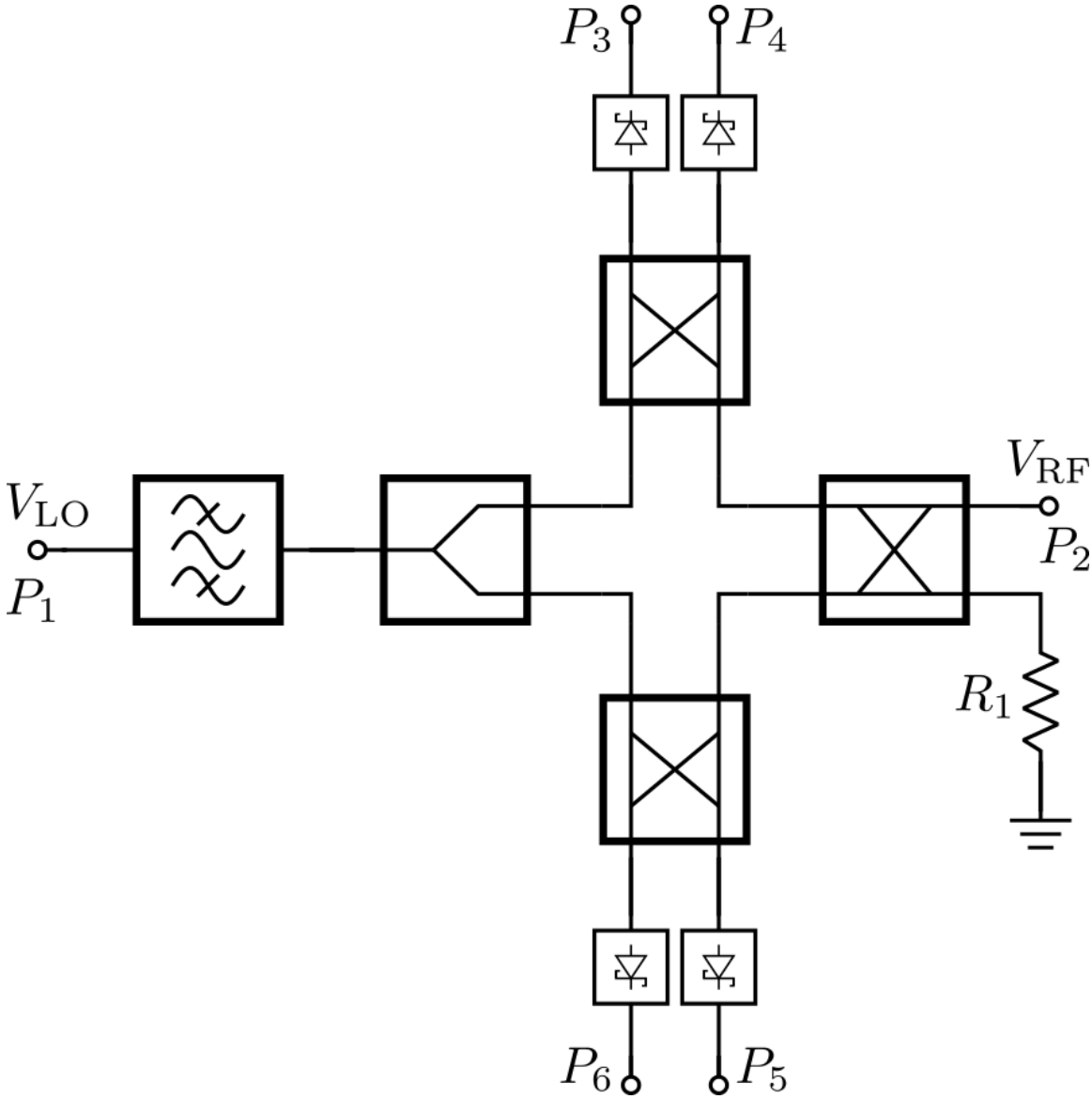
# the runtime of the whole notebook is measured from here to the last cell
NOTEBOOK_START = time.time()
print(f"Notebook started at {time.strftime('%Y-%m-%d %H:%M:%S %Z')}")

# branch of the SPARX repository the assets (logos, block diagram, video) are
taken from
SPARX_BRANCH = "next_release"
ASSET_URL    = f"https://raw.githubusercontent.com/iic-
jku/SG13CMOS_SPARX/{SPARX_BRANCH}/sscs-ose-code-a-chip/assets"
ASSET_DIR    = "/tmp/assets"

# download the block diagram and the logo GDS files, the video is streamed below
shutil.rmtree(ASSET_DIR, ignore_errors=True)
os.makedirs(ASSET_DIR)
for asset in ["sparx_blockdiagram.png", "jku_logo_m5.gds", "IWS.gds",
"D_Kellerer.gds", "supervisors.gds"]:
    urllib.request.urlretrieve(f"{ASSET_URL}/{asset}", f"{ASSET_DIR}/{asset}")

from IPython.display import Image, display
display(Image(filename=f"{ASSET_DIR}/sparx_blockdiagram.png", width=576))
```

```
Notebook started at 2026-10-12 14:46:08 UTC
```



Video Demonstration

In the following video, the generation of six-port receivers from 60 GHz to 300 GHz in under one minute is demonstrated.

In [2]:
```
from IPython.display import HTML, display

# the video is streamed from GitHub instead of embedded, which keeps the
notebook small enough for Colab to render
video_url = f"{ASSET_URL}/sparx_gen_60ghz_to_300ghz.mp4"
display(HTML(
    f'<video src="{video_url}" width="1024" controls muted playsinline></video>'
    f'<p>If the video does not play, <a href="{video_url}" target="_blank">open
it here</a>.</p>'
))
```



If the video does not play, open it here.

References

To understand the principle of six-port receivers and their architectures, it is recommended to read the following references:

- S. Dorrer, D. Kellerer-Pirklbauer, G. Zachl, and H. Pretl, "SPARX: An Open-Source, Automated, Programmatically Generated, Frequency-Scalable Six-Port Receiver in 130-nm CMOS," accepted for publication in the 2026 IEEE Nordic Circuits and Systems Conference (NorCAS), 2026. The design, the frequency scaling, and the verification of SPARX are described in this paper, which this notebook accompanies.
- A. Koelpin, G. Vinci, B. Laemmle, D. Kissinger and R. Weigel, "The Six-Port in Modern Society," in IEEE Microwave Magazine, vol. 11, no. 7, pp. 35-43, Dec. 2010, doi: 10.1109/MMM.2010.938584: https://ieeexplore.ieee.org/document/5590352
- T. Hentschel, "The six-port as a communications receiver," in IEEE Transactions on Microwave Theory and Techniques, vol. 53, no. 3, pp. 1039-1047, March 2005, doi: 10.1109/TMTT.2005.843507: https://ieeexplore.ieee.org/document/1406309
- M. Mailand, "System Analysis of Six-Port-Based RF-Receivers," in IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 65, no. 1, pp. 319-330, Jan. 2018, doi: 10.1109/TCSI.2017.2734922: https://ieeexplore.ieee.org/document/8011483

Requirements

To build this six-port receiver, the following tools and their respective dependencies are required:

- GDSFactory: https://github.com/gdsfactory/gdsfactory
- Updated IHP-Open-PDK GDSFactory version: https://github.com/iic-jku/IHP-GDSFactory-Addon
- IHP-Open-PDK: https://github.com/iic-jku/IHP-Open-PDK

The updated IHP-Open-PDK GDSFactory version contains all self-made RF devices and wraps existing PCells provided by the IHP-Open-PDK, allowing them to be used directly within the GDSFactory framework. We choose this approach because it requires very little maintenance. If IHP changes the layout of a cell, no wrapper update is necessary. Only interface changes to a PCell function require updates on our side.

> **Note:** This notebook demonstrates the layout generation of the six-port receiver, a top-level DRC, and the circuit simulation of the power detector and the receiver with VACASK, including the conversion of the EM results into a lumped-element netlist with snp2le. For the full design flow, refer to the official SPARX GitHub repository, which includes S-parameter simulation of the passive RF structures with AWS Palace, a complete LVS, DRC, and RCX verification flow using KLayout, Magic, and Netgen, and SBD-based power detector design in Xschem with ngspice and VACASK simulation. The entire flow is controlled by a Makefile. Simply install the IIC-OSIC-TOOLS container from JKU, clone the repository and run `make all` to build and verify the six-port receiver at 160 GHz, or `make build-layout FREQ=<GHz>` to generate it at another target frequency. The SPARX documentation describes the design, the flow, and every result in detail.

> **Running on Google Colab:** Open the notebook with the badge at the top and select *Runtime → Run all*. All tools are installed by the notebook itself, no local setup is required. If Colab asks to restart the session after the package installation, confirm and select *Runtime → Run all* again.

Installation of IHP-Open-PDK GDSFactory version:

First, our GDSFactory integration of the IHP-Open-PDK is cloned and installed. This IHP-Open-PDK version contains all self-made RF devices and wraps existing PCells provided by the IHP-Open-PDK, allowing them to be used directly within the GDSFactory framework. We choose this approach because it requires very little maintenance. If IHP changes the layout of a cell, no wrapper update is necessary. Only interface changes to a PCell function require updates on our side.

Running `pip install` on the cloned add-on installs the required Python dependencies automatically. The add-on and the IHP-Open-PDK are fetched at the commits this notebook was tested with, `IHP_COMMIT` and `PDK_COMMIT`. To use a newer version, replace the commit hash. The Python packages, among them GDSFactory, are pinned to the versions the notebook was tested with as well.

In [3]:
```
PDK_ROOT = "/tmp/pdks"  # IHP-Open-PDK checkout
IHP_DIR  = "/tmp/ihp"   # IHP-Open-PDK GDSFactory add-on checkout

# the commits this notebook was tested with, of the add-on main branch and the
IHP-Open-PDK dev branch
IHP_COMMIT = "4e5b2960f4b40507bb0ac54e8de3770517f291d3"
PDK_COMMIT = "6f788ad1f64130ced1d8849f5e4a84d7803576c8"

# the add-on reads PDK_ROOT to find the KLayout PCells of the IHP-Open-PDK
```

```
os.environ["PDK_ROOT"] = PDK_ROOT
os.environ["PDK"]      = "ihp-sg13g2"

! rm -rf {IHP_DIR} && git -c init.defaultBranch=main init -q {IHP_DIR}
! git -C {IHP_DIR} fetch -q --depth 1 https://github.com/iic-jku/IHP-GDSFactory-
Addon.git {IHP_COMMIT} && git -C {IHP_DIR} checkout -q FETCH_HEAD
# the Python packages are pinned to the versions this notebook was tested with
%pip install -q {IHP_DIR} gdsfactory==9.51.0 kfactory==3.0.4 klayout==0.30.12
doroutes==0.5.0 scipy==1.18.1 "schemdraw[svgmath]==0.23" ziamath==0.13

# gdsfactory upgrades IPython, but the Colab kernel only starts with its
preinstalled version, which is restored
import sys, IPython
if "google.colab" in sys.modules:
    %pip install -q ipython=={IPython.__version__}
```

```
Note: you may need to restart the kernel to use updated packages.
```

Installation of IHP-Open-PDK:

Next, the IHP-Open-PDK itself is fetched at `PDK_COMMIT` of its `dev` branch and placed in the correct folder. The IHP-Open-PDK provides the technology files required for layout generation. Of its submodules, only the two that the KLayout PCells import are fetched.

In [4]:
```
! rm -rf {PDK_ROOT} && git -c init.defaultBranch=main init -q {PDK_ROOT}
! git -C {PDK_ROOT} fetch -q --depth 1 https://github.com/iic-jku/IHP-Open-
PDK.git {PDK_COMMIT} && git -C {PDK_ROOT} checkout -q FETCH_HEAD
! git -C {PDK_ROOT} submodule update -q --init --depth 1 \
    ihp-sg13g2/libs.tech/klayout/python/pycell4klayout-api \
    ihp-sg13g2/libs.tech/klayout/python/pypreprocessor
```

Import of Python Modules:

GDSFactory builds a layout as a hierarchy of Python objects. A `gf.Component` holds polygons, ports, and references to other components, and `connect()` places a reference so that one of its ports meets a port of another component, which is how every block of this chip is positioned. `ihp` is our GDSFactory add-on, and `ihp.PDK.activate()` makes the layers, cross sections, and cells of IHP SG13G2 the active PDK. The helper `snap_to_grid()` rounds a length down to the 5 nm manufacturing grid, so that computed dimensions such as $\lambda_g/4$ cannot produce off-grid geometry.

In [5]:
```
import gdsfactory as gf
import ihp
from pathlib import Path
import scipy
from math import sqrt

ihp.PDK.activate()


def snap_to_grid(value: float) -> float:
    """Snap a length in um down to the nearest PDK grid point."""
    return round(value - value % ihp.tech.nm, 3)
```

Design Constants:

The device dimensions, routing widths, pad geometry, and layout spacings used throughout the notebook are collected here, identical to the constants of `scripts/six_port_gen.py` in the SPARX repository.

They are grouped by purpose: the MIM capacitor, transistor, and resistor dimensions of the power detector, the routing widths of its supply and signal nets, the probe pad geometry, the Metal5 no-fill zones around the pads, and the spacings that place the detectors, the pads, and the sealring. Unlike the RF devices, these dimensions are fixed and do not scale with `f`.

In [6]:
```
# CMIM capacitor dimensions (um)
CMIM_SIZE = 10  # width and length of standard CMIM cap
CMIM_PITCH = 11.2  # center-to-center pitch in cap arrays
CMIM_SPACING_TM1 = 0.52  # min TM1 spacing between caps (design rule 5.17)
CMIM_FILL_WIDTH = 6.8  # fill capacitor width
CMIM_FILL_LENGTH = 9  # fill capacitor length
CAP_BUS_WIDTH = 4  # metal bus width connecting cap plates

# MOSFET output stage parameters
NUM_GATES = 24  # total gate fingers per transistor (incl. dummies)
NMOS_WIDTH = 60  # nmos finger width (um)
PMOS_WIDTH = 120  # pmos finger width (um)
GATE_LENGTH = 0.13  # drawn gate length (um)
GUARD_RING_DISTANCE = 2  # guard ring distance (um)
GATE_EXTENSION_NMOS = 2.15  # gate poly extension for nmos via connection
GATE_EXTENSION_PMOS = 3.55  # gate poly extension for pmos via connection
NMOS_PMOS_SPACING = 15  # vertical gap between nmos and pmos drain vias

# Resistor dimensions (um)
RSIL_WIDTH = 0.5  # rsil resistor width (XR1, XR5, termination)
RSIL_LENGTH = 2.5  # rsil resistor length
RPPD_WIDTH = 0.5  # rppd resistor width (XR2, XR4)
RPPD_LENGTH = 1.5  # rppd resistor length
RHIGH_WIDTH = 0.5  # rhigh resistor width (XR3)
RHIGH_LENGTH = 2  # rhigh resistor length

# Routing widths (um)
ROUTE_WIDTH_VDD = 8  # VDD power bus (TM1)
ROUTE_WIDTH_VSS = 2  # VSS ground bus (M5)
ROUTE_WIDTH_SIGNAL = 4  # signal lines to probe pads (M3/M4)
ROUTE_WIDTH_MIN_M1 = 0.26  # minimum Metal1 route width
ROUTE_WIDTH_BIAS = 0.5  # bias network routing
ROUTE_WIDTH_BIAS_M1 = 0.46  # bias Metal1 routing
ROUTE_WIDTH_GATE = 0.6  # gate connection routing (M2/M3)
ROUTE_WIDTH_REPLICA = 0.61  # replica circuit drain routing
ROUTE_WIDTH_SBD_SUB = 5.7  # Schottky diode substrate connection
ROUTE_WIDTH_SBD_GUARD = 0.3  # SBD D1-D2 guard ring connection
ROUTE_MIN_STRAIGHT = 10  # minimum straight length to avoid self-routing

# Probe pad parameters (um)
PROBE_PITCH = 125  # probe pad pitch
PROBE_GROUND_WIDTH = 120  # ground pad width (left probe)
PROBE_GROUND_WIDTH_RIGHT = 125  # ground pad width (right probe)
PROBE_SIGNAL_SIZE = 65  # signal pad width and length

# Nofill exclusion zones (um)
NOFILL_GSG_SIZE = 105  # nofill square around GSG probes
```

```
NOFILL_VDD_SIZE = 110  # nofill square around VDD pads
NOFILL_SIDE_WIDTH = 235  # side nofill width (85 + pitch + 25)
NOFILL_SIDE_OFFSET = 12.5  # side nofill centering offset ((110 - 85) / 2)

# Layout spacing (um)
RFIN_GAP = 40  # gap between BLC port and PD rfin
RFIN_PD = 18.4 # part of the connection that is part of the PD
PROBE_PD_GAP = 5  # gap between PD edge and probe edge
SEALRING_MARGIN = 50  # margin around circuit for sealring

# Six-port network geometry
CORNER_Z_FACTOR = 1.314  # impedance scaling for tline corners
CONNECTION_LEN_TERM = 30  # termination connection (before freq_scale, um)
CONNECTION_LEN_BPF_PAD = 30  # BPF-to-pad connection (before freq_scale, um)
CONNECTION_LEN_BLC_PAD = 40  # BLC-to-pad connection (before freq_scale, um)
CONNECTION_LEN_BLC_EXTRA = 25  # extra pad clearance added to BLC-to-pad (um)

# Logo placement
LOGO_VERTICAL_CLEARANCE = 20  # minimal vertical clearance between logo and
probe pads
```

Power Detector Design based on Schottky Barrier Diodes

Circuit Operation:

- **R1** provides a 52 Ω input termination.
- **C1** ac-couples the RF + LO signal to the demodulating SBD **D1** and is implemented as a MIM capacitor.
- **M3 (= 10 M1) – M4 (= 10 M2)** and **R2** form a transimpedance amplifier (TIA) that delivers a buffered voltage output.
- **M1–M2**, **D2 (= D1)**, and **R4 (= R2) – R5 (= R1)** implement a replica bias. With this replica circuit, it is now possible to measure the demodulated output signal differentially.
- **C2-C4** are MIM capacitor arrays that provide proper decoupling of the supply rail and bias voltages (not shown in the schematic).

Layout Considerations:

- Minimize RF input parasitics (R, L, C):
  - Route TopMetal2 directly down to the top plate (TopMetal1) of C1 with a VIA.
  - Use a VIA-stack from the bottom plate (Metal5) of C1 directly down to the SBD.
- Apply **Metal5 no-fill regions** to reduce parasitic capacitance.
- Match components of the main and replica paths.
- **C2–C4** values may be adapted to layout requirements, provided they remain large enough for sufficient supply-rail decoupling.

> In the following code block, the SBD-based power detector (PD) is defined as a standalone `@gf.cell` function. This modular approach enables the same detector design to be reused across all four output channels.

Schematic of the power detector:

The following cell draws the schematic with `schemdraw`. The voltages at the nodes are the simulated DC operating point, and the currents at the bottom are the bias currents of the main branch (4 mA) and the replica branch (400 µA). Every replica device carries the label of the main device it copies, and the main transistors are labeled $10\,M_1$ and $10\,M_2$ because they are ten times the size of the replica transistors M_1 and M_2.

In [7]:
```
# Drawing of SBD-based Power Detector
import matplotlib
matplotlib.rcParams.update({
    "mathtext.fontset": "cm",
    "font.family": "serif",
})
import schemdraw as sd
import schemdraw.elements as elm
sd.svgconfig.svg2 = False

with sd.Drawing() as d:
    d.config(unit=2, fontsize=16)

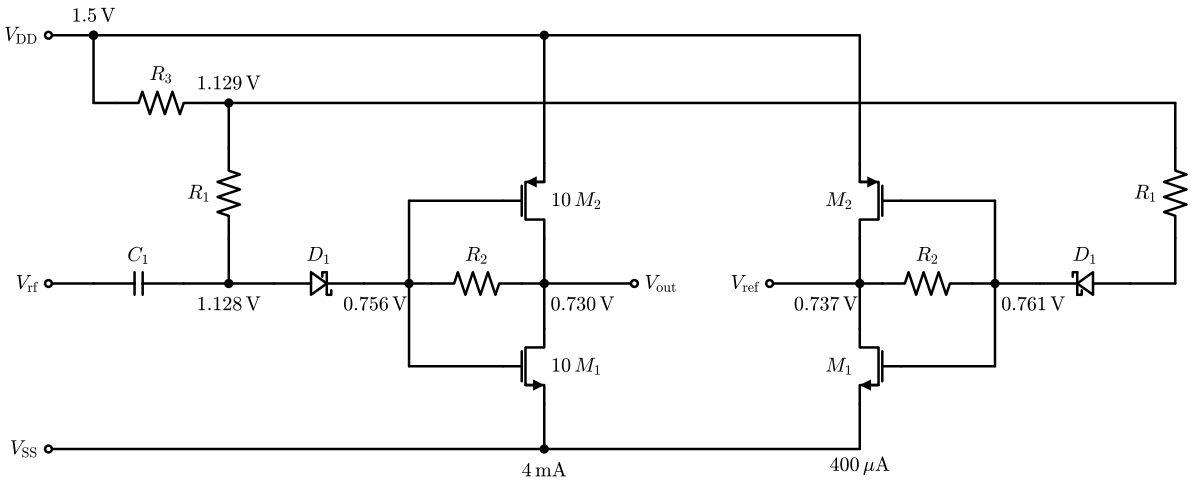
    # Vrf Input, C1, D1
    Vrf = elm.Dot(open=True).label(r'$V_\mathrm{rf}$', loc='left', ofst=0.15)
    elm.Line().right().length(1.0)
    C1 = elm.Capacitor().right().label(r'$C_\mathrm{1}$', loc='top', ofst=0.15)
    elm.Line().right().length(1.0)
    node_rfin_int = elm.Dot().label(r'$1.128\,\mathrm{V}$', loc='bottom',
ofst=0.15)
    elm.Line().right().length(1.0)
    D1 = elm.Schottky().right().label(r'$D_\mathrm{1}$', loc='top', ofst=0.15)
    elm.Line().right().length(1.0)
    node_bb_int = elm.Dot().label(r'$0.756\,\mathrm{V}$', loc='bottom', ofst=
(-0.75, -0.15))

    # R1, R3, VDD
    elm.Line().at(node_rfin_int.end).up().length(1.0)
    R1 = elm.Resistor().up().label(r'$R_\mathrm{1}$', loc='top', ofst=0.15)
    node_bias1 = elm.Line().up().length(1.0).dot().label(r'$1.129\,\mathrm{V}$',
loc='right', ofst=0.15)
    elm.Line().left().length(0.5)
    R3 = elm.Resistor().left().label(r'$R_\mathrm{3}$', loc='top', ofst=0.15)
    elm.Line().left().length(0.5)
    node_vdd = elm.Line().up().length(1.5).dot()
    elm.Dot().label(r'$1.5\,\mathrm{V}$', loc='top', ofst=0.15)
    elm.Line().left().tox(Vrf.start)
    vdd = elm.Dot(open=True).label(r'$V_\mathrm{DD}$', loc='left', ofst=0.15)

    # M3, M4, R2, VSS
    elm.Line().at(node_bb_int.end).right().length(0.5)
    R2 = elm.Resistor().right().label(r'$R_\mathrm{2}$', loc='top', ofst=0.15)
    elm.Line().right().length(0.5)
    node_vout = elm.Dot().label(r'$0.730\,\mathrm{V}$', loc='bottom', ofst=
(0.85, -0.15))
    elm.Line().down().length(1.0)
    M3 =
elm.AnalogNFet(offset_gate=False).anchor('drain').theta(0).label(r'$10\,M_\mathr
m{1}$', loc='right', ofst=0.15).reverse()
    elm.Line().at(M3.source).down().length(1.0)
```

```
    elm.Dot().label(r'$4\,\mathrm{mA}$', loc='bottom', ofst=0.15)
    elm.Line().left().tox(Vrf.start)
    vss = elm.Dot(open=True).label(r'$V_\mathrm{SS}$', loc='left', ofst=0.15)
    elm.Line().at(M3.gate).left().tox(node_bb_int.end)
    elm.Line().up().toy(node_bb_int.end)
    elm.Line().at(node_vout.end).up().length(1.0)
    M4 =
elm.AnalogPFet(offset_gate=False).anchor('drain').theta(0).label(r'$10\,M_\mathr
m{2}$', loc='right', ofst=0.15).reverse()
    elm.Line().at(M4.source).up().toy(node_vdd.end).dot()
    elm.Line().left().tox(node_vdd.end)
    elm.Line().at(M4.gate).left().tox(node_bb_int.end)
    elm.Line().down().toy(node_bb_int.end)
    elm.Line().at(node_vout.end).right().length(2.0)
    Vout = elm.Dot(open=True).label(r'$V_\mathrm{out}$', loc='right',
ofst=0.15)

    # Replica Circuit: R5, D2, R4, M1, M2
    elm.Line().at(node_bias1.end).right().length(21.0)
    elm.Line().down().length(1.0)
    R5 = elm.Resistor().down().label(r'$R_\mathrm{1}$', loc='top', ofst=0.15)
    elm.Line().down().length(1.0)
    elm.Line().left().length(1.0)
    D2 = elm.Schottky().left().label(r'$D_\mathrm{1}$', loc='top', ofst=0.15)
    elm.Line().left().length(1.0)
    node_bias2 = elm.Dot().label(r'$0.761\,\mathrm{V}$', loc='bottom', ofst=
(0.85, -0.15))
    elm.Line().at(node_bias2.end).left().length(0.5)
    R4 = elm.Resistor().left().label(r'$R_\mathrm{2}$', loc='top', ofst=0.15)
    elm.Line().left().length(0.5)
    node_vref = elm.Dot().label(r'$0.737\,\mathrm{V}$', loc='bottom', ofst=
(-0.75, -0.15))
    elm.Line().down().length(1.0)
    M1 =
elm.AnalogNFet(offset_gate=False).anchor('drain').theta(0).label(r'$M_\mathrm{1}
$', loc='left', ofst=0.15)
    elm.Line().at(M1.source).down().toy(vss.end).label(r'$400\,\mu\mathrm{A}$',
loc='left', ofst=0.15)
    elm.Line().left().tox(node_vout.end)
    elm.Line().at(M1.gate).right().tox(node_bias2.end)
    elm.Line().up().toy(node_bias2.end)
    elm.Line().at(node_vref.end).up().length(1.0)
    M2 =
elm.AnalogPFet(offset_gate=False).anchor('drain').theta(0).label(r'$M_\mathrm{2}
$', loc='left', ofst=0.15)
    elm.Line().at(M2.source).up().toy(node_vdd.end)
    elm.Line().left().tox(node_vout.end)
    elm.Line().at(M2.gate).right().tox(node_bias2.end)
    elm.Line().down().toy(node_bias2.end)
    elm.Line().at(node_vref.end).left().length(2.0)
    Vref = elm.Dot(open=True).label(r'$V_\mathrm{ref}$', loc='left', ofst=0.15)
```



Layout of the power detector:

The next cell defines `powdet_sbd()`, which draws the complete detector as one GDSFactory cell. The `@gf.cell` decorator caches the result, so the four detectors on the chip are four references to the same cell. The function builds the detector from these parts:

1. **RF input:** a via stack from the TopMetal2 input down to the top plate of the MIM capacitor C_1 on TopMetal1, and a second via stack from its bottom plate on Metal5 down to the Schottky diode D_1, the shortest path with the least parasitics. The replica diode D_2 is placed right next to D_1 for matching.
2. **Supply decoupling C_2:** a 3 x 4 array of 10 µm x 10 µm MIM capacitors with one position left out for C_1, whose plates are connected row by row.
3. **Input and replica termination R_1:** two silicided poly resistors (`rsil`), routed to D_1 and D_2 on Metal1 and Metal2.
4. **Transimpedance amplifier:** one NMOS and one PMOS transistor with 24 gate fingers each. 20 fingers form the main amplifier, 2 the replica, and the 2 outermost fingers are grounded dummies. The sources connect to VSS and VDD, and the drains are collected through via stacks.
5. **Feedback resistors R_2:** p+ poly resistors (`rppd`) between the diode nodes and the outputs `vout` and `vref` .
6. **Bias network:** a second MIM array for the bias decoupling, MIM fill capacitors, and the bias resistor R_3 (`rhigh`) from VDD, with the routing of the bias nodes to both diodes and the substrate connection of the two diodes.
7. **Ports and no-fill zones:** the TopMetal2 input line `RFIN_PD` with the port `rfin` , and the ports `vout` , `vref` , `vdd` , and `vss` that the top level connects to. No-fill regions on each metal layer keep the metal fill out of the detector, and the cell is finally snapped to the grid.

In [8]:
```python
@gf.cell
def powdet_sbd() -> gf.Component:
    """Schottky-diode power detector with transimpedance amplifier.

    Ports: rfin (TM2), vout (M4), vref (M4), vss (M5), vdd (TM1).
    """
    c = gf.Component("powdet_sbd")
```

```
    # via from rfin on TM2 to C1 on TM1
    via_tm1_tm2 = ihp.cells.via_stack(
        top_layer="TopMetal2",
        bottom_layer="TopMetal1",
        vt2_columns=4,
        vt2_rows=3,
    )

    via_tm1_tm2.ports["bottom"].orientation = 0
    via_tm1_tm2_ref = c.add_ref(via_tm1_tm2)
    # reference cmim capacitor and connect to via
    cmim = ihp.cells.cmim(width=CMIM_SIZE, length=CMIM_SIZE)
    c1_ref = c.add_ref(cmim)
    c1_ref.connect("T", via_tm1_tm2_ref.ports["bottom"],
allow_width_mismatch=True)


    # via cell from Metal2 to Metal5
    via_m2_m5 = ihp.cells.via_stack(
        top_layer="Metal5",
        bottom_layer="Metal2",
        vn_columns=6,
        vn_rows=3,
    )

    # via from CMIM C1 to XQ1
    via_m2_m5.ports["top"].orientation = 0
    via_m2_m5_ref = c.add_ref(via_m2_m5)
    via_m2_m5_ref.connect("top", c1_ref.ports["B"], allow_width_mismatch=True)

    # create D1 and connect to rfin via C1 and the vias
    schottky = ihp.cells.schottky()
    schottky.locked = False
    schottky.add_label(text="schottky_nbl1", layer=ihp.tech.LAYER.TEXTdrawing)

    d1_ref = c.add_ref(schottky)
    d1_ref.connect("e4", via_m2_m5_ref.ports["bottom"],
allow_width_mismatch=True, allow_layer_mismatch=True)

    # Create D2 nearby D1 for better matching
    d2_ref = c.add_ref(schottky)
    d2_ref.ymin = d1_ref.ymin
    d2_ref.xmin = d1_ref.xmax  # min spacing according to design rules 5.10

    # Create capacitor array for c2
    c2 = gf.Component("C2")
    cmim = ihp.cells.cmim(width=CMIM_SIZE, length=CMIM_SIZE)
    cap_connection_thickness = CAP_BUS_WIDTH  # thickness of the metal
connection between caps
    # create 3 * 4 array of cmim capacitors with column pitch of 0.52 + 11.2 to
fit the required spacing for TM1 and the size of the caps
    # with one missing cap to fit C1 in there
    spacing_between_caps = CMIM_SPACING_TM1
    top_row = c2.add_ref(
        gf.components.array(component=cmim, columns=6, rows=1,
column_pitch=spacing_between_caps + CMIM_PITCH)
    )

    middle_row = c2.add_ref(
        gf.components.array(component=cmim, columns=6, rows=1,
```

```
column_pitch=spacing_between_caps + CMIM_PITCH)
    )
    bottom_row = c2.add_ref(
        gf.components.array(
            component=cmim,
            columns=6,
            rows=3,
            column_pitch=spacing_between_caps + CMIM_PITCH,
            row_pitch=-(spacing_between_caps + CMIM_PITCH),
        )
    )
    top_row.ymin = middle_row.ymax + spacing_between_caps  # min spacing
according to design rules 5.17
    bottom_row.ymax = middle_row.ymin - spacing_between_caps  # min spacing
according to design rules 5.17
    c2.add_ports(top_row.ports, prefix="TR_")
    c2.add_ports(middle_row.ports, prefix="MR_")
    c2.add_ports(bottom_row.ports, prefix="BR_")


    # Orient outermost ports inward for horizontal cap-to-cap connection
    for prefix in [
        "TR_T_1_",
        "TR_B_1_",
        "MR_T_1_",
        "MR_B_1_",
        "BR_T_1_",
        "BR_B_1_",
        "BR_T_2_",
        "BR_B_2_",
        "BR_T_3_",
        "BR_B_3_",
    ]:
        c2.ports[f"{prefix}1"].orientation = 0
        c2.ports[f"{prefix}6"].orientation = 180

    c2_connection_t = gf.routing.route_bundle_electrical(
        component=c2,
        ports1=[
            c2.ports["TR_T_1_1"],
            c2.ports["BR_T_2_1"],
            c2.ports["BR_T_3_1"],
            c2.ports["MR_T_1_1"],
            c2.ports["BR_T_1_1"],
        ],
        ports2=[
            c2.ports["TR_T_1_6"],
            c2.ports["BR_T_2_6"],
            c2.ports["BR_T_3_6"],
            c2.ports["MR_T_1_6"],
            c2.ports["BR_T_1_6"],
        ],
        route_width=cap_connection_thickness,
        layer=ihp.tech.LAYER.TopMetal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )
```

```
    c2_connection_b = gf.routing.route_bundle_electrical(
        component=c2,
        ports1=[
            c2.ports["TR_B_1_1"],
            c2.ports["BR_B_2_1"],
            c2.ports["BR_B_3_1"],
            c2.ports["MR_B_1_1"],
            c2.ports["BR_B_1_1"],
        ],
        ports2=[
            c2.ports["TR_B_1_6"],
            c2.ports["BR_B_2_6"],
            c2.ports["BR_B_3_6"],
            c2.ports["MR_B_1_6"],
            c2.ports["BR_B_1_6"],
        ],
        route_width=cap_connection_thickness,
        layer=ihp.tech.LAYER.Metal5drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    # orient top row ports to face downwards to connect to bottom row ports
    ihp.cells.utils.change_port_orientation(
        c2, ["TR_T_1_1", "TR_T_1_2", "TR_T_1_3", "TR_T_1_4", "TR_T_1_5",
"TR_T_1_6"], 270
    )
    ihp.cells.utils.change_port_orientation(
        c2, ["TR_B_1_1", "TR_B_1_2", "TR_B_1_3", "TR_B_1_4", "TR_B_1_5",
"TR_B_1_6"], 270
    )

    # orient bottom row ports to face downwards to connect to top row ports
    ihp.cells.utils.change_port_orientation(
        c2, ["BR_T_3_1", "BR_T_3_2", "BR_T_3_3", "BR_T_3_4", "BR_T_3_5",
"BR_T_3_6"], 90
    )
    ihp.cells.utils.change_port_orientation(
        c2, ["BR_B_3_1", "BR_B_3_2", "BR_B_3_3", "BR_B_3_4", "BR_B_3_5",
"BR_B_3_6"], 90
    )

    c2_connection_top = gf.routing.route_bundle_electrical(
        component=c2,
        ports1=[
            c2.ports["BR_T_3_1"],
            c2.ports["BR_T_3_2"],
            c2.ports["BR_T_3_3"],
            c2.ports["BR_T_3_4"],
            c2.ports["BR_T_3_5"],
            c2.ports["BR_T_3_6"],
        ],
        ports2=[
            c2.ports["TR_T_1_1"],
            c2.ports["TR_T_1_2"],
            c2.ports["TR_T_1_3"],
            c2.ports["TR_T_1_4"],
            c2.ports["TR_T_1_5"],
```

```
            c2.ports["TR_T_1_6"],
        ],
        route_width=cap_connection_thickness,  # adjust for thicker connections
        layer=ihp.tech.LAYER.TopMetal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    c2_connection_bottom = gf.routing.route_bundle_electrical(
        component=c2,
        ports1=[
            c2.ports["BR_B_3_1"],
            c2.ports["BR_B_3_2"],
            c2.ports["BR_B_3_3"],
            c2.ports["BR_B_3_4"],
            c2.ports["BR_B_3_5"],
            c2.ports["BR_B_3_6"],
        ],
        ports2=[
            c2.ports["TR_B_1_1"],
            c2.ports["TR_B_1_2"],
            c2.ports["TR_B_1_3"],
            c2.ports["TR_B_1_4"],
            c2.ports["TR_B_1_5"],
            c2.ports["TR_B_1_6"],
        ],
        route_width=cap_connection_thickness,  # adjust for thicker connections
        layer=ihp.tech.LAYER.Metal5drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    # orient top row ports to face upwards to connect the ground plates to the
other caps
    ihp.cells.utils.change_port_orientation(c2, ["TR_B_1_1", "TR_B_1_2",
"TR_B_1_3", "TR_B_1_6"], 90)
    ihp.cells.utils.change_port_orientation(c2, ["TR_T_1_6"], 0)  # for
connection to bias1
    ihp.cells.utils.change_port_orientation(c2, ["BR_B_3_4", "BR_B_3_5"], 270)

    c2_ref = c.add_ref(c2)
    c2_ref.rotate(-90)
    c2_ref.center = c1_ref.center
    c2_ref.ymin = c1_ref.ymax + 10
    c2_ref.xmax = c1_ref.xmax + c1_ref.xsize + CMIM_SPACING_TM1

    ## end of c2 generation

    r_sil = ihp.cells.rsil(width=RSIL_WIDTH, length=RSIL_LENGTH)

    # let the ports of xr1 and xr5 face each other for easier routing
    r_sil.ports["e2"].orientation = 0
    xr1_ref = c.add_ref(r_sil.copy())

    r_sil.ports["e2"].orientation = 180
    xr5 = r_sil.copy()
```

```
    xr5_ref = c.add_ref(xr5)

    # spacing between D1 and xr1/xr5
    spacing = 1
    xr1_ref.xmax = d1_ref.xmax - 1.88
    xr1_ref.ymax = d1_ref.ymin - spacing

    xr5_ref.xmin = d2_ref.xmin + 1.88
    xr5_ref.ymin = xr1_ref.ymin

    route_xr1_xr5 = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[xr1_ref.ports["e2"]],
        ports2=[xr5_ref.ports["e2"]],
        route_width=ROUTE_WIDTH_MIN_M1,
        layer=ihp.tech.LAYER.Metal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    # via for connection D1 to xr1 and xr5 to D2
    via_m1_m2 = ihp.cells.via_stack(
        top_layer="Metal2",
        bottom_layer="Metal1",
        vn_columns=6,
        vn_rows=2,
    )

    # via D1 -> xr1
    via_m1_m2.ports["top"].orientation = 90
    via_m1_m2.ports["bottom"].orientation = 0
    via_m1_m2_ref = c.add_ref(via_m1_m2.copy())
    via_m1_m2_ref.center = d1_ref.center
    via_m1_m2_ref.ymax = d1_ref.ymin

    # route from D1 to via
    route = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[d1_ref.ports["e4"]],
        ports2=[via_m1_m2_ref.ports["top"]],
        route_width=max(via_m1_m2_ref.xsize, via_m1_m2_ref.ysize),
        layer=ihp.tech.LAYER.Metal2drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    # route from via to xr1
    route = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[via_m1_m2_ref.ports["bottom"]],
        ports2=[xr1_ref.ports["e1"]],
        route_width=min(via_m1_m2_ref.xsize, via_m1_m2_ref.ysize),
        layer=ihp.tech.LAYER.Metal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
```

```
    separation=0,
)

# connect second diode to input resistors
# via D2 -> xr5
via_m1_m2.ports["top"].orientation = 90
via_m1_m2.ports["bottom"].orientation = 180
via_m1_m2_ref = c.add_ref(via_m1_m2.copy())
via_m1_m2_ref.center = d2_ref.center
via_m1_m2_ref.ymax = d2_ref.ymin

# route from D2 to via
route = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[d2_ref.ports["e4"]],
    ports2=[via_m1_m2_ref.ports["top"]],
    route_width=max(via_m1_m2_ref.xsize, via_m1_m2_ref.ysize),
    layer=ihp.tech.LAYER.Metal2drawing,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

# route from via to xr5
route = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[via_m1_m2_ref.ports["bottom"]],
    ports2=[xr5_ref.ports["e1"]],
    route_width=min(via_m1_m2_ref.xsize, via_m1_m2_ref.ysize),
    layer=ihp.tech.LAYER.Metal1drawing,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

## Build transimpedance amplifier with Schottky diode as input device.
# M1 (nmos, 20 gates) + M3 (nmos, 2 gates) + 2 dummy gates = 24
# M2 (pmos, 20 gates) + M4 (pmos, 2 gates) + 2 dummy gates = 24
ng_mos = NUM_GATES
width_nmos = NMOS_WIDTH  # um
width_pmos = PMOS_WIDTH  # um

c_output = gf.Component("output_stage")

m1_3 = ihp.cells.nmos(
    w=width_nmos,
    l=GATE_LENGTH,
    ng=ng_mos,
    guardRingType="psub",
    guardRingDistance=GUARD_RING_DISTANCE,
)

via_gat_m1 = ihp.cells.via_stack(
    top_layer="Metal1",
    bottom_layer="GatPoly",
    vn_columns=1,
    vn_rows=2,
)
```

```
    via_gat_m2 = ihp.cells.via_stack(
        top_layer="Metal2",
        bottom_layer="GatPoly",
        vn_columns=1,
        vn_rows=3,
    )

    via_gat_m3 = ihp.cells.via_stack(
        top_layer="Metal3",
        bottom_layer="GatPoly",
        vn_columns=1,
        vn_rows=3,
    )

    m1_3.locked = False
    via_gat_m1.ports["bottom"].orientation = 90
    via_gat_m2.ports["bottom"].orientation = 90
    via_gat_m3.ports["bottom"].orientation = 90
    via_gat_m3.ports["top"].orientation = 270  # turn top port for later
connection
    gate_ports = m1_3.get_ports_list(layer=ihp.tech.LAYER.GatPolydrawing)
    gate_ports.sort(key=lambda p: p.center[0])  # left to right, so list indices
match the G_ numbering
    gate_straight = ihp.cells.straight(
        length=GATE_EXTENSION_NMOS, cross_section="gatpoly_routing",
width=GATE_LENGTH  # gate poly width
    )
    gate_via_refs_nmos = []
    for i, p in enumerate(gate_ports):
        if p.name in ["G_1", "G_24"]:  # these gates will be connected to metal1
            gate_via_refs_nmos.append(m1_3.add_ref(via_gat_m2))
        elif p.name in ["G_12", "G_13"]:  # these gates will be connected to
metal2
            gate_via_refs_nmos.append(m1_3.add_ref(via_gat_m2))
        else:  # all others to metal3
            gate_via_refs_nmos.append(m1_3.add_ref(via_gat_m3))
        gate_straight_ref = m1_3.add_ref(gate_straight)
        gate_straight_ref.connect("e1", m1_3.ports[p.name],
allow_width_mismatch=True, allow_layer_mismatch=True)
        via_gat_m2_ref = gate_via_refs_nmos[-1]
        via_gat_m2_ref.connect(
            "bottom", gate_straight_ref.ports["e2"], allow_width_mismatch=True,
allow_layer_mismatch=True
        )

    # connect gates 1 and 24 with metal1 to ground
    via_gat_m2.ports["top"].orientation = 0
    start_point = gate_via_refs_nmos[23].ports["top"].center[0]  # G_24's via
    end_point = m1_3.ports["DS_29"].center[0]  # right guard-ring tap
    straight_ref = m1_3.add_ref(
        ihp.cells.straight(
            length=abs(end_point - start_point),
            cross_section="metal2_routing",
            width=1.11,  # manual measure of the height of the via
        )
    )
    straight_ref.connect("e1", gate_via_refs_nmos[23].ports["top"],
allow_width_mismatch=True, allow_layer_mismatch=True)
    via_m1_m2 = ihp.cells.via_stack(
```

```
        top_layer="Metal2",
        bottom_layer="Metal1",
        vn_columns=3,
        vn_rows=1,
    )
    via_m1_m2_ref = m1_3.add_ref(via_m1_m2)
    via_m1_m2_ref.connect("top", straight_ref.ports["e2"],
allow_width_mismatch=True, allow_layer_mismatch=True)

    via_gat_m2.ports["top"].orientation = 180
    straight_ref = m1_3.add_ref(
        ihp.cells.straight(
            length=abs(end_point - start_point),
            cross_section="metal2_routing",
            width=1.11,  # manual measure of the height of the via
        )
    )
    straight_ref.connect(
        "e1", gate_via_refs_nmos[0].ports["top"], allow_width_mismatch=True,
allow_layer_mismatch=True
    )
    via_m1_m2_ref = m1_3.add_ref(via_m1_m2)
    via_m1_m2_ref.connect("top", straight_ref.ports["e2"],
allow_width_mismatch=True, allow_layer_mismatch=True)

    # connect Gates 2-11 and 14-23 (all 20 gates) together with metal3
    start_point = m1_3.ports["G_2"].center
    end_point = m1_3.ports["G_23"].center
    straight_ref = m1_3.add_ref(
        ihp.cells.straight(
            length=abs(end_point[0] - start_point[0]),
            cross_section="metal3_routing",
            width=max(
                gate_via_refs_nmos[1].xsize, gate_via_refs_nmos[1].ysize
            ),  # manual measure of the height of the via
        )
    )
    straight_ref.center = (
        (start_point[0] + end_point[0]) / 2,
        gate_via_refs_nmos[1].center[1],
    )  # align with the via of the first M1 gate

    # connect Gates 12-13 (2 gates) together with metal2
    start_point = m1_3.ports["G_12"].center
    end_point = m1_3.ports["G_13"].center
    straight_ref = m1_3.add_ref(
        ihp.cells.straight(
            length=abs(end_point[0] - start_point[0]),
            cross_section="metal2_routing",
            width=0.6,  # manual measure of the height of the via
        )
    )
    straight_ref.center = (
        (start_point[0] + end_point[0]) / 2,
        gate_via_refs_nmos[11].center[1],
    )  # align with the via of the first M3 gate

    # connect source to GND
    # Source and drain fingers by their deterministic left-to-right port names
    # (DS_ ports are renumbered by position in the IHP PDK, so these lists are
```

```
    # stable across tool versions). DS_15 is the center column that also holds
    # the shortened drain via stack.
    m1_3_source_ports = [m1_3.ports[f"DS_{i}"] for i in (2, 3, 5, 7, 9, 11, 13,
17, 19, 21, 23, 25, 27, 28)]
    m1_3_drain_ports = [m1_3.ports[f"DS_{i}"] for i in (4, 6, 8, 10, 12, 15,
18, 20, 22, 24, 26)]

    # source ground connection
    source_straight = ihp.cells.straight(
        length=3.28,  # source finger to the center of the top guardring segment
        # (was |DS_1.y - DS_27.y| under the old port enumeration; those
        # guard-rail pin ports are no longer emitted as ports)
        cross_section="metal2_routing",
        width=0.26,  # manual measure of the height of the via
    )
    via_m1_m2 = ihp.cells.via_stack(
        top_layer="Metal2",
        bottom_layer="Metal1",
        vn_columns=7,
        vn_rows=1,
    )
    via_m1_m2.ports["bottom"].orientation = 0
    via_m1_m2.ports["top"].orientation = 180
    for p in m1_3_source_ports:
        p.orientation = 90  # orient ports to face the top
        via_m1_m2_ref = m1_3.add_ref(via_m1_m2)
        via_m1_m2_ref.connect("bottom", p, allow_width_mismatch=True,
allow_layer_mismatch=True)
        source_straight_ref = m1_3.add_ref(source_straight)
        source_straight_ref.connect(
            "e1", via_m1_m2_ref.ports["top"], allow_width_mismatch=True,
allow_layer_mismatch=True
        )

    # connect drains to M3
    via_m1_m4 = ihp.cells.via_stack(
        top_layer="Metal4",
        bottom_layer="Metal1",
        vn_columns=1,
        vn_rows=7,
    )

    drain_via_refs_nmos = []
    for p in m1_3_drain_ports:
        p.orientation = 90  # orient ports to face the top
        if p.name in ["DS_15"]:  # center drain column (x = 6.27), gets the
shortened stack
            via_m1_m4_short = ihp.cells.via_stack(
                top_layer="Metal4",
                bottom_layer="Metal1",
                vn_columns=1,
                vn_rows=3,
            )
            via_m1_m4_short.ports["top"].orientation = 270
            drain_via_refs_nmos.append(m1_3.add_ref(via_m1_m4_short))
            drain_via_refs_nmos[-1].center = p.center
            drain_via_refs_nmos[-1].ymax = p.center[1] - 0.245  # manual
measurement to align with metal1 below
        else:
            via_m1_m4.ports["bottom"].orientation = 90
```

```
            via_m1_m4.ports["top"].orientation = 90
            via_m1_m4_ref = m1_3.add_ref(via_m1_m4)
            via_m1_m4_ref.connect("bottom", p, allow_width_mismatch=True,
allow_layer_mismatch=True)
            drain_via_refs_nmos.append(via_m1_m4_ref)

    # horizontal drain connection for nmos
    straight_drain_connection = ihp.cells.straight(
        length=abs(
            drain_via_refs_nmos[0].center[0] - drain_via_refs_nmos[-1].center[0]
        ),  # span from the first to the last drain via
        cross_section="metal4_routing",
        width=2,  # manual measure of the height of the via
    )
    straight_drain_connection_ref = m1_3.add_ref(straight_drain_connection)
    straight_drain_connection_ref.xmin = drain_via_refs_nmos[0].center[0]  #
leftmost drain via
    straight_drain_connection_ref.ymin = drain_via_refs_nmos[0].center[1]

    m1_3.add_ports(gate_via_refs_nmos[6].ports, prefix="gate_M1_")  # G_7's via
    m1_3.ports["DS_16"].orientation = 90  # so the top-level vdd via stack
connects unrotated
    m1_3.add_port(name="drain_connection_M1",
port=drain_via_refs_nmos[8].ports["top"])  # DS_22 column (x = 9.33)
    m1_3.add_port(name="drain_connection_M3",
port=drain_via_refs_nmos[5].ports["top"])
    m1_3.add_port(
        name="gate_connection_M3",
        center=(
            (gate_via_refs_nmos[11].center[0] +
gate_via_refs_nmos[12].center[0]) / 2,
            gate_via_refs_nmos[11].center[1],
        ),
        cross_section="metal2_routing",
        orientation=270,
        port_type="electrical",
    )
    m1_3_ref = c_output.add_ref(m1_3)
    m1_3_ref.move((0, 30))

    m2_4 = ihp.cells.pmos(
        w=width_pmos,
        l=GATE_LENGTH,
        ng=ng_mos,
        guardRingType="nwell",
        guardRingDistance=GUARD_RING_DISTANCE,
    )

    # connect gates of M2 and M4 together

    m2_4.locked = False
    gate_ports = m2_4.get_ports_list(layer=ihp.tech.LAYER.GatPolydrawing)
    gate_ports.sort(key=lambda p: p.center[0])  # left to right, so list indices
match the G_ numbering
    gate_straight = ihp.cells.straight(
        length=GATE_EXTENSION_PMOS, cross_section="gatpoly_routing",
width=GATE_LENGTH  # gate poly width
    )
    gate_via_refs_pmos = []
    for i, p in enumerate(gate_ports):
```

```
        p.orientation += 180  # orient gates to face downwards for easier
connection to nmos gates
        if p.name in ["G_1", "G_24"]:  # these gates will be connected to metal1
            gate_via_refs_pmos.append(m2_4.add_ref(via_gat_m2))
        elif p.name in ["G_12", "G_13"]:  # these gates will be connected to
metal2
            gate_via_refs_pmos.append(m2_4.add_ref(via_gat_m2))
        else:  # all others to metal3
            gate_via_refs_pmos.append(m2_4.add_ref(via_gat_m3))
        gate_straight_ref = m2_4.add_ref(gate_straight)
        gate_straight_ref.connect("e1", m2_4.ports[p.name],
allow_width_mismatch=True, allow_layer_mismatch=True)
        gate_via_refs_pmos[-1].connect(
            "bottom", gate_straight_ref.ports["e2"], allow_width_mismatch=True,
allow_layer_mismatch=True
        )

    # connect gates 1 and 24 with metal1 to ground
    via_gat_m2.ports["top"].orientation = 180
    start_point = gate_via_refs_pmos[23].ports["top"].center[0]  # G_24's via
    end_point = m2_4.ports["DS_29"].center[0]  # right guard-ring tap
    straight_ref = m2_4.add_ref(
        ihp.cells.straight(
            length=abs(end_point - start_point),
            cross_section="metal2_routing",
            width=1.11,  # manual measure of the height of the via
        )
    )

    straight_ref.connect("e1", gate_via_refs_pmos[23].ports["top"],
allow_width_mismatch=True, allow_layer_mismatch=True)
    via_m1_m2 = ihp.cells.via_stack(
        top_layer="Metal2",
        bottom_layer="Metal1",
        vn_columns=3,
        vn_rows=1,
    )
    via_m1_m2_ref = m2_4.add_ref(via_m1_m2)
    via_m1_m2_ref.connect("top", straight_ref.ports["e2"],
allow_width_mismatch=True, allow_layer_mismatch=True)

    straight_ref.connect("e2", via_m1_m2_ref.ports["top"],
allow_width_mismatch=True, allow_layer_mismatch=True)

    via_gat_m2.ports["bottom"].orientation = 0
    straight_ref.connect("e1", gate_via_refs_pmos[23].ports["top"],
allow_width_mismatch=True, allow_layer_mismatch=True)
    straight_ref = m2_4.add_ref(
        ihp.cells.straight(
            length=abs(end_point - start_point),
            cross_section="metal2_routing",
            width=1.11,  # manual measure of the height of the via
        )
    )
    straight_ref.connect(
        "e1", gate_via_refs_pmos[0].ports["bottom"], allow_width_mismatch=True,
allow_layer_mismatch=True
    )

    via_m1_m2_ref = m2_4.add_ref(via_m1_m2)
```

```
    via_m1_m2_ref.connect("top", straight_ref.ports["e2"],
allow_width_mismatch=True, allow_layer_mismatch=True)

    # connect Gates 2-11 and 14-23 (20 gates) together with metal3
    start_point = m2_4.ports["G_2"].center
    end_point = m2_4.ports["G_23"].center
    straight_ref = m2_4.add_ref(
        ihp.cells.straight(
            length=abs(end_point[0] - start_point[0]),
            cross_section="metal3_routing",
            width=max(
                gate_via_refs_pmos[1].xsize, gate_via_refs_pmos[1].ysize
            ),  # manual measure of the height of the via
        )
    )
    straight_ref.center = (
        (start_point[0] + end_point[0]) / 2,
        gate_via_refs_pmos[1].center[1],
    )  # align with the via of the first M1 gate

    # connect Gates 12-13 (2 gates) together with metal2
    start_point = m2_4.ports["G_12"].center
    end_point = m2_4.ports["G_13"].center
    straight_ref = m2_4.add_ref(
        ihp.cells.straight(
            length=abs(end_point[0] - start_point[0]),
            cross_section="metal2_routing",
            width=0.6,  # manual measure of the height of the via
        )
    )
    straight_ref.center = (
        (start_point[0] + end_point[0]) / 2,
        gate_via_refs_pmos[11].center[1],
    )  # align with the via of the first M3 gate

    # connect source to vdd
    # Source and drain fingers by their deterministic left-to-right port names
    # (same column assignment as the nmos above).
    m2_4_source_ports = [m2_4.ports[f"DS_{i}"] for i in (2, 3, 5, 7, 9, 11, 13,
17, 19, 21, 23, 25, 27, 28)]
    m2_4_drain_ports = [m2_4.ports[f"DS_{i}"] for i in (4, 6, 8, 10, 12, 15,
18, 20, 22, 24, 26)]

    # source vdd connection
    source_straight = ihp.cells.straight(
        length=4.66,  # source finger to the center of the top guardring segment
        # (was |DS_1.y - DS_29.y| under the old port enumeration; those
        # guard-rail pin ports are no longer emitted as ports)
        cross_section="metal2_routing",
        width=0.26,  # manual measure of the height of the via
    )
    via_m1_m2 = ihp.cells.via_stack(
        top_layer="Metal2",
        bottom_layer="Metal1",
        vn_columns=13,
        vn_rows=1,
    )
    via_m1_m2.ports["bottom"].orientation = 180
    via_m1_m2.ports["top"].orientation = 0
    for p in m2_4_source_ports:
```

```
        p.orientation = 270  # orient ports to face the top
        via_m1_m2_ref = m2_4.add_ref(via_m1_m2)
        via_m1_m2_ref.connect("bottom", p, allow_width_mismatch=True,
allow_layer_mismatch=True)
        source_straight_ref = m2_4.add_ref(source_straight)
        source_straight_ref.connect(
            "e1", via_m1_m2_ref.ports["top"], allow_width_mismatch=True,
allow_layer_mismatch=True
        )

    # connect drains to metal4
    via_m1_m4 = ihp.cells.via_stack(
        top_layer="Metal4",
        bottom_layer="Metal1",
        vn_columns=1,
        vn_rows=13,
    )
    drain_via_refs_pmos = []
    for p in m2_4_drain_ports:
        p.orientation = 90  # orient ports to face the top
        if p.name in ["DS_15"]:  # center drain column (x = 6.27), gets the
shortened stack
            via_m1_m4_short = ihp.cells.via_stack(
                top_layer="Metal4",
                bottom_layer="Metal1",
                vn_columns=1,
                vn_rows=6,
            )
            via_m1_m4_short.ports["top"].orientation = 90
            drain_via_refs_pmos.append(m2_4.add_ref(via_m1_m4_short))
            drain_via_refs_pmos[-1].center = p.center
            drain_via_refs_pmos[-1].ymin = p.center[1] + 0.245  # manual
measurement to align with metal1 below
        else:
            via_m1_m4.ports["bottom"].orientation = 90
            via_m1_m4.ports["top"].orientation = 270
            via_m1_m4_ref = m2_4.add_ref(via_m1_m4)
            via_m1_m4_ref.connect("bottom", p, allow_width_mismatch=True,
allow_layer_mismatch=True)
            drain_via_refs_pmos.append(via_m1_m4_ref)

    # horizontal drain connection for pmos
    straight_drain_connection = ihp.cells.straight(
        length=abs(
            drain_via_refs_pmos[0].center[0] - drain_via_refs_pmos[-1].center[0]
        ),  # span from the first to the last drain via
        cross_section="metal4_routing",
        width=2.61,  #  manual measurement
    )
    straight_drain_connection_ref = m2_4.add_ref(straight_drain_connection)
    straight_drain_connection_ref.xmin = drain_via_refs_pmos[0].center[0]  #
leftmost drain via
    straight_drain_connection_ref.ymin = drain_via_refs_pmos[0].ymin

    m2_4.add_ports(gate_via_refs_pmos[6].ports, prefix="gate_M2_")  # G_7's via
    m2_4.add_port(name="drain_connection_M2",
port=drain_via_refs_pmos[8].ports["top"])  # DS_22 column (x = 9.33)
    m2_4.add_port(name="drain_connection_M4",
port=drain_via_refs_pmos[5].ports["top"])
    m2_4.add_port(
```

```
        name="gate_connection_M4",
        center=(
            (gate_via_refs_pmos[11].center[0] +
gate_via_refs_pmos[12].center[0]) / 2,
            gate_via_refs_pmos[11].center[1],
        ),
        cross_section="metal2_routing",
        orientation=90,
        port_type="electrical",
    )

    m2_4_ref = c_output.add_ref(m2_4)

    spacing_between_nmos_pmos = NMOS_PMOS_SPACING  # manual measurement of the
spacing between the drain vias of M1 and M2, used to set the length of the
connection between M1 and M2 drains
    drain_connection_m1_m2 = c_output.add_ref(
        ihp.cells.straight(
            length=spacing_between_nmos_pmos,  # manual measurement, used to set
the distance between M1 and M2
            cross_section="metal4_routing",
            width=4.28,  #  manual measurement
        )
    )

    drain_connection_m1_m2.connect(
        "e1", m1_3_ref.ports["drain_connection_M1"], allow_width_mismatch=True,
allow_layer_mismatch=True
    )
    m2_4_ref.connect(
        "drain_connection_M2", drain_connection_m1_m2.ports["e2"],
allow_width_mismatch=True, allow_layer_mismatch=True
    )

    # drain connection for replica circuit
    routing_drain_m1_m3 = gf.routing.route_bundle_electrical(
        component=c_output,
        ports1=[m1_3_ref.ports["drain_connection_M3"]],
        ports2=[m2_4_ref.ports["drain_connection_M4"]],
        route_width=ROUTE_WIDTH_REPLICA,
        layer=ihp.tech.LAYER.Metal4drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    # vias for resistor xr2 and xr4 connections
    via_m1_m2 = ihp.cells.via_stack(
        top_layer="Metal2",
        bottom_layer="Metal1",
        vn_columns=2,
        vn_rows=2,
    )
    via_m1_m2.ports["top"].orientation = 90
    via_m1_m3 = ihp.cells.via_stack(
        top_layer="Metal3",
        bottom_layer="Metal1",
        vn_columns=2,
        vn_rows=2,
```

```
    )
    via_m1_m4 = ihp.cells.via_stack(
        top_layer="Metal4",
        bottom_layer="Metal1",
        vn_columns=2,
        vn_rows=2,
    )

    xr4 = ihp.cells.rppd(width=RPPD_WIDTH, length=RPPD_LENGTH)
    xr4_ref = c_output.add_ref(xr4).rotate(90)
    xr4_ref.center = m1_3_ref.ports["DS_18"].center
    xr4_ref.movex(0.01)  # manual measurement to align with the gate connection
    xr4_ref.ymax = m1_3_ref.ymin - 2  # manual measurement to set the spacing
between the nmos and the resistors

    via_m1_m4_ref = c_output.add_ref(via_m1_m4)
    via_m1_m4_ref.connect("bottom", xr4_ref.ports["e1"],
allow_width_mismatch=True, allow_layer_mismatch=True)
    c_output.add_port(name="vref", port=via_m1_m4_ref.ports["top"])


    via_m1_m2.ports["top"].orientation = 180
    via_m1_m2_ref = c_output.add_ref(via_m1_m2)
    via_m1_m2_ref.connect("bottom", xr4_ref.ports["e2"],
allow_width_mismatch=True, allow_layer_mismatch=True)
    c_output.add_ports(via_m1_m2_ref.ports, prefix="XR4_")

    xr2 = ihp.cells.rppd(width=RPPD_WIDTH, length=RPPD_LENGTH)
    xr2_ref = c_output.add_ref(xr2).rotate(90)
    xr2_ref.ymax = xr4_ref.ymin - 1  # manual measurement to set the spacing
between the two resistors to 0
    xr2_ref.xmax = xr4_ref.xmax

    via_m1_m4_ref = c_output.add_ref(via_m1_m4)
    via_m1_m4_ref.connect("bottom", xr2_ref.ports["e2"],
allow_width_mismatch=True, allow_layer_mismatch=True)

    c_output.add_port(name="vout", port=via_m1_m4_ref.ports["top"])
    via_m1_m3_ref = c_output.add_ref(via_m1_m3)
    via_m1_m3_ref.connect("bottom", xr2_ref.ports["e1"],
allow_width_mismatch=True, allow_layer_mismatch=True)


    # bias2
    route_gates_m3_m4 = gf.routing.route_bundle_electrical(
        component=c_output,
        ports1=[m1_3_ref.ports["gate_connection_M3"]],
        ports2=[via_m1_m2_ref.ports["top"]],
        route_width=ROUTE_WIDTH_BIAS,
        layer=ihp.tech.LAYER.Metal2drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
        steps=[
            {"dy": -2.46, "dx": 0.5},
            {"dx": 1},
        ],
    )
```

```
    # node bb_int routing
    route_gates_m1_m2 = gf.routing.route_bundle_electrical(
        component=c_output,
        ports1=[m1_3_ref.ports["gate_M1_top"]],
        ports2=[m2_4_ref.ports["gate_M2_top"]],
        route_width=ROUTE_WIDTH_GATE,
        layer=ihp.tech.LAYER.Metal3drawing,
        start_straight_length=0.5,
        end_straight_length=0.5,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )
    route_gates_m1_m2 = gf.routing.route_bundle_electrical(
        component=c_output,
        ports1=[m1_3_ref.ports["gate_M1_top"]],
        ports2=[xr2_ref.ports["e1"]],
        route_width=ROUTE_WIDTH_GATE,
        layer=ihp.tech.LAYER.Metal3drawing,
        start_straight_length=0.5,
        end_straight_length=0.5,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    c_output.add_ports(m1_3_ref.ports, prefix="M1_")
    c_output.add_ports(m2_4_ref.ports, prefix="M2_")
    output_stage_ref = c.add_ref(c_output.rotate(90))
    output_stage_ref.center = c2_ref.center
    output_stage_ref.ymin = c2_ref.ymin
    output_stage_ref.move((17.99 + 1.64, 9.02 + 5.23))

    # create capacitor array for cap 3
    c3 = gf.Component("C3")
    # create 3 * 4 array of cmim capacitors with column pitch of 0.52 + 11.2 to
fit the required spacing for TM1 and the size of the caps
    # with one missing cap to fit C1 in there
    spacing_between_caps = CMIM_SPACING_TM1
    left_col = c3.add_ref(
        gf.components.array(
            component=cmim,
            columns=3,
            rows=5,
            column_pitch=spacing_between_caps + CMIM_PITCH,
            row_pitch=-(spacing_between_caps + CMIM_PITCH),
        )
    )
    middle_col = c3.add_ref(
        gf.components.array(
            component=cmim,
            columns=2,
            rows=4,
            column_pitch=spacing_between_caps + CMIM_PITCH,
            row_pitch=-(spacing_between_caps + CMIM_PITCH),
        )
    )
    right_col = c3.add_ref(
```

```
        gf.components.array(
            component=cmim,
            columns=1,
            rows=5,
            column_pitch=spacing_between_caps + CMIM_PITCH,
            row_pitch=-(spacing_between_caps + CMIM_PITCH),
        )
    )
    left_col.xmax = middle_col.xmin - spacing_between_caps  # min spacing
according to design rules 5.17
    right_col.xmin = middle_col.xmax + spacing_between_caps  # min spacing
according to design rules 5.17
    c3.add_ports(left_col.ports, prefix="LC_")
    c3.add_ports(middle_col.ports, prefix="MC_")
    c3.add_ports(right_col.ports, prefix="RC_")

    # orient the outer most port of for horizontal connection
    ihp.cells.utils.change_port_orientation(c3, ["LC_T_1_1", "LC_T_2_1",
"LC_T_3_1", "LC_T_4_1", "LC_T_5_1"], 0)
    ihp.cells.utils.change_port_orientation(c3, ["LC_B_1_1", "LC_B_2_1",
"LC_B_3_1", "LC_B_4_1", "LC_B_5_1"], 0)

    ihp.cells.utils.change_port_orientation(c3, ["RC_T_1_1", "RC_T_2_1",
"RC_T_3_1", "RC_T_4_1", "LC_T_5_3"], 180)
    ihp.cells.utils.change_port_orientation(c3, ["RC_B_1_1", "RC_B_2_1",
"RC_B_3_1", "RC_B_4_1", "LC_B_5_3"], 180)

    c3_connection_t = gf.routing.route_bundle_electrical(
        component=c3,
        ports1=[
            c3.ports["LC_T_1_1"],
            c3.ports["LC_T_2_1"],
            c3.ports["LC_T_3_1"],
            c3.ports["LC_T_4_1"],
            c3.ports["LC_T_5_1"],
        ],
        ports2=[
            c3.ports["RC_T_1_1"],
            c3.ports["RC_T_2_1"],
            c3.ports["RC_T_3_1"],
            c3.ports["RC_T_4_1"],
            c3.ports["LC_T_5_3"],
        ],
        route_width=cap_connection_thickness,
        layer=ihp.tech.LAYER.TopMetal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    c3_connection_b = gf.routing.route_bundle_electrical(
        component=c3,
        ports1=[
            c3.ports["LC_B_1_1"],
            c3.ports["LC_B_2_1"],
            c3.ports["LC_B_3_1"],
            c3.ports["LC_B_4_1"],
            c3.ports["LC_B_5_1"],
        ],
```

```
        ports2=[
            c3.ports["RC_B_1_1"],
            c3.ports["RC_B_2_1"],
            c3.ports["RC_B_3_1"],
            c3.ports["RC_B_4_1"],
            c3.ports["LC_B_5_3"],
        ],
        route_width=cap_connection_thickness,
        layer=ihp.tech.LAYER.Metal5drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    # orient top row ports to face downwards to connect to bottom row ports
    ihp.cells.utils.change_port_orientation(
        c3, ["LC_T_1_1", "LC_T_1_2", "LC_T_1_3", "MC_T_1_1", "MC_T_1_2",
"RC_T_1_1"], 270
    )
    ihp.cells.utils.change_port_orientation(
        c3, ["LC_B_1_1", "LC_B_1_2", "LC_B_1_3", "MC_B_1_1", "MC_B_1_2",
"RC_B_1_1"], 270
    )

    # orient bottom row ports to face downwards to connect to top row ports
    ihp.cells.utils.change_port_orientation(
        c3, ["LC_T_5_1", "LC_T_5_2", "LC_T_5_3", "MC_T_4_1", "MC_T_4_2",
"RC_T_5_1"], 90
    )
    ihp.cells.utils.change_port_orientation(
        c3, ["LC_B_5_1", "LC_B_5_2", "LC_B_5_3", "MC_B_4_1", "MC_B_4_2",
"RC_B_5_1"], 90
    )

    c3_connection_top = gf.routing.route_bundle_electrical(
        component=c3,
        ports1=[
            c3.ports["LC_T_1_1"],
            c3.ports["LC_T_1_2"],
            c3.ports["LC_T_1_3"],
            c3.ports["MC_T_1_1"],
            c3.ports["MC_T_1_2"],
            c3.ports["RC_T_1_1"],
        ],
        ports2=[
            c3.ports["LC_T_5_1"],
            c3.ports["LC_T_5_2"],
            c3.ports["LC_T_5_3"],
            c3.ports["MC_T_4_1"],
            c3.ports["MC_T_4_2"],
            c3.ports["RC_T_5_1"],
        ],
        route_width=cap_connection_thickness,
        layer=ihp.tech.LAYER.TopMetal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )
```

```
    c3_connection_bottom = gf.routing.route_bundle_electrical(
        component=c3,
        ports1=[
            c3.ports["LC_B_1_1"],
            c3.ports["LC_B_1_2"],
            c3.ports["LC_B_1_3"],
            c3.ports["MC_B_1_1"],
            c3.ports["MC_B_1_2"],
            c3.ports["RC_B_1_1"],
        ],
        ports2=[
            c3.ports["LC_B_5_1"],
            c3.ports["LC_B_5_2"],
            c3.ports["LC_B_5_3"],
            c3.ports["MC_B_4_1"],
            c3.ports["MC_B_4_2"],
            c3.ports["RC_B_5_1"],
        ],
        route_width=cap_connection_thickness,
        layer=ihp.tech.LAYER.Metal5drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    # orient top row ports to face downwards to connect to face the middle for
connection purposes
    ihp.cells.utils.change_port_orientation(c3, ["MC_T_4_1", "MC_T_4_2",
"LC_T_5_1", "LC_T_5_2", "LC_T_5_3"], 270)
    # orient top row ports to face downwards to connect to face c2 for
connection purposes
    ihp.cells.utils.change_port_orientation(
        c3, ["MC_B_4_1", "MC_B_4_2", "LC_B_5_1", "LC_B_5_2", "LC_B_5_3",
"RC_B_5_1"], 270
    )

    c3_ref = c.add_ref(c3)

    c3_ref.rotate(-90)
    c3_ref.xmin = c2_ref.xmax + CMIM_SPACING_TM1  # manual measurement to set
the spacing between c2 and c3
    c3_ref.ymin = c2_ref.ymin

    c_fill = ihp.cells.cmim(width=CMIM_FILL_WIDTH, length=CMIM_FILL_LENGTH)
    c_fill.ports["B"].orientation = 0
    c_fill_1_ref = c.add_ref(c_fill)
    c_fill_2_ref = c.add_ref(c_fill)

    straight_m5 = ihp.cells.straight(
        length=10.12,
        cross_section="metal5_routing",
        width=cap_connection_thickness,  # manual measurement to align with the
via
    )
    straight_m5_ref = c.add_ref(straight_m5)
    straight_m5_ref.connect("e1", c3_ref.ports["MC_B_4_1"],
allow_width_mismatch=True, allow_layer_mismatch=True)
    c_fill_1_ref.connect("B", straight_m5_ref.ports["e2"],
```

```
allow_width_mismatch=True, allow_layer_mismatch=True)

    straight_m5_ref = c.add_ref(straight_m5)
    straight_m5_ref.connect("e1", c3_ref.ports["MC_B_4_2"],
allow_width_mismatch=True, allow_layer_mismatch=True)
    c_fill_2_ref.connect("B", straight_m5_ref.ports["e2"],
allow_width_mismatch=True, allow_layer_mismatch=True)

    route_c2_c3_vss = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[c2_ref.ports["TR_B_1_1"], c2_ref.ports["TR_B_1_2"],
c2_ref.ports["TR_B_1_3"], c2_ref.ports["TR_B_1_6"]],
        ports2=[c3_ref.ports["LC_B_5_1"], c3_ref.ports["LC_B_5_2"],
c3_ref.ports["LC_B_5_3"], c3_ref.ports["RC_B_5_1"]],
        route_width=cap_connection_thickness,  # adjust for thicker connections
        layer=ihp.tech.LAYER.Metal5drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    # connect nmos source/bulk to vss
    via_m1_m5 = ihp.cells.via_stack(
        top_layer="Metal5",
        bottom_layer="Metal1",
        vn_columns=3,
        vn_rows=35,
    )
    via_m1_m5.ports["bottom"].orientation = 0
    via_m1_m5_ref = c.add_ref(via_m1_m5)
    via_m1_m5_ref.connect(
        "bottom", output_stage_ref.ports["M1_DS_16"], allow_width_mismatch=True,
allow_layer_mismatch=True
    )

    # connect nmos source/bulk to vdd
    via_m1_tm1 = ihp.cells.via_stack(
        top_layer="TopMetal1",
        bottom_layer="Metal1",
        vn_columns=3,
        vn_rows=35,
        vt1_columns=1,
        vt1_rows=17,
    )
    via_m1_tm1.ports["bottom"].orientation = 90
    via_m1_tm1_ref = c.add_ref(via_m1_tm1)
    via_m1_tm1_ref.connect(
        "bottom", output_stage_ref.ports["M2_DS_14"], allow_width_mismatch=True,
allow_layer_mismatch=True
    )

    tm1_straight = ihp.cells.straight(
        length=abs(output_stage_ref.ports["M2_DS_14"].center[0] -
c3_ref.ports["MC_T_4_1"].center[0]),
        cross_section="topmetal1_routing",
        width=2,  # manual measurement to align with the via
    )
    tm1_straight_ref = c.add_ref(tm1_straight)
    tm1_straight_ref.connect("e1", c3_ref.ports["MC_T_4_1"],
```

```
allow_width_mismatch=True, allow_layer_mismatch=True)
    tm1_straight_ref = c.add_ref(tm1_straight)
    tm1_straight_ref.connect("e1", c3_ref.ports["MC_T_4_2"],
allow_width_mismatch=True, allow_layer_mismatch=True)

    # connect c2 to bias1 network
    via_m1_tm1 = ihp.cells.via_stack(
        top_layer="TopMetal1",
        bottom_layer="Metal1",
        vn_columns=3,
        vn_rows=4,
        vt1_columns=1,
        vt1_rows=2,
    )
    via_m1_tm1.ports["top"].orientation = 0
    via_m1_tm1.ports["bottom"].orientation = 180
    via_m1_tm1_ref = c.add_ref(via_m1_tm1).rotate(90)
    via_m1_tm1_ref.xmax = c2_ref.xmax - 4.55
    via_m1_tm1_ref.ymax = c3_ref.ymin - 5  # min M5 spacing

    route_c2_bias = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[c2_ref.ports["TR_T_1_6"]],
        ports2=[via_m1_tm1_ref.ports["top"]],
        route_width=max(via_m1_tm1_ref.xsize, via_m1_tm1_ref.ysize),  # manual
change to fit width
        layer=ihp.tech.LAYER.TopMetal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    route = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[via_m1_tm1_ref.ports["bottom"]],
        ports2=[xr1_ref.ports["e2"]],
        route_width=ROUTE_WIDTH_BIAS_M1,
        layer=ihp.tech.LAYER.Metal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )
    via_m1_tm1.ports["bottom"].orientation = 270
    xr3 = ihp.cells.rhigh(width=RHIGH_WIDTH, length=RHIGH_LENGTH)
    xr3_ref = c.add_ref(xr3).rotate(90)
    xr3_ref.center = via_m1_tm1_ref.center
    xr3_ref.xmin = c3_ref.xmin
    route_xr3 = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[via_m1_tm1_ref.ports["bottom"]],
        ports2=[xr3_ref.ports["e1"]],
        route_width=ROUTE_WIDTH_BIAS_M1,
        layer=ihp.tech.LAYER.Metal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )
```

```
    via_m1_tm1 = ihp.cells.via_stack(
        top_layer="TopMetal1",
        bottom_layer="Metal1",
        vn_columns=4,
        vn_rows=3,
        vt1_columns=2,
        vt1_rows=1,
    )
    via_m1_tm1.ports["bottom"].orientation = 0
    via_m1_tm1_ref = c.add_ref(via_m1_tm1)
    via_m1_tm1_ref.connect("bottom", xr3_ref.ports["e2"],
allow_width_mismatch=True, allow_layer_mismatch=True)

    route_c2_bias = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[c3_ref.ports["RC_T_5_1"]],
        ports2=[via_m1_tm1_ref.ports["top"]],
        route_width=max(via_m1_tm1_ref.xsize, via_m1_tm1_ref.ysize),  # manual
change to fit width
        layer=ihp.tech.LAYER.TopMetal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    # bb_int connect D1 to network
    via_m1_m3 = ihp.cells.via_stack(
        top_layer="Metal3",
        bottom_layer="Metal1",
        vn_columns=6,
        vn_rows=3,
    )
    via_m1_m3.ports["top"].orientation = 90
    via_m1_m3.ports["bottom"].orientation = 270
    via_m1_m3_ref = c.add_ref(via_m1_m3)
    via_m1_m3_ref.connect("bottom", d1_ref.ports["e2"],
allow_width_mismatch=True, allow_layer_mismatch=True)

    # bb_int
    routing_bb_int = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[output_stage_ref.ports["M1_gate_M1_top"]],
        ports2=[via_m1_m3_ref.ports["top"]],
        route_width=ROUTE_WIDTH_GATE,
        layer=ihp.tech.LAYER.Metal3drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        end_straight_length=2,
        auto_taper=False,
        separation=0,
        steps=[
            {"dx": 3.65, "dy": -1},  # strange behavior without the dy direction
            {"dy": -6},
            {"dx": -4, "dy": 0},
        ],
    )

    # bias2 connect D2 to network
    via_m1_m2 = ihp.cells.via_stack(
```

```
        top_layer="Metal2",
        bottom_layer="Metal1",
        vn_columns=6,
        vn_rows=3,
    )
    via_m1_m2.ports["top"].orientation = 90
    via_m1_m2.ports["bottom"].orientation = 270
    via_m1_m2_ref = c.add_ref(via_m1_m2)
    via_m1_m2_ref.connect("bottom", d2_ref.ports["e2"],
allow_width_mismatch=True, allow_layer_mismatch=True)

    # bias2 connection from gate of M3 to D2
    routing_bias2 = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[output_stage_ref.ports["M1_gate_connection_M3"]],
        ports2=[via_m1_m2_ref.ports["top"]],
        route_width=ROUTE_WIDTH_BIAS,
        layer=ihp.tech.LAYER.Metal2drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        end_straight_length=3,
        auto_taper=False,
        separation=0,
        steps=[
            {"dx": 2.46, "dy": -1},  # strange behavior without the dy direction
            {"dy": -1},
            {"dx": 2.17, "dy": 0},
            {"dy": -2},
        ],
    )
    # bias2 connection from gate of M4 to D2
    routing_bias2 = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[output_stage_ref.ports["M2_gate_connection_M4"]],
        ports2=[via_m1_m2_ref.ports["top"]],
        route_width=ROUTE_WIDTH_BIAS,
        layer=ihp.tech.LAYER.Metal2drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        end_straight_length=3,
        auto_taper=False,
        separation=0,
        steps=[
            {"dx": -2.46, "dy": -1},  # strange behavior without the dy
direction
            {"dy": -1},
            {"dx": -2.21, "dy": 0},
            {"dy": -2},
        ],
    )

    # SBD D1 D2 sub connection
    route_d1_d2 = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[d1_ref.ports["e3"]],
        ports2=[d2_ref.ports["e1"]],
        route_width=ROUTE_WIDTH_SBD_SUB,
        layer=ihp.tech.LAYER.Metal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
```

```
        auto_taper=False,
        separation=0,
    )
    via_m1_m3 = ihp.cells.via_stack(
        top_layer="Metal3",
        bottom_layer="Metal1",
        vn_columns=1,
        vn_rows=5,
    )
    via_m1_m3.ports["bottom"].orientation = 0
    via_m1_m3.ports["top"].orientation = 180
    via_m1_m3_ref = c.add_ref(via_m1_m3)
    via_m1_m3_ref.connect("bottom", d2_ref.ports["e3"],
allow_width_mismatch=True, allow_layer_mismatch=True)

    straight_m3 = ihp.cells.straight(
        length=18,
        cross_section="metal3_routing",
        width=max(via_m1_m3_ref.xsize, via_m1_m3_ref.ysize),
    )
    straight_m3_ref = c.add_ref(straight_m3)
    straight_m3_ref.connect("e1", via_m1_m3_ref.ports["top"],
allow_width_mismatch=True, allow_layer_mismatch=True)
    via_m3_m5 = ihp.cells.via_stack(
        top_layer="Metal5",
        bottom_layer="Metal3",
        vn_columns=1,
        vn_rows=5,
    )
    via_m3_m5.ports["bottom"].orientation = 180
    via_m3_m5_ref = c.add_ref(via_m3_m5)
    via_m3_m5_ref.connect("bottom", straight_m3_ref.ports["e2"],
allow_width_mismatch=True, allow_layer_mismatch=True)

    schottky.ports["e1"].orientation = 90
    # connect D1 and D2 sub connection together
    schottky.ports["e3"].orientation = 90
    route_sbd_d1_d2 = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[d1_ref.ports["e1"]],
        ports2=[d2_ref.ports["e3"]],
        route_width=ROUTE_WIDTH_SBD_GUARD,
        layer=ihp.tech.LAYER.Metal1drawing,
        start_straight_length=2.85,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    straight_m3 = ihp.cells.straight(
        length=10,
        cross_section="metal3_routing",
        width=2,
    )
    straight_m3_ref = c.add_ref(straight_m3)
    straight_m3_ref.connect("e1", output_stage_ref.ports["vref"],
allow_width_mismatch=True, allow_layer_mismatch=True)
    rfin_connection = c.add_ref(ihp.cells.straight(length=RFIN_PD,
cross_section="topmetal2_routing", width=7.2))
```

```
    rfin_connection.connect("e1", via_tm1_tm2_ref.ports["top"],
allow_width_mismatch=True)
    # add ports to the power detector
    vout_port = c.add_port(name="vout", port=output_stage_ref.ports["vout"])
    c.ports["vout"].orientation = 90

    vref_port = c.add_port(name="vref", port=straight_m3_ref.ports["e2"])
    c.ports["vref"].center = (c.ports["vref"].center[0],
c.ports["vref"].center[1] - 0.1)  # preventing notch
    c.ports["vref"].orientation = 90

    rfin_port = c.add_port(name="rfin", port=rfin_connection.ports["e2"])
    c.ports["rfin"].orientation = 270

    vss_port = c.add_port(name="vss", port=c3_ref.ports["LC_B_1_1"])
    c.ports["vss"].orientation = 90

    vdd_port = c.add_port(name="vdd", port=c3_ref.ports["LC_T_5_1"])
    c.ports["vdd"].orientation = 90
    c.pprint_ports()

    gf.labels.add_port_labels(c, ports= [vout_port], layer =
ihp.tech.LAYER.Metal4text)
    gf.labels.add_port_labels(c, ports= [vref_port], layer =
ihp.tech.LAYER.Metal3text)
    gf.labels.add_port_labels(c, ports= [rfin_port], layer =
ihp.tech.LAYER.TopMetal2text)
    gf.labels.add_port_labels(c, ports= [vss_port], layer =
ihp.tech.LAYER.Metal5text)
    gf.labels.add_port_labels(c, ports= [vdd_port], layer =
ihp.tech.LAYER.TopMetal1text)
    gf.add_pins.add_pin_rectangle(c, port=vout_port,
layer=ihp.tech.LAYER.Metal4pin)
    gf.add_pins.add_pin_rectangle(c, port=vref_port,
layer=ihp.tech.LAYER.Metal3pin)
    gf.add_pins.add_pin_rectangle_inside(c, port=rfin_port,
layer=ihp.tech.LAYER.TopMetal2pin)
    gf.add_pins.add_pin_rectangle(c, port=vss_port,
layer=ihp.tech.LAYER.Metal5pin)
    gf.add_pins.add_pin_rectangle(c, port=vdd_port,
layer=ihp.tech.LAYER.TopMetal1pin)

    # No-fill exclusion zones for each metal layer
    c.xmin = 0
    c.ymin = 0
    c_center = c.center
    size_nofill_x = round(c.xsize,2)
    size_nofill_y = round(c.ysize,2)
    print(f"size_nofill_x: {size_nofill_x}, size_nofill_y: {size_nofill_y}")
    nofill = c.add_ref(gf.components.rectangle(size=(size_nofill_x + 2,
size_nofill_y + 2), layer=ihp.tech.LAYER.Activnofill))
    nofill.center = c_center
    nofill.xmin = snap_to_grid(nofill.xmin)
    nofill.ymin = snap_to_grid(nofill.ymin)

    for nofill_layer in [
        ihp.tech.LAYER.GatPolynofill,
        ihp.tech.LAYER.Metal1nofill,
        ihp.tech.LAYER.Metal2nofill,
        ihp.tech.LAYER.Metal3nofill,
```

```
        ihp.tech.LAYER.Metal4nofill,
    ]:
        nofill = c.add_ref(gf.components.rectangle(size=(size_nofill_x,
size_nofill_y), layer=nofill_layer, centered=True))
        # snap to grid
        nofill.center = c_center
        nofill.xmin = snap_to_grid(nofill.xmin)
        nofill.ymin = snap_to_grid(nofill.ymin)


    c.add_ref(
        gf.components.rectangle(size=(c1_ref.xsize + 40, c1_ref.ysize + 20),
layer=ihp.tech.LAYER.Metal5nofill)
    ).center = c1_ref.center

    size_x = c2_ref.xsize + c3_ref.xsize
    size_y = c2_ref.ysize
    x = c2_ref.xmin + size_x / 2
    y = c2_ref.ymin + size_y / 2
    cap_m5_nofill = c.add_ref(
        gf.components.rectangle(size=(size_x - 10, size_y - 10),
layer=ihp.tech.LAYER.Metal5nofill)
    ).center = (x, y)

    return c
```

mmWave Frequency-Scalable Six-Port Receiver Design

First, a GDSFactory Component is created as the top-level container for the entire six-port receiver design. All RF devices and signal routing will be added as references within this Component.

In [9]:
```
c = gf.Component("Six-Port-CAC")
```

Design Parameters and Wavelength Calculations

Define the key design parameters that govern the entire six-port receiver layout.

Process & Six-Port Specifications & Transmission Line Metal Stack:

Parameter	Description	Value
`e_r`	Relative permittivity (from process specs)	4.1
`Z0`	Characteristic impedance	50 Ω
`f`	Operating frequency (e.g. 160 GHz), scaling this value automatically resizes the entire six-port layout	variable
`signal_cross_section`	Metal layer for transmission line signal conductors	TopMetal2
`ground_cross_section`	Metal layer for transmission line ground conductors	Metal5

Closed-form transmission-line models:

All passive RF devices are built from embedded microstrip lines, a signal conductor on TopMetal2 over a ground plane on Metal5. Their widths and lengths follow from closed-form models, so no optimization loop or training data is needed, and changing `f` rescales the layout directly. The width w sets the characteristic impedance, from the IPC-2141 surface-microstrip formula solved for w,

$$Z_0 = \frac{87}{\sqrt{\varepsilon_r + 1.41}} \ln \frac{5.98\, h_1}{0.8\, w + t},$$

and the length sets the phase, through the guided wavelength $\lambda_g = c_0/(f\sqrt{\varepsilon_{\mathrm{r,eff}}})$ with the embedded-microstrip approximation

$$\varepsilon_{\mathrm{r,eff}} = \varepsilon_r \left[1 - \exp\left(-1.55\, \frac{h_2}{h_1}\right)\right],$$

which accounts for the dielectric above the conductor. For TopMetal2 over Metal5, the conductor thickness is t = 3 µm, the dielectric between the two layers h_1 = 5.65 µm, and the total dielectric height h_2 = 10.15 µm, which includes 1.5 µm of passivation above TopMetal2. With ε_r = 4.1, this gives at 160 GHz:

Quantity	Value at 160 GHz
effective permittivity $\varepsilon_{\mathrm{r,eff}}$	3.85
free-space wavelength λ_0	1874 µm
guided wavelength λ_g, quarter wavelength $\lambda_g/4$	955 µm, 238.8 µm
width for $Z_0/\sqrt{2}$ = 35.4 Ohm (branch-line coupler)	12.51 µm
width for Z_0 = 50 Ohm (interconnects, branch-line coupler)	7.20 µm
width for $\sqrt{2}\, Z_0$ = 70.7 Ohm (Wilkinson power divider)	2.51 µm

The widths are truncated to the 10 nm grid, and the lengths to 5 nm. An EM simulation of the same lines gives 39.4 Ohm, 51.1 Ohm, and 74.1 Ohm at these widths, 11.5 %, 2.2 %, and 4.7 % above the closed form, and an attenuation of 1.26 dB/mm for the 50 Ohm line at 160 GHz. The formula is stated for $0.1 \le w/h_1 \le 2$, and for stack-ups where h_1 is smaller than t, such as TopMetal2 over TopMetal1, it is off by more than 14 %, so Metal5 is the ground plane. The code cell after the parameters prints these quantities for the frequency `f` that is set.

Bandpass Filter Specifications:

Parameter	Description	Value
`order`	Filter order	3
`bandwidth`	3 dB bandwidth	1 GHz
`filter_type`	Filter type, `"butter"` (Butterworth) or `"cheby"` (Chebyshev)	`"butter"`
`ripple_dB`	Passband ripple for Chebyshev filter	3 dB

Interconnect Lengths:

Parameter	Description	Value
`connection_length_bpf`	Connection length between the bandpass filter and the rest of the circuit	10 µm
`wpd_shape`	Routing shape of the Wilkinson power divider (`"C"` or `"U"`)	`"U"`
`connection_length_wpd`	Connection length at the Wilkinson power divider ports	0 µm
`connection_length_bpf_wpd`	Connection length between the branch-line couplers and the Wilkinson power divider	3.5/5 · λ/4
`connection_length_blc`	Connection length at the branch-line coupler ports	0 µm

In [10]:
```
# RF Design Parameters
e_r = 4.1   # relative permittivity (from process specifications)
Z0  = 50    # characteristic impedance in Ω
f   = 160e9 # operating frequency in Hz

# Transmission Line Metal Stack
signal_cross_section = "topmetal2_routing"  # signal conductor layer (TopMetal2)
ground_cross_section = "metal5_routing"     # ground conductor layer (Metal5)

# Effective Dielectric Constant & Wavelength
e_eff = ihp.cells.waveguides._calculate_effective_dielectric_constant(
    signal_cross_section=signal_cross_section,
    ground_cross_section=ground_cross_section,
    e_r=e_r,
)

c0           = scipy.constants.c            # speed of light in vacuum in m/s
wavelength   = c0 / f * 1e6 / sqrt(e_eff)   # guided wavelength in µm
wavelength_4 = wavelength / 4               # quarter-wavelength in µm

# Snap to process grid (nearest nm)
wavelength   = snap_to_grid(wavelength)       # λ in µm
wavelength_4 = snap_to_grid(wavelength_4)     # λ/4 in µm
wavelength_8 = snap_to_grid(wavelength / 8)   # λ/8 in µm

# Bandpass Filter Parameters
order                = 3        # filter order
bandwidth            = 1e9      # 3 dB bandwidth in Hz
filter_type          = "butter" # filter type: "butter" (Butterworth) or
"cheby" (Chebyshev)
ripple_dB            = 3        # passband ripple in dB, applies to Chebyshev
only
connection_length_bpf = 10      # connection length between the bandpass filter
and the circuit in µm

# Wilkinson Power Divider Parameters
wpd_shape                = "U"                                  # routing
shape of the WPD: "C" or "U"
connection_length_wpd    = 0                                    # connection
length at WPD ports in µm
```

```
connection_length_bpf_wpd = snap_to_grid(wavelength_4 * 3.5 / 5)   # connection
length between the WPD and the branch-line couplers in µm

# Branch-Line Coupler Parameters
connection_length_blc = 0  # connection length at branch-line coupler ports in
µm
```

The effective permittivity, the guided wavelength, and the three line widths of the closed-form models, for the frequency `f` set above:

In [11]:
```
import contextlib
import io

# the quantities the RF devices below are generated from, for the frequency f
set above
print(f"effective permittivity: {e_eff:.3f}")
print(f"guided wavelength: {wavelength:.2f} um, lambda/4 = {wavelength_4:.2f}
um, lambda/8 = {wavelength_8:.2f} um")
for name, Z in (("Z0/sqrt(2)", Z0 / sqrt(2)), ("Z0", Z0), ("sqrt(2) Z0", Z0 *
sqrt(2))):
    with contextlib.redirect_stdout(io.StringIO()):  # the add-on prints every
width it calculates
        width = ihp.cells.waveguides._calculate_width_from_Z0(Z0=Z, e_r=e_r,
signal_cross_section=signal_cross_section,
ground_cross_section=ground_cross_section)
    print(f"line width for {name} = {Z:.1f} Ohm: {width:.2f} um")
```

```
effective permittivity: 3.847
guided wavelength: 955.33 um, lambda/4 = 238.83 um, lambda/8 = 119.42 um
line width for Z0/sqrt(2) = 35.4 Ohm: 12.51 um
line width for Z0 = 50.0 Ohm: 7.20 um
line width for sqrt(2) Z0 = 70.7 Ohm: 2.51 um
```

Branch-Line Coupler Design

The first RF device is the branch-line coupler (BLC), a 90° hybrid: a square of four $\lambda_g/4$ lines, two opposite branches with $Z_0/\sqrt{2}$ and the other two with Z_0. A wave entering the input port splits into two halves (-3 dB each) at the through and the coupled port, 90° apart in phase, while the fourth port is isolated. The three BLCs combine the LO and RF waves under the relative phases of 0, π/2, π, and 3π/2 that produce the four six-port outputs.

The BLC is a custom cell of our IHP-Open-PDK GDSFactory add-on (`ihp.cells.branch_line_coupler`). It takes the widths for Z_0 and $Z_0/\sqrt{2}$ from the closed form above, the branch length $\lambda_g/4$ from the effective permittivity, and joins the branches with chamfered corner pieces. All dimensions follow from `f`:

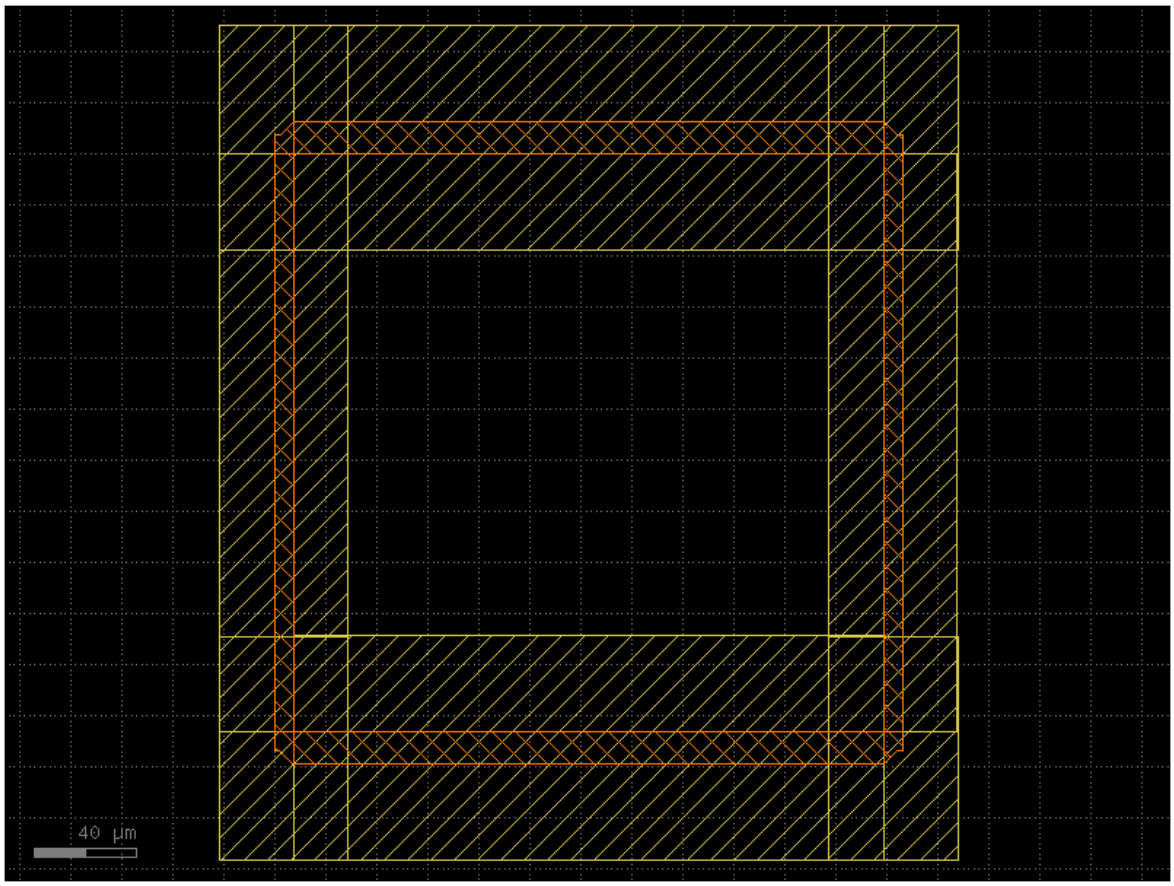
BLC at 160 GHz	Value
branches with $Z_0/\sqrt{2}$ = 35.4 Ohm	12.51 µm wide, 238.8 µm long
branches with Z_0 = 50 Ohm	7.20 µm wide, 238.8 µm long
footprint	289 µm x 326 µm

As generated, without any tuning, the EM simulation gives an input matching of -16 dB or better and an isolation of -15 dB to -17 dB at the design frequency, for designs from 60 GHz to 300 GHz. The amplitude imbalance between the through and the coupled port grows from 1.2 dB at 60 GHz to 1.8 dB at 300 GHz, and the phase difference stays within 1° to 6° of the ideal -90°, small enough for six-port operation.

The code cell generates the BLC from `f`, `Z0`, `e_r`, and the two cross sections, and plots it. The next cell adds it to the chip as BLC_3, the coupler of the RF input, and two more references of the same cell become BLC_1 and BLC_2.

In [12]:
```
blc = ihp.cells.branch_line_coupler(
    frequency=f,
    connection_length=connection_length_blc,
    signal_cross_section=signal_cross_section,
    ground_cross_section=ground_cross_section,
    Z0=Z0,
    e_r=e_r,
)
blc.plot()
```

```
Calculated width: 7.2 um for Z0: 50 ohm
Calculated width: 12.51 um for Z0: 35.36 ohm
Calculated Z0: 35.3772968531251 ohm for width: 12.51 um
Calculated Z0: 50.03094234385018 ohm for width: 7.2 um
Calculated Z0: 50.03094234385018 ohm for width: 7.2 um
```



The first branch line coupler (BLC_3) is added as a reference to the main component at the center position (0, 0).

In [13]:
```
blc_3_ref = c.add_ref(blc)
```

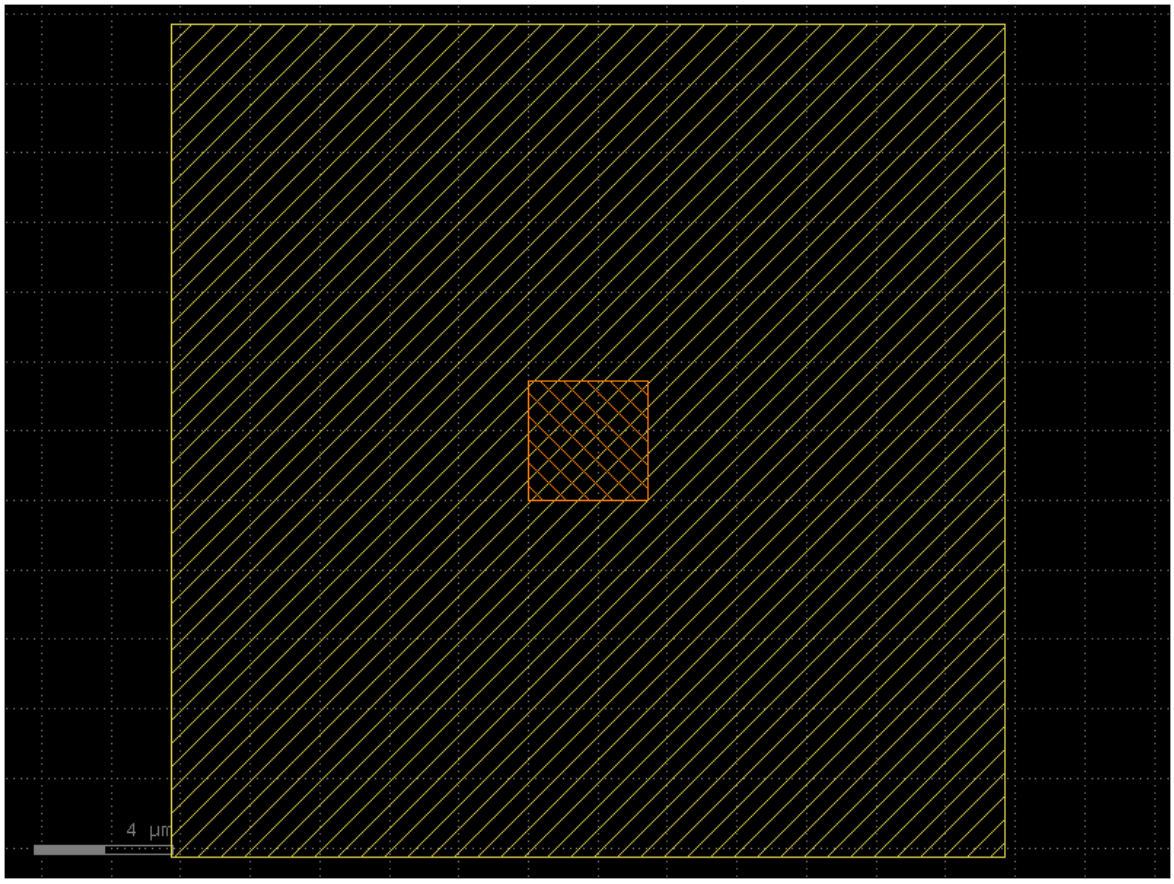
Corner Transition Piece

The three BLCs are placed corner to corner, which keeps the paths between them as short as possible and so minimizes conductor loss and phase mismatch. Where two BLCs meet, `tline_corner` joins their corner ports: a square TopMetal2 patch with a port on each side over an oversized Metal5 ground plate. Its side is the width of a line with `CORNER_Z_FACTOR` x Z_0 = 65.7 Ohm, 3.42 µm.

In [14]:
```
corner = ihp.cells.tline_corner(
    signal_cross_section=signal_cross_section,
    ground_cross_section=ground_cross_section,
    Z0=Z0 * CORNER_Z_FACTOR,
)

corner.plot()
corner_top_ref = c.add_ref(corner)
```

```
Calculated width: 3.42 um for Z0: 65.7 ohm
```

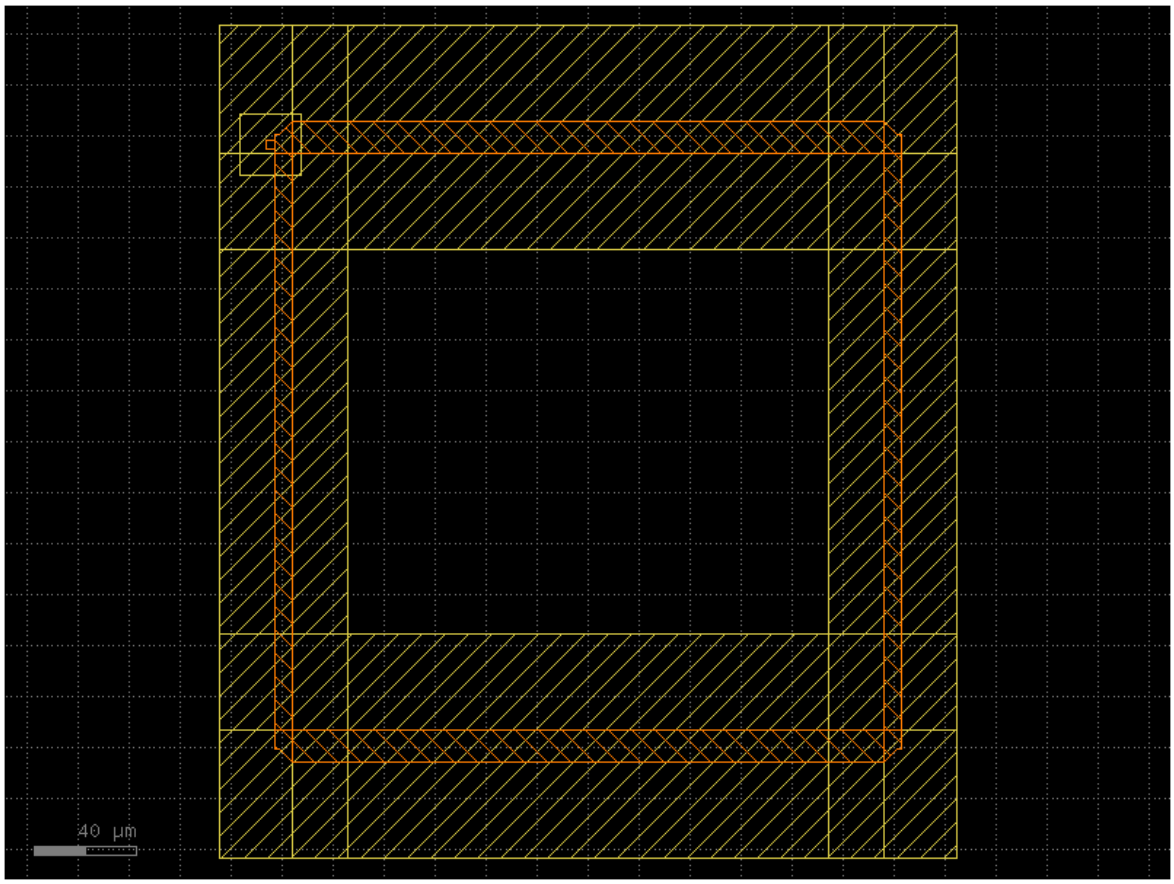


The first BLC (BLC_3) is positioned at the origin (0, 0) and connects the top corner piece to its port (e1) at 45 degrees. To verify the placement, the result is plotted.

The corner square is narrower than the 7.2 µm coupler port it attaches to, which is why the width check of `connect()` is switched off with `allow_width_mismatch=True`.

In [15]:
```
blc_3_ref.center = (0, 0)
corner_top_ref.connect("e1", blc_3_ref.ports["e1"], allow_width_mismatch=True)

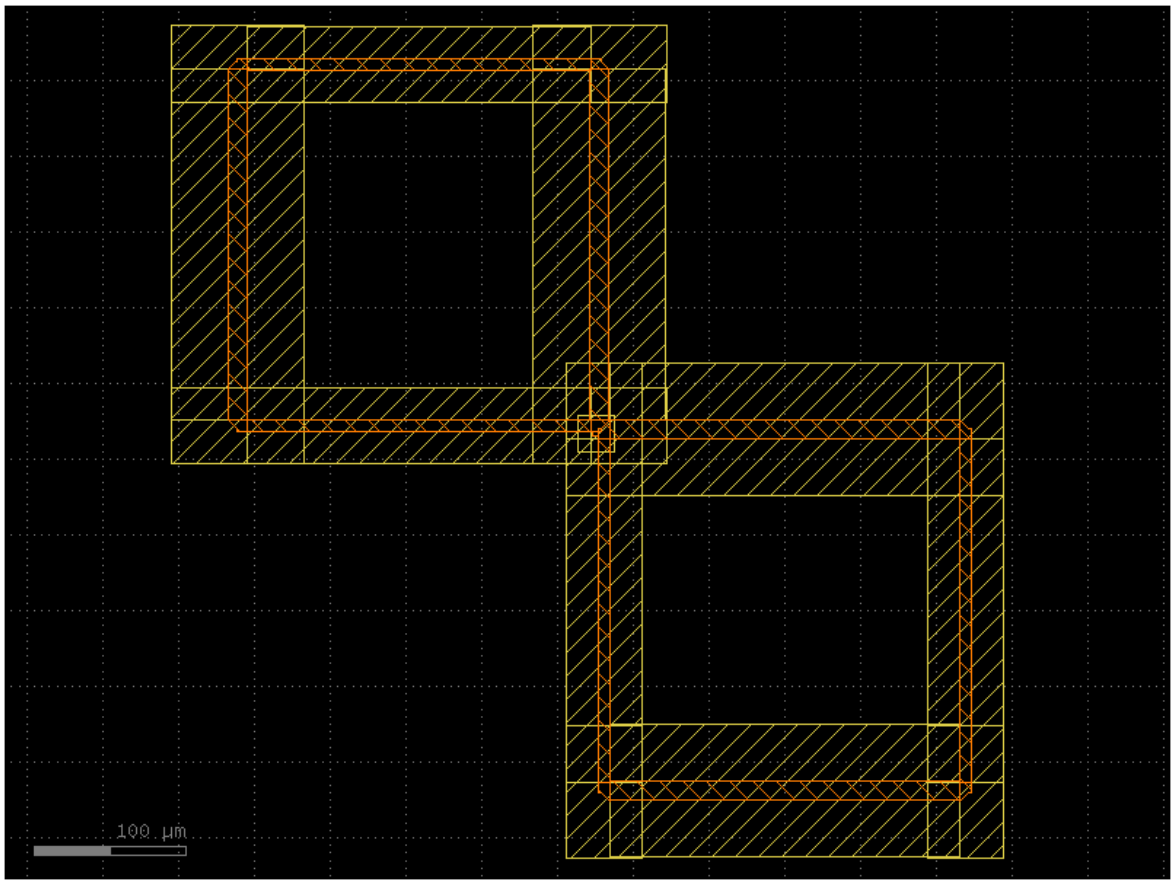
c.plot()
```



The second branch line coupler (BLC_1) is added at the top by connecting it to the top corner pieces port (e4).

In [16]:
```
blc_1_ref = c.add_ref(blc)
blc_1_ref.connect("e4", corner_top_ref.ports["e4"], allow_width_mismatch=True)

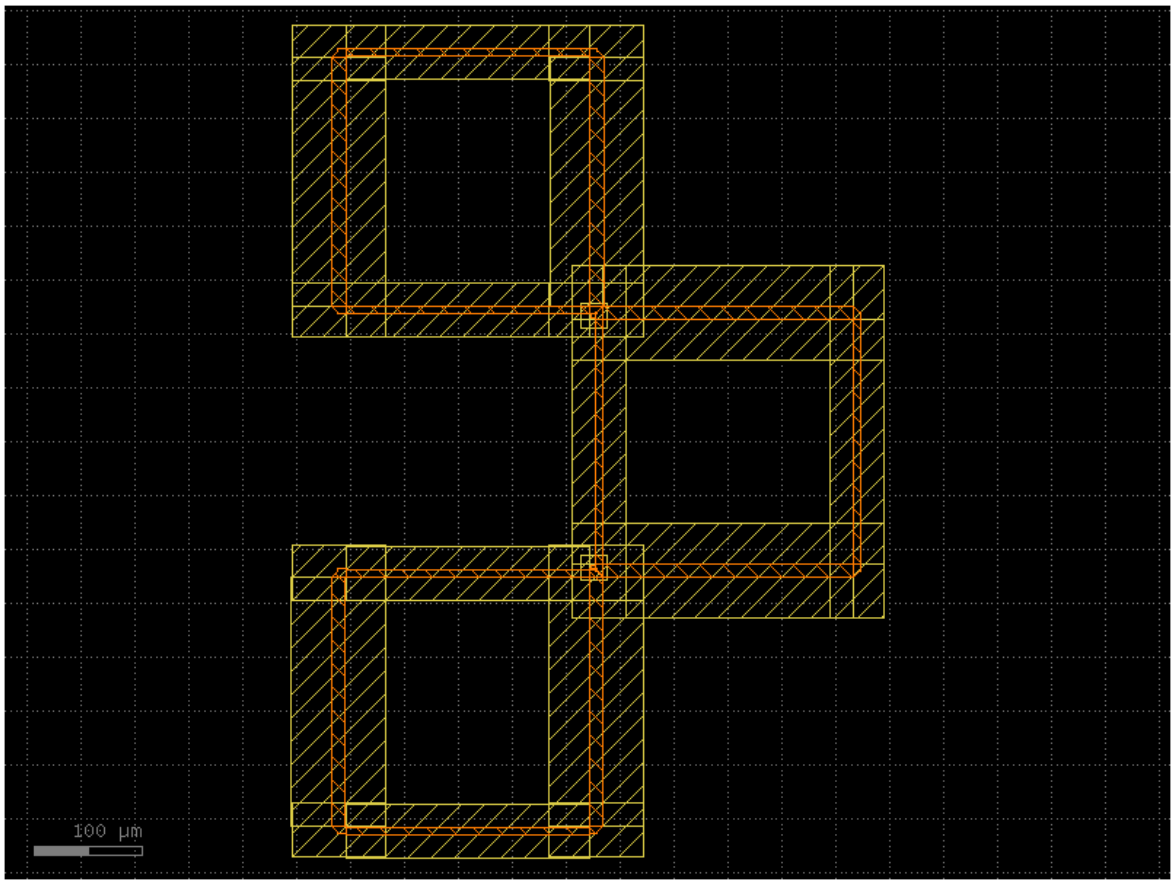
c.plot()
```



A second corner piece is added for the bottom branch, and the third BLC (BLC_2) is placed by connecting it through this corner piece to BLC_3's bottom port (e4).

In [17]:
```
corner_bot_ref = c.add_ref(corner)
corner_bot_ref.connect("e1", blc_3_ref.ports["e4"], allow_width_mismatch=True)
blc_2_ref = c.add_ref(blc)
blc_2_ref.connect("e1", corner_bot_ref.ports["e2"], allow_width_mismatch=True)

c.plot()
```



Wilkinson Power Divider Design

Now that the three BLCs form a corner structure, the second RF device is the Wilkinson power divider (WPD). It splits the LO from the bandpass filter equally and in phase into the two paths to the top and the bottom BLC. Each arm is a $\lambda_g/4$ line with $\sqrt{2}\,Z_0$, which transforms the 50 Ohm load at its output to 100 Ohm, so that the two arms in parallel present 50 Ohm at the input.

The textbook WPD has an isolation resistor of $2Z_0$ between its outputs, which only carries current when the two outputs see different signals. Under the matched, in-phase, equal-amplitude condition of this forward-dividing path it can be omitted, and at mm-wave frequencies it has to be: a 100 Ohm resistor with its connections exceeds the $\lambda_0/10$ rule of thumb for lumped elements and would add an unintended, frequency-dependent element. The price is an output isolation of about -7 dB, close to the -6 dB of an ideal resistor-less divider, which is acceptable because both outputs see symmetric, matched loads.

The WPD is a custom cell of the add-on (`ihp.cells.wilkinson_power_divider`) and is placed in the central space between BLC_1 (top) and BLC_2 (bottom) to save area:

WPD at 160 GHz	Value
arms with $\sqrt{2}\,Z_0$ = 70.7 Ohm	2.51 µm wide, 238.8 µm long
input and outputs with Z_0 = 50 Ohm	7.20 µm wide
footprint, `wpd_shape = "U"`	174 µm x 174 µm

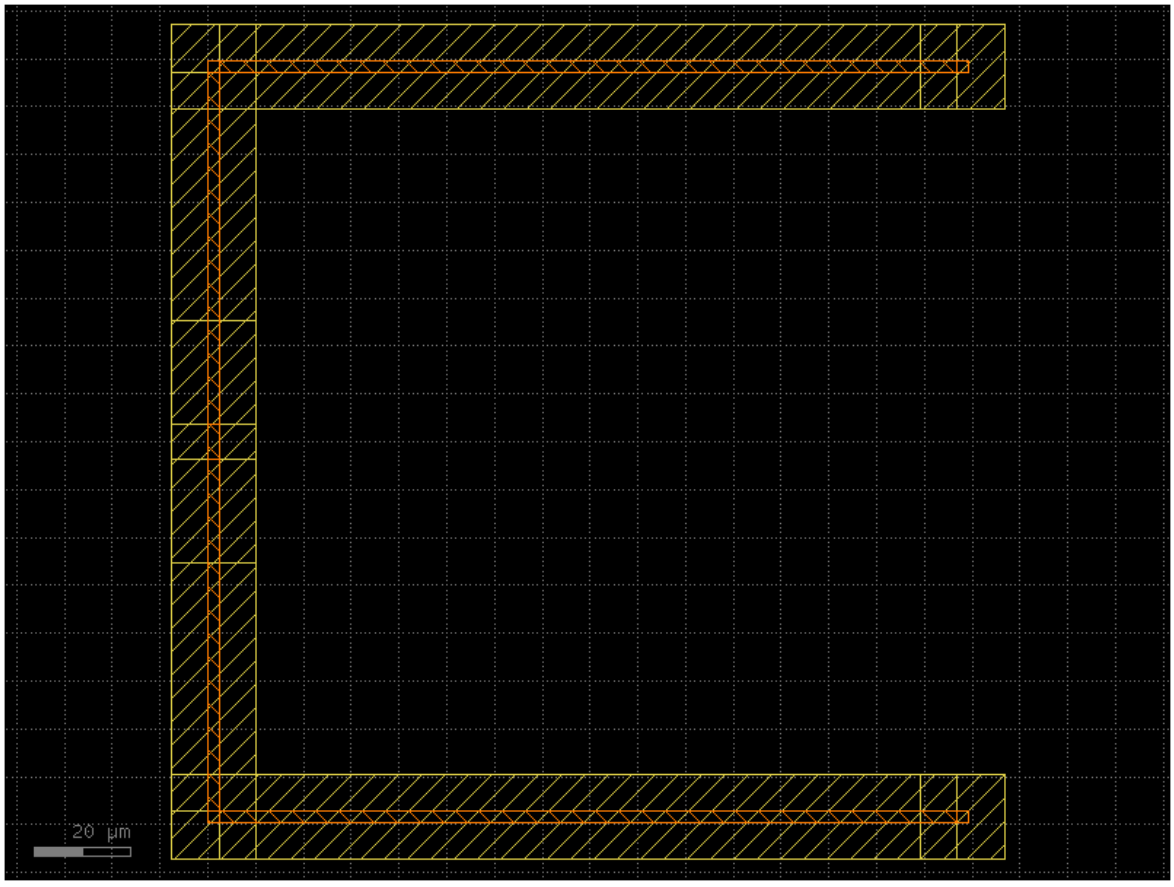
The EM simulation gives an input matching below -16 dB and only 0.3 dB to 0.7 dB of excess loss above the ideal 3 dB split, and, owing to the symmetry, an amplitude and phase imbalance between the outputs within 0.02 dB and 0.1°.

The code cell generates the WPD and plots it.

In [18]:
```
wpd = ihp.cells.wilkinson_power_divider(
    frequency=f,
    connection_length=connection_length_wpd,
    signal_cross_section=signal_cross_section,
    ground_cross_section=ground_cross_section,
    Z0=Z0,
    e_r=e_r,
    shape=wpd_shape,
)

wpd.plot()
```

```
Calculated width: 7.2 um for Z0: 50 ohm
Calculated width: 2.5100000000000002 um for Z0: 70.71 ohm
Calculated Z0: 70.75520001248569 ohm for width: 2.51 um
Calculated width: 2.5100000000000002 um for Z0: 70.71 ohm
Calculated Z0: 70.75520001248569 ohm for width: 2.51 um
```



The input port of the WPD is rotated to face left (orientation = 0), then is added as a reference to the main component. This ensures the input transmission line connects properly from the left side.

In [19]:
```
wpd.ports["e1"].orientation = 0
wpd_ref = c.add_ref(wpd)
```

WPD-to-BLC Connections

The transmission line segments that connect the WPD outputs to the BLC ports are calculated as follows:

- Measure the vertical distance between BLC_1 and BLC_2
- Subtract the WPD height
- Divide by 2 to get the connection length for each side (top and bottom)

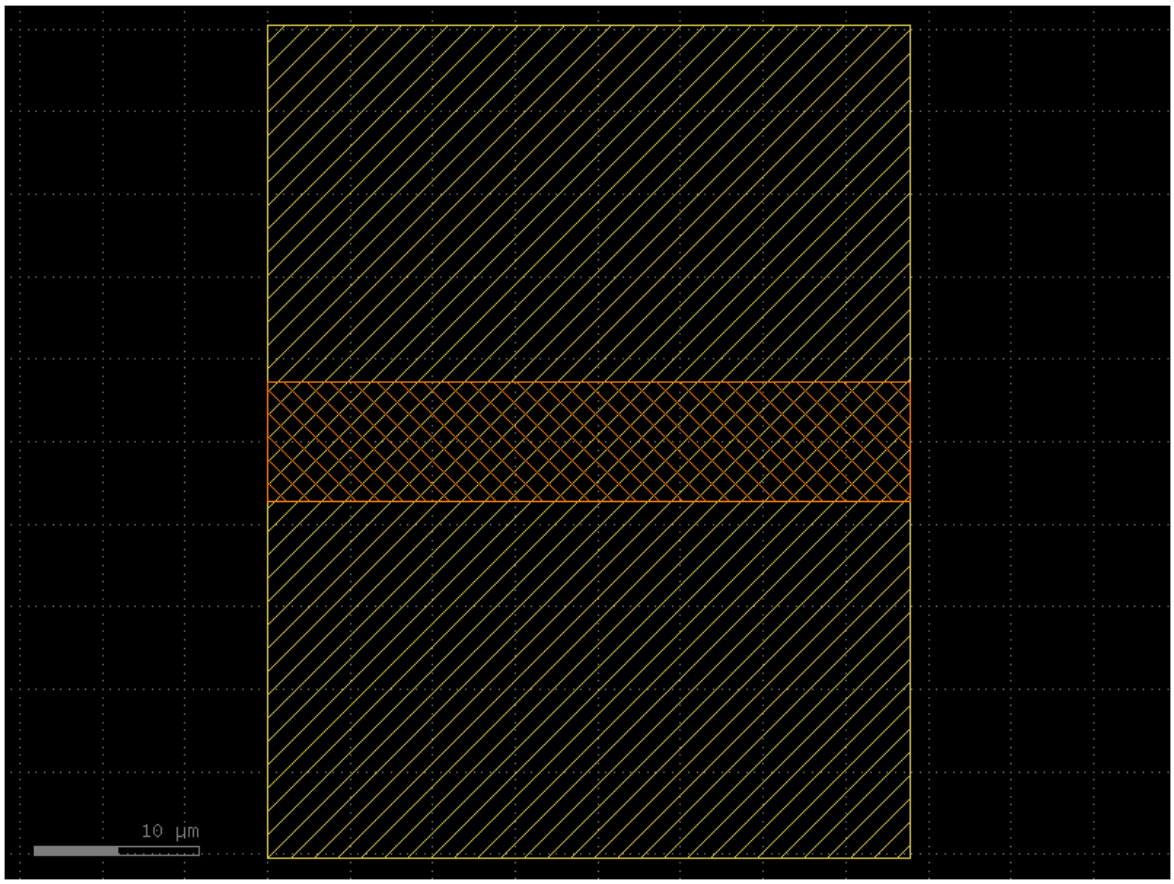
A single connection line segment is created, which is used for both the top and bottom connections to balance the layout.

In [20]:
```
connection_length_wpd_blc_one_leg = blc_1_ref.ports["e1"].center[1] -
blc_2_ref.ports["e4"].center[1] - (wpd_ref.ports["e2"].center[1] -
wpd_ref.ports["e3"].center[1])

connection_wpd_blc = ihp.cells.tline(
    length=snap_to_grid(connection_length_wpd_blc_one_leg / 2) + ihp.tech.nm,  # one grid step of margin
    signal_cross_section=signal_cross_section,
    ground_cross_section=ground_cross_section,
    Z0=Z0,
)

connection_wpd_blc.plot()
```

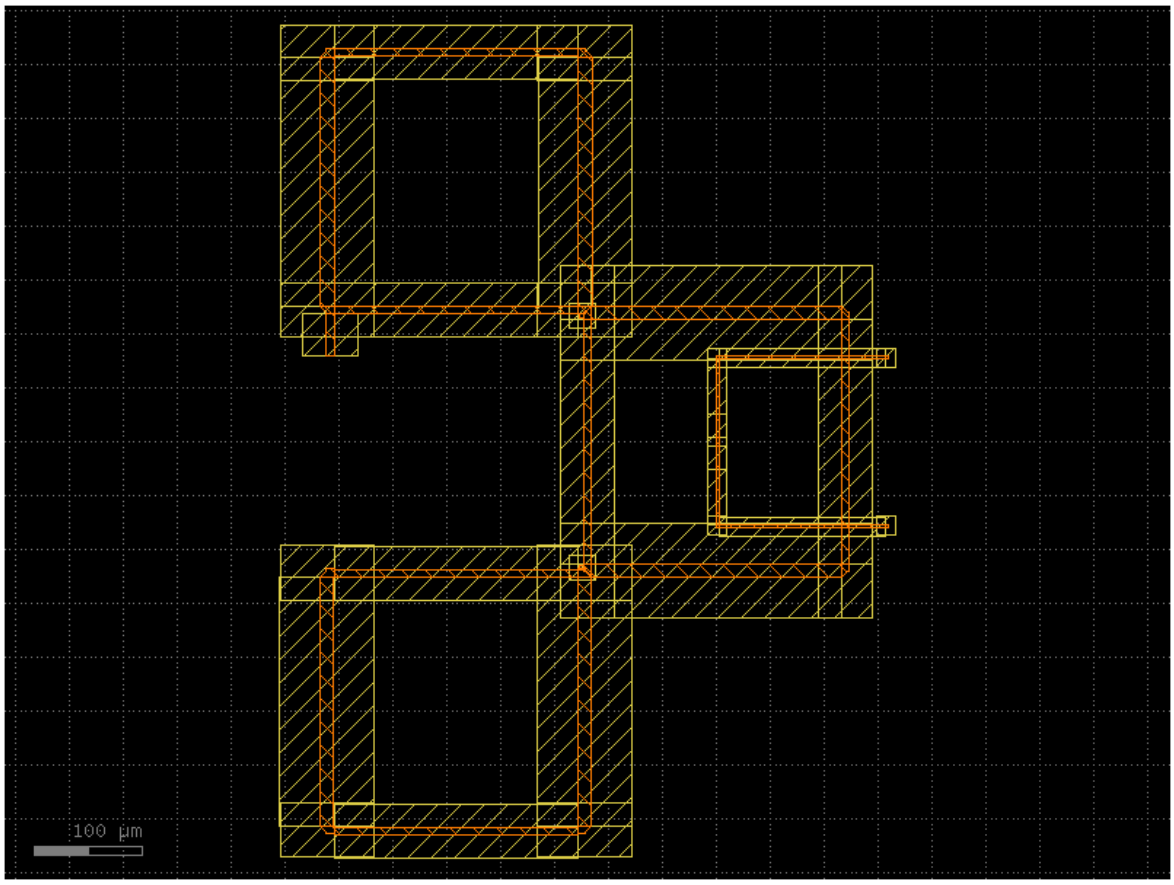
```
Calculated width: 7.2 um for Z0: 50 ohm
```



The top leg is placed first. One end connects to port `e1` of BLC_1, the coupler above.

In [21]:
```
connection_wpd_blc_top_ref = c.add_ref(connection_wpd_blc)
connection_wpd_blc_top_ref.connect("e1", blc_1_ref.ports["e1"])

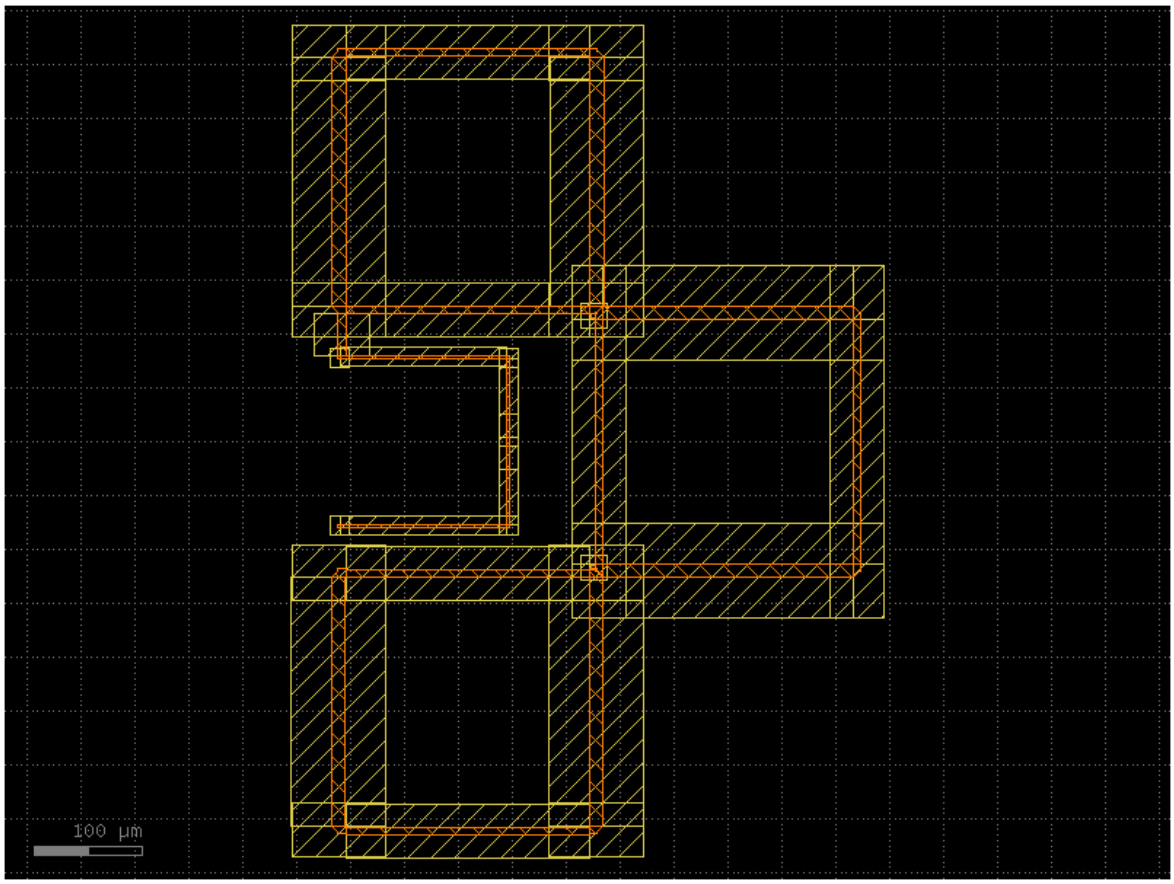
c.plot()
```



The WPD is attached with its output `e3` to the free end of the top leg, so its position follows from the couplers instead of being set by coordinates.

In [22]:
```
wpd_ref.connect("e3", connection_wpd_blc_top_ref.ports["e2"])

c.plot()
```

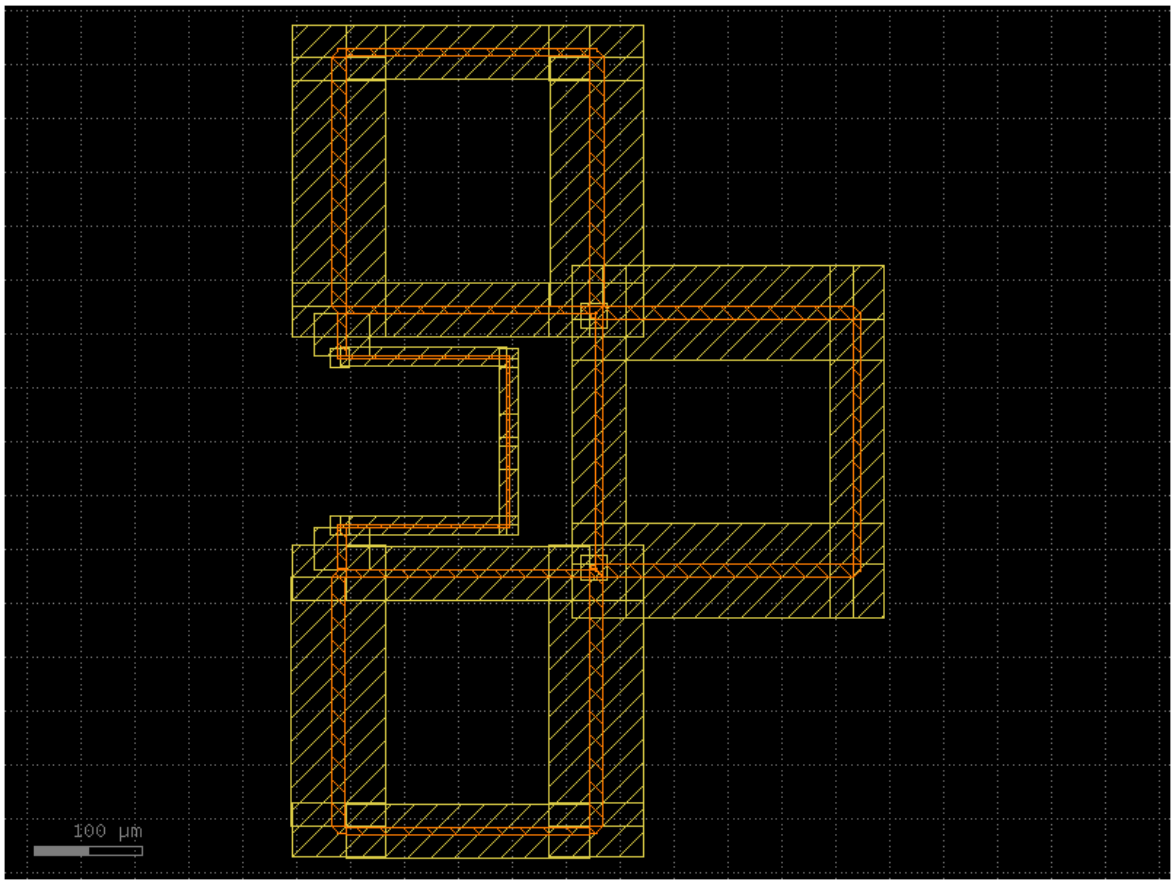


The bottom leg connects port `e4` of BLC_2 to the second WPD output `e2`. Both legs are the same line, so the two LO paths from the WPD to the couplers are identical.

In [23]:
```
connection_wpd_blc_bot_ref = c.add_ref(connection_wpd_blc)
connection_wpd_blc_bot_ref.connect("e1", blc_2_ref.ports["e4"])

wpd_ref.connect("e2", connection_wpd_blc_bot_ref.ports["e2"])

c.plot()
```



Hairpin Coupled-Line Bandpass Filter Design

The third and last RF device is the hairpin coupled-line bandpass filter (BPF) at the LO input, which limits the LO to the band around `f`. It is a third-order Butterworth filter built from coupled $\lambda_g/4$ line sections, which are folded into hairpins for a compact layout. The BPF is a custom cell of the add-on (`ihp.cells.hairpin_coupled_line_bandpass_filter`), generated from `f`, the filter order, type, and bandwidth defined above, and connects to the WPD input through a short line:

BPF at 160 GHz	Value
section length	238.8 µm
footprint	219 µm x 289 µm

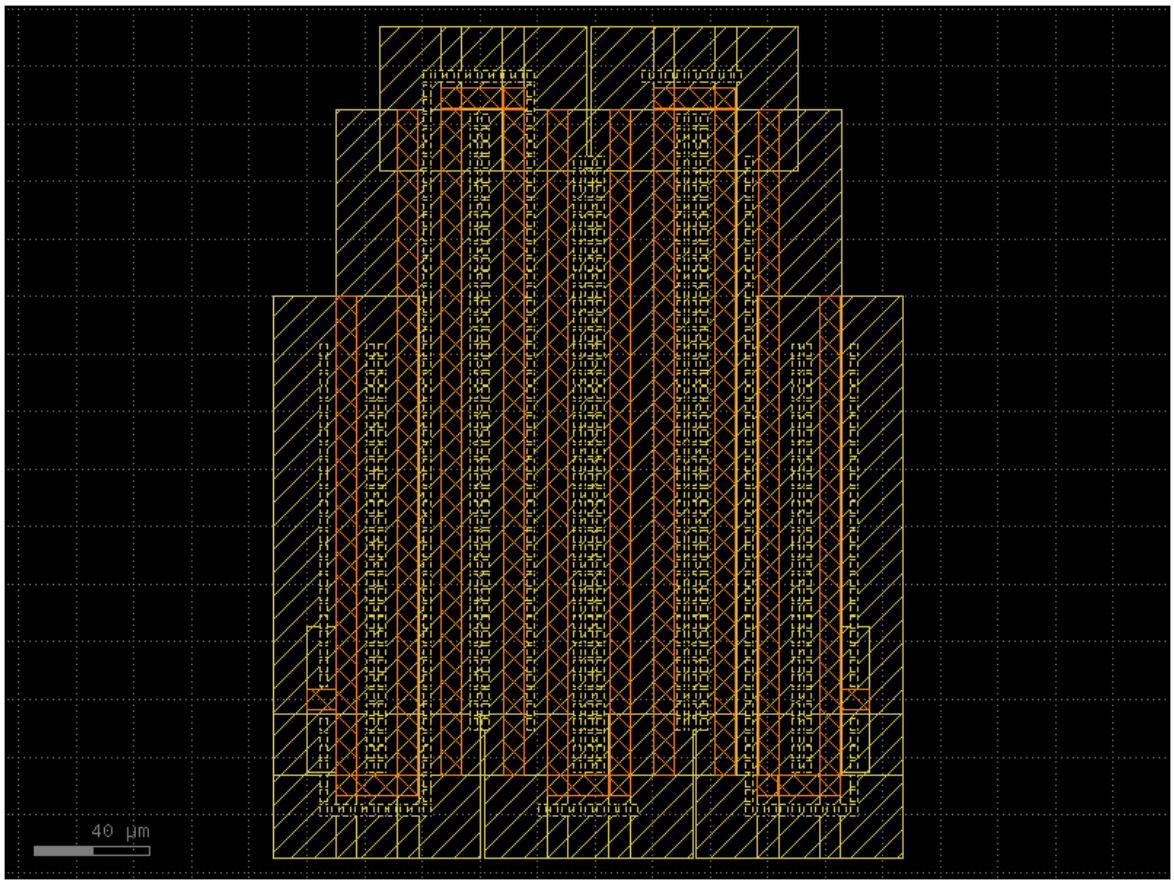
The code cell generates the BPF from the filter parameters defined above, adds it to the chip, and plots it.

In [24]:
```
bandpass_filter = ihp.cells.hairpin_coupled_line_bandpass_filter(
    frequency=f,
    bandwidth=bandwidth,
    order=order,
    filter_type=filter_type,
    ripple_dB=ripple_dB,
    connection_length=connection_length_bpf,
    signal_cross_section=signal_cross_section,
    ground_cross_section=ground_cross_section,
```

```
    Z0=Z0,
    e_r=e_r,
)

bandpass_filter_ref = c.add_ref(bandpass_filter)
bandpass_filter.plot()
```

```
Calculated width: 7.2 um for Z0: 50 ohm
Calculated width: 7.2 um for Z0: 50 ohm
Calculated width: 7.2 um for Z0: 50 ohm
Calculated width: 7.13 um for Z0: 50.25 ohm
Calculated Z0: 50.26863690347976 ohm for width: 7.13 um
Calculated width: 7.2 um for Z0: 50 ohm
Calculated width: 7.2 um for Z0: 50.0 ohm
Calculated Z0: 50.03094234385018 ohm for width: 7.2 um
Calculated width: 7.2 um for Z0: 50 ohm
```



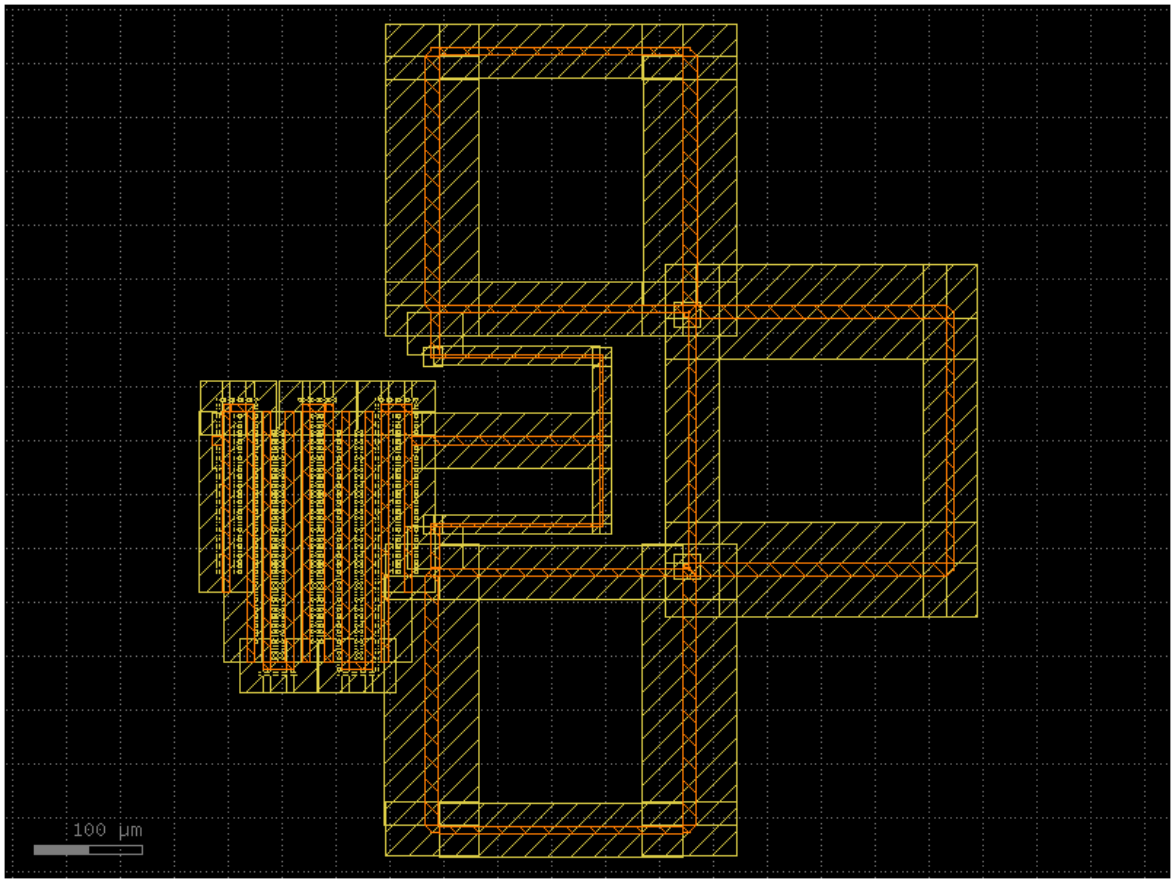
A 50 Ohm line of `connection_length_bpf_wpd`, 0.7 x $\lambda_g/4$ or 167 µm at 160 GHz, leads from the WPD input `e1` to the left, and the BPF is attached to its end. The LO then flows from the probe pad through the filter into the WPD.

In [25]:
```
connection_bpf_wpd = c.add_ref(ihp.cells.tline(
    length=connection_length_bpf_wpd,
    signal_cross_section=signal_cross_section,
    ground_cross_section=ground_cross_section,
    Z0=Z0,
))

connection_bpf_wpd.connect("e1", wpd_ref.ports["e1"])
bandpass_filter_ref.connect("e1", connection_bpf_wpd.ports["e2"])

c.plot()
```

```
Calculated width: 7.2 um for Z0: 50 ohm
```



Termination of BLC_3

With the RF devices complete, the fourth port of BLC_3 (`e3`), the coupler of the RF input, is terminated with 52 Ohm, which six-port operation requires. The resistor is contacted on Metal1, so the termination is built in four steps. The first cell adds a 30 µm long 50 Ohm line from that port and a via stack from TopMetal2 down to Metal1 at its end.

In [26]:
```
connection_blc_R_termination = ihp.cells.straight(
    length=CONNECTION_LEN_TERM,
    cross_section=signal_cross_section,
    width=ihp.cells.waveguides._calculate_width_from_Z0(Z0=Z0, e_r=e_r,
signal_cross_section=signal_cross_section,
ground_cross_section=ground_cross_section),
)
connection_blc_R_termination_ref = c.add_ref(connection_blc_R_termination)
connection_blc_R_termination_ref.connect("e1", blc_3_ref.ports["e3"])

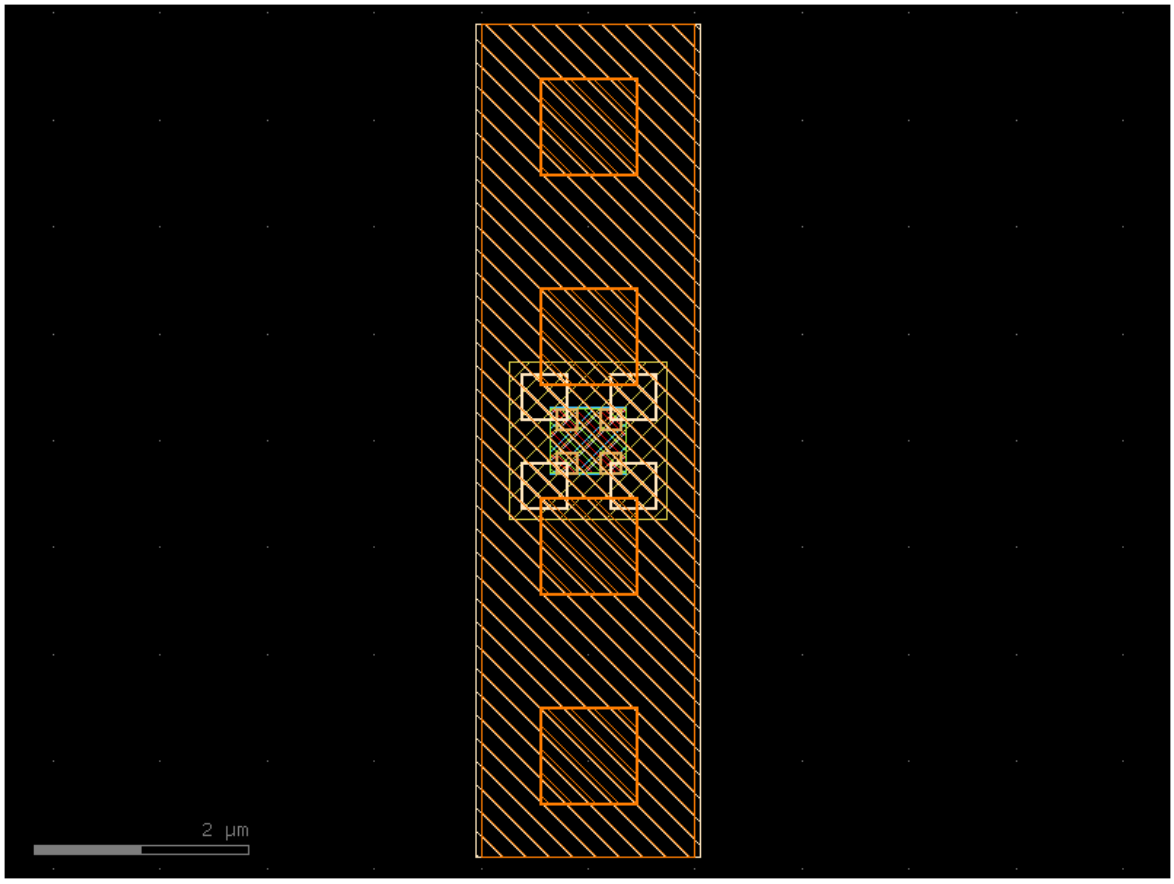
via_M1_TM2 = ihp.cells.via_stack(
    top_layer="TopMetal2",
    bottom_layer="Metal1",
    vn_columns=2,
    vn_rows=2,
    vt1_columns=2,
    vt1_rows=2,
    vt2_columns=1,
    vt2_rows=4,
)

via_M1_TM2.plot()
```

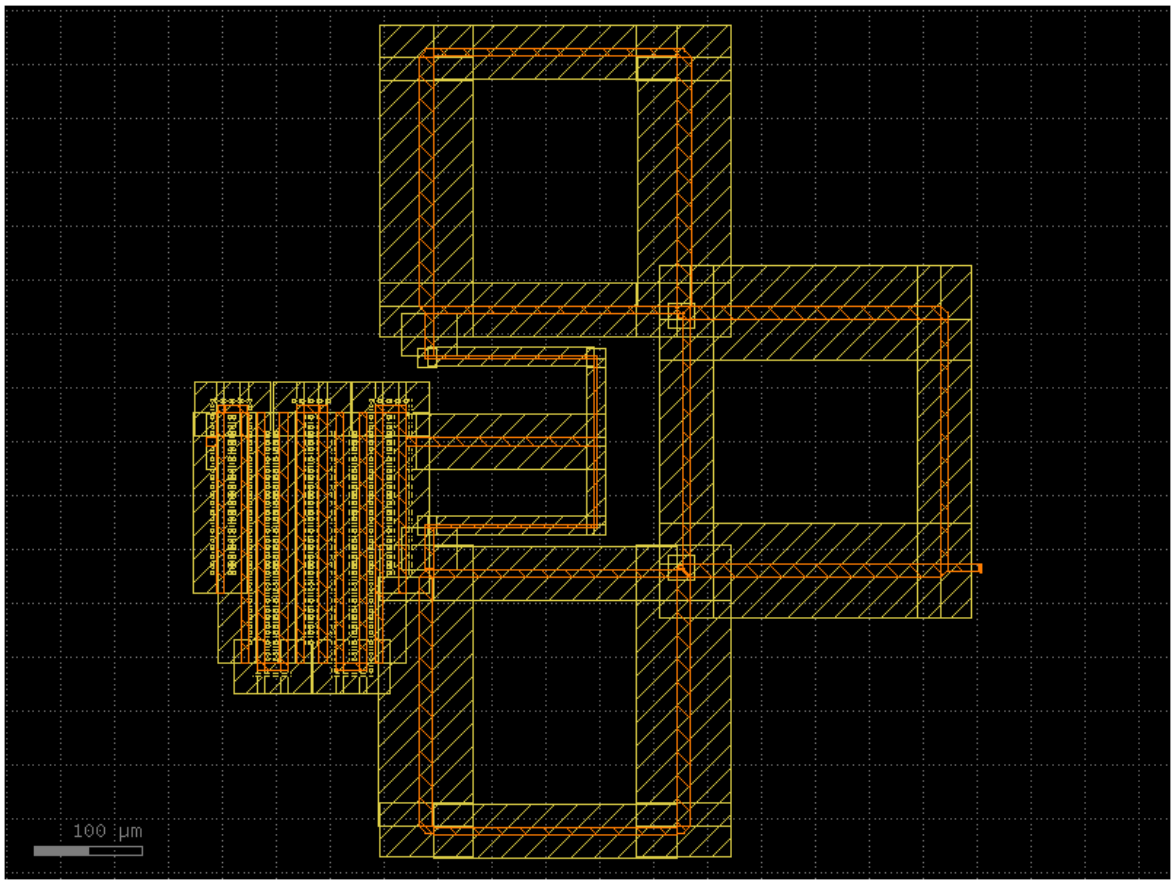
```
via_M1_TM2.ports["top"].orientation = 180
via_M1_TM2.ports["bottom"].orientation = 180
via_M1_TM2_ref = c.add_ref(via_M1_TM2)
via_M1_TM2_ref.connect("top", connection_blc_R_termination_ref.ports["e2"],
allow_width_mismatch=True)
```

```
Calculated width: 7.2 um for Z0: 50 ohm
```

Out[26]:
```
Six-Port-CAC: ports ['bottom', 'top'], KCell(name=via_stack_BLMetal1_TLTopMetal2
_VC2_VR2_VC2_VR2_VC1_VR4, ports=['bottom', 'top'], pins=[], instances=[], locked
=True, kcl=DEFAULT)
```

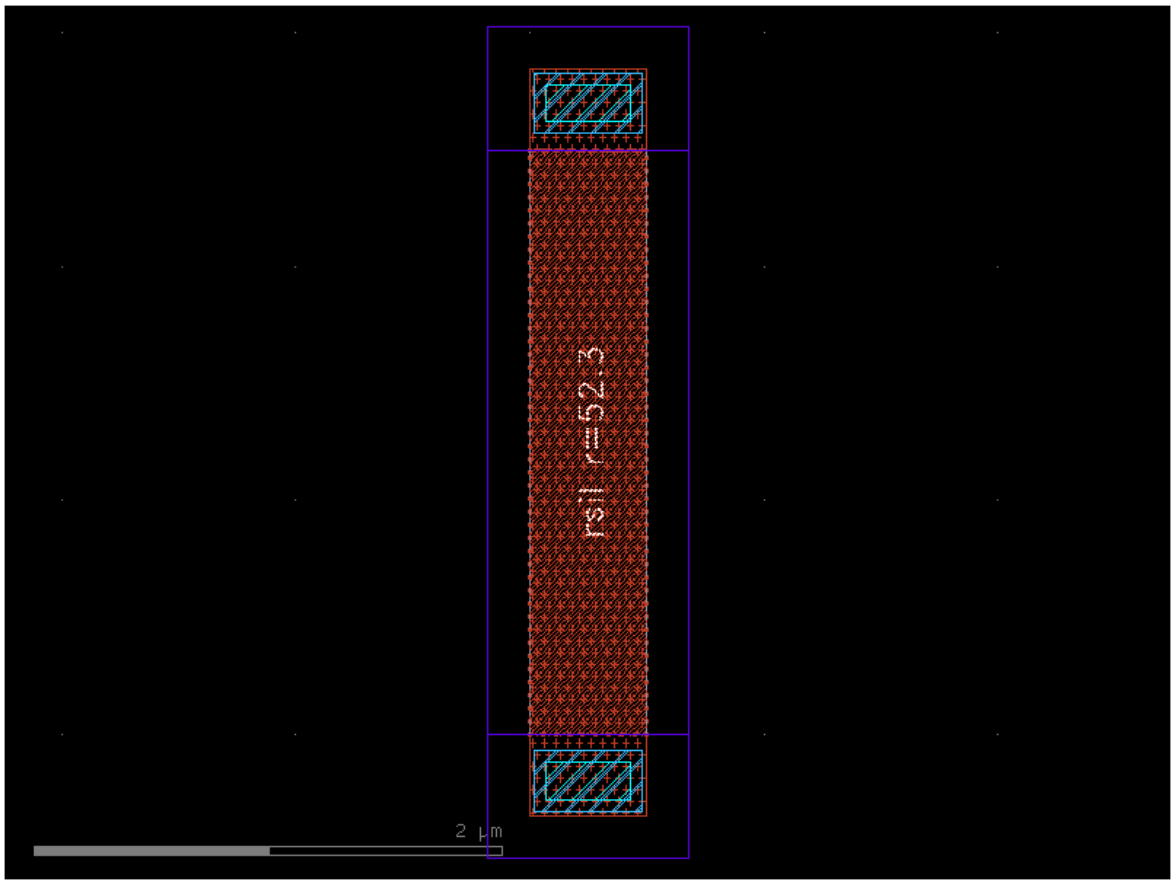


In [27]:
```
c.plot()
```



The termination resistor is a silicided poly resistor (`rsil`) of 0.5 µm x 2.5 µm (`RSIL_WIDTH` , `RSIL_LENGTH`), the same device as the input termination R_1 of the power detector.

In [28]:
```
R_termination = ihp.cells.rsil(
    width=RSIL_WIDTH,
    length=RSIL_LENGTH,
)
R_termination.plot()
```



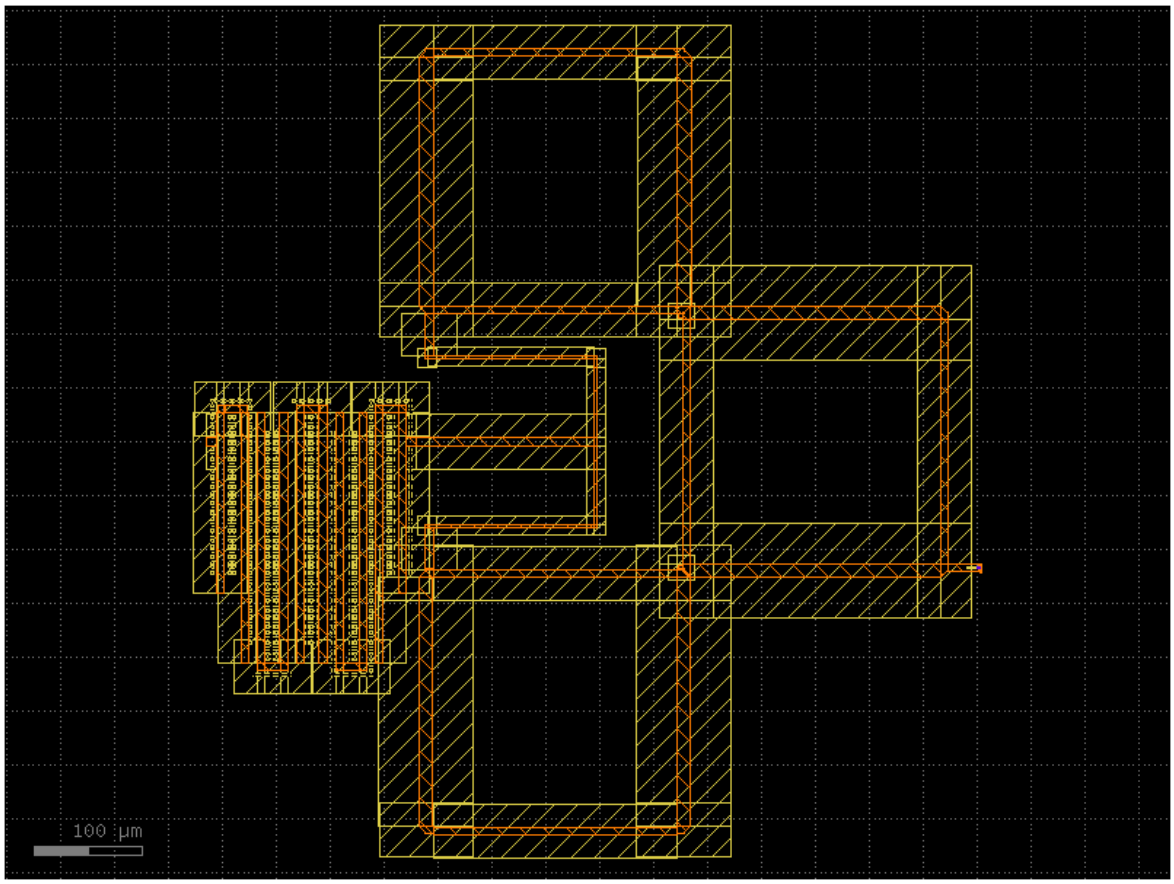
The resistor is connected to the bottom of the via stack. Its other end is taken up to Metal5 through a second via stack, where a short Metal5 line joins the ground plane (VSS) that the metal fill creates later.

In [29]:
```
R_termination_ref = c.add_ref(R_termination)
R_termination_ref.connect("e1", via_M1_TM2_ref.ports["bottom"],
allow_width_mismatch=True, allow_layer_mismatch=True)

via_M1_M5 = ihp.cells.via_stack(
    top_layer="Metal5",
    bottom_layer="Metal1",
    vn_columns=2,
    vn_rows=2,
)
via_M1_M5.ports["top"].orientation = 180
via_M1_M5.ports["bottom"].orientation = 0
via_M1_M5_ref = c.add_ref(via_M1_M5)
via_M1_M5_ref.connect("bottom", R_termination_ref.ports["e2"],
allow_width_mismatch=True, allow_layer_mismatch=True)

connection_R_termination_vss = ihp.cells.straight(
    length=10, # manual measurement to fit the design
    cross_section=ground_cross_section,
    width=2, # manual measurement to fit the design
)
connection_R_termination_vss_ref = c.add_ref(connection_R_termination_vss)
connection_R_termination_vss_ref.connect("e1", via_M1_M5_ref.ports["top"],
allow_width_mismatch=True, allow_layer_mismatch=True)

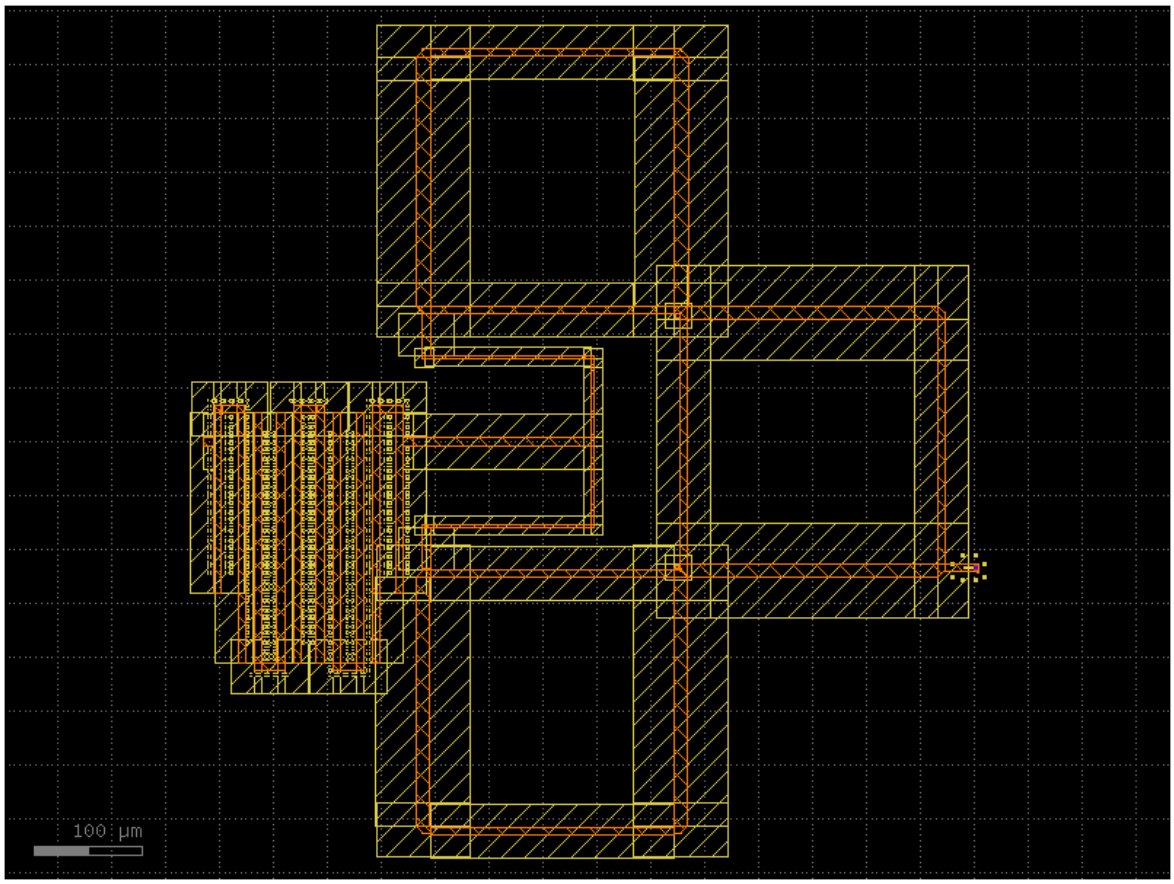
c.plot()
```



A Metal5 no-fill rectangle, 20 µm larger than the ground connection, keeps the ground plane away from the resistor and its vias, which reduces the parasitic capacitance of the termination.

In [30]:
```
# r termination metal5 nofill
c.add_ref(gf.components.rectangle(size=
(connection_R_termination_vss_ref.xsize+20,
connection_R_termination_vss_ref.ysize+20),
layer=ihp.tech.LAYER.Metal5nofill)).center =
connection_R_termination_vss_ref.center

c.plot()
```

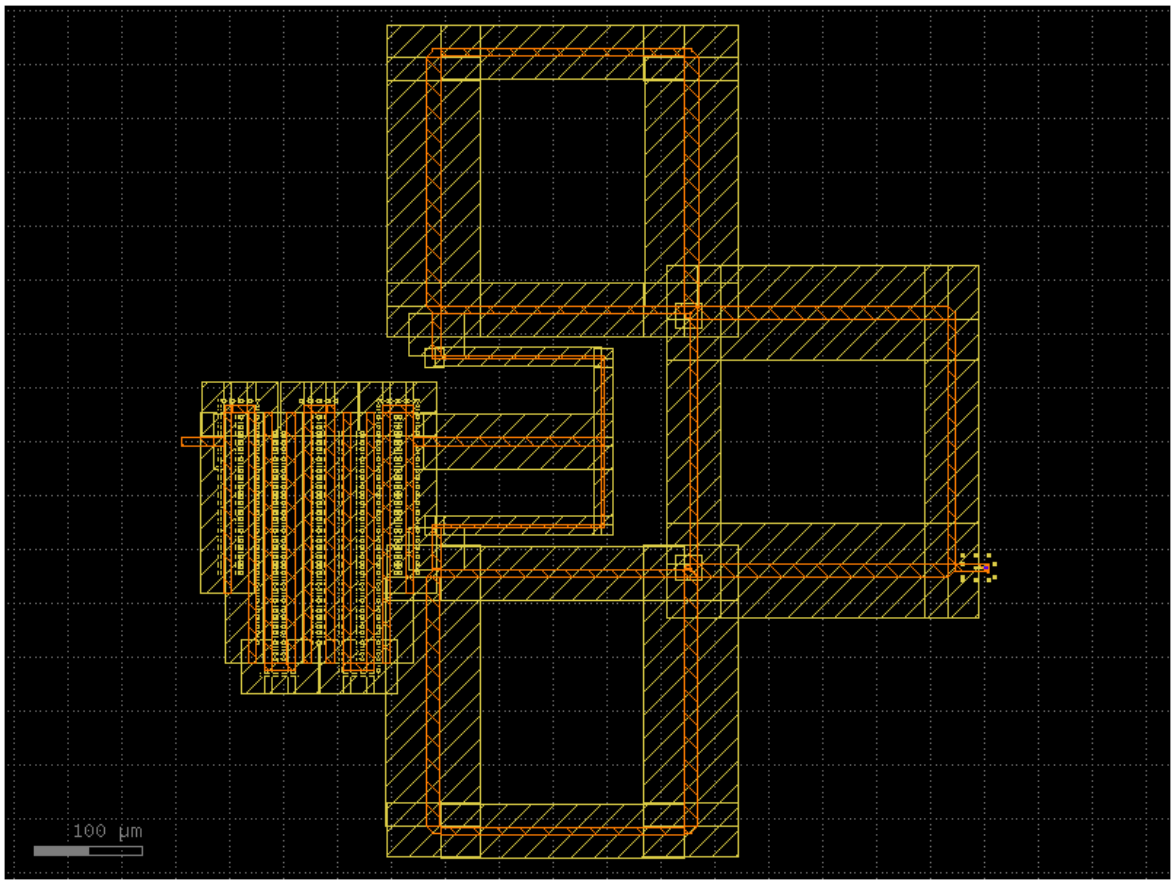


Back at the LO path, a 30 µm long 50 Ohm line extends the free port of the BPF towards the LO probe pads, which are placed next.

In [31]:
```
connection_bpf_pad = c.add_ref(ihp.cells.straight(
    length=CONNECTION_LEN_BPF_PAD,
    cross_section=signal_cross_section,
    width=ihp.cells.waveguides._calculate_width_from_Z0(Z0=Z0, e_r=e_r,
signal_cross_section=signal_cross_section,
ground_cross_section=ground_cross_section),
))
connection_bpf_pad.connect("e1", bandpass_filter_ref.ports["e2"],
allow_width_mismatch=True)

c.plot()
```

```
Calculated width: 7.2 um for Z0: 50 ohm
```



Probepads for LO Input

For the external LO input signal, a GSG (Ground-Signal-Ground) probepad array on the left side is added. This probepad:

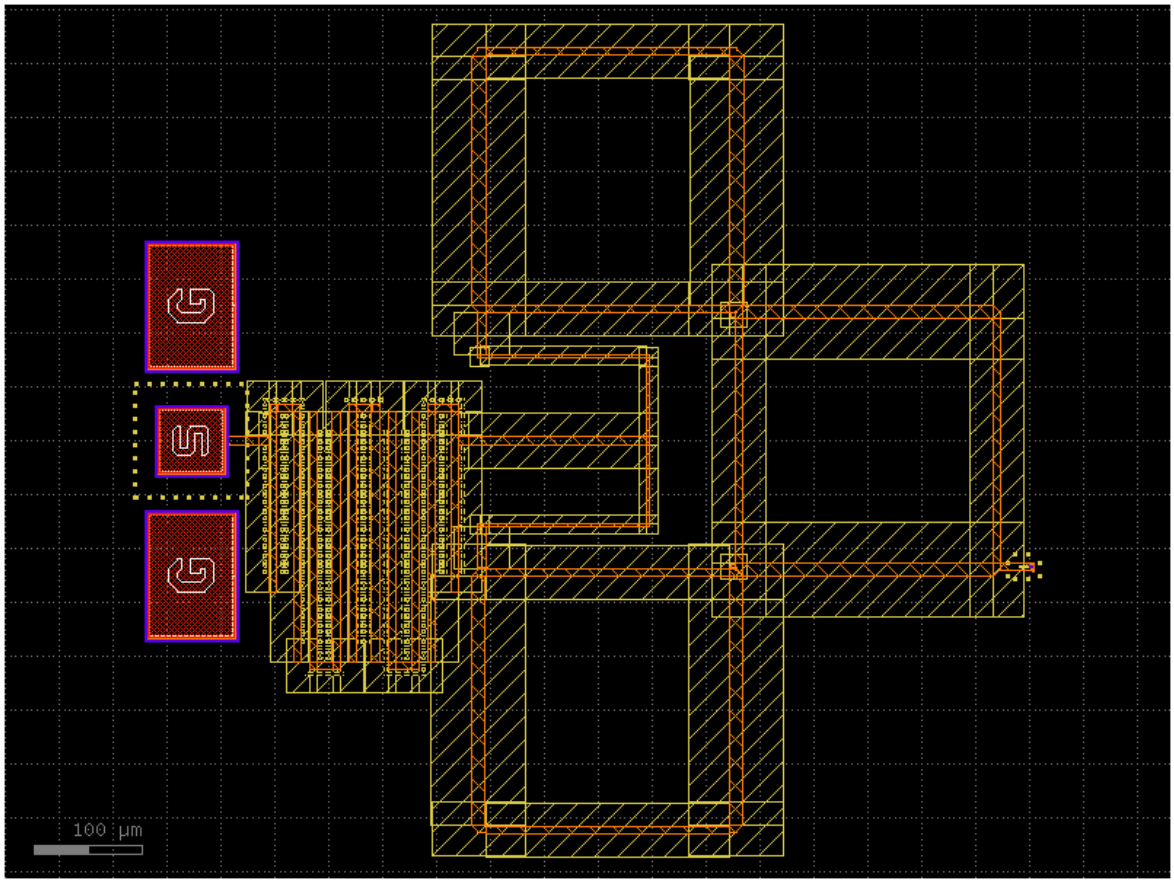
- connects to the bandpass filter input path through a transmission line.
- provides controlled 50-ohm probing interface.
- includes a Metal5 no-fill region to minimize parasitic capacitance.

The code cell creates a GSG pad array (`bondpad_array`) with a pitch of 125 µm and its ground pads tied to the substrate, rotates it by -90° into a vertical column, and connects its signal pad to the end of the line from the filter. It also adds a 105 µm square Metal5 no-fill region at the center of the array, and adds the ports of the array to the chip with the prefix `probe_left_` , which the pad labels at the end of the layout use.

In [32]:
```
probe_left = c.add_ref(ihp.cells.bondpad_array(
    config="GSG",
    pitch=PROBE_PITCH,
    width_ground=PROBE_GROUND_WIDTH,
    width_signal=PROBE_SIGNAL_SIZE,
    length_signal=PROBE_SIGNAL_SIZE,
    signal_cross_section="topmetal1_routing",
    ground_cross_section="metal1_routing",
    ground_connection="psub",
)).rotate(-90)
probe_left.center = bandpass_filter_ref.ports["e2"].center
probe_left.xmax = bandpass_filter_ref.ports["e2"].center[0] - 15
probe_left.connect("e1", connection_bpf_pad.ports["e2"],
```

```
allow_width_mismatch=True)

c.add_ref(gf.components.rectangle(size=(NOFILL_GSG_SIZE, NOFILL_GSG_SIZE),
layer=ihp.tech.LAYER.Metal5nofill)).center = probe_left.center
c.add_ports(probe_left.ports, prefix="probe_left_")
c.plot()
```



Probepads for RF Input

For the external RF input signal, a GSG (Ground-Signal-Ground) probepad array on the right side is added. This probepad:

- connects to the BLC_3 through a transmission line.
- is placed at the RF input node of the six-port network.
- includes a Metal5 no-fill region to minimize parasitic capacitance.

Two 50 Ohm line pieces lead from port `e2` of BLC_3 to the right, the first three line widths long, the second `CONNECTION_LEN_BLC_PAD` + `CONNECTION_LEN_BLC_EXTRA` minus one line width. The GSG array at their end is built like the one of the LO input, with the port prefix `probe_right_`.

In [33]:
```
width_Z0 = ihp.cells.waveguides._calculate_width_from_Z0(Z0=Z0, e_r=e_r,
signal_cross_section=signal_cross_section,
ground_cross_section=ground_cross_section)

connection_blc_pad_1 = c.add_ref(ihp.cells.straight(
    length=width_Z0 * 3,
    cross_section=signal_cross_section,
    width=width_Z0,
```

```
))
connection_blc_pad_1.connect("e1", blc_3_ref.ports["e2"])

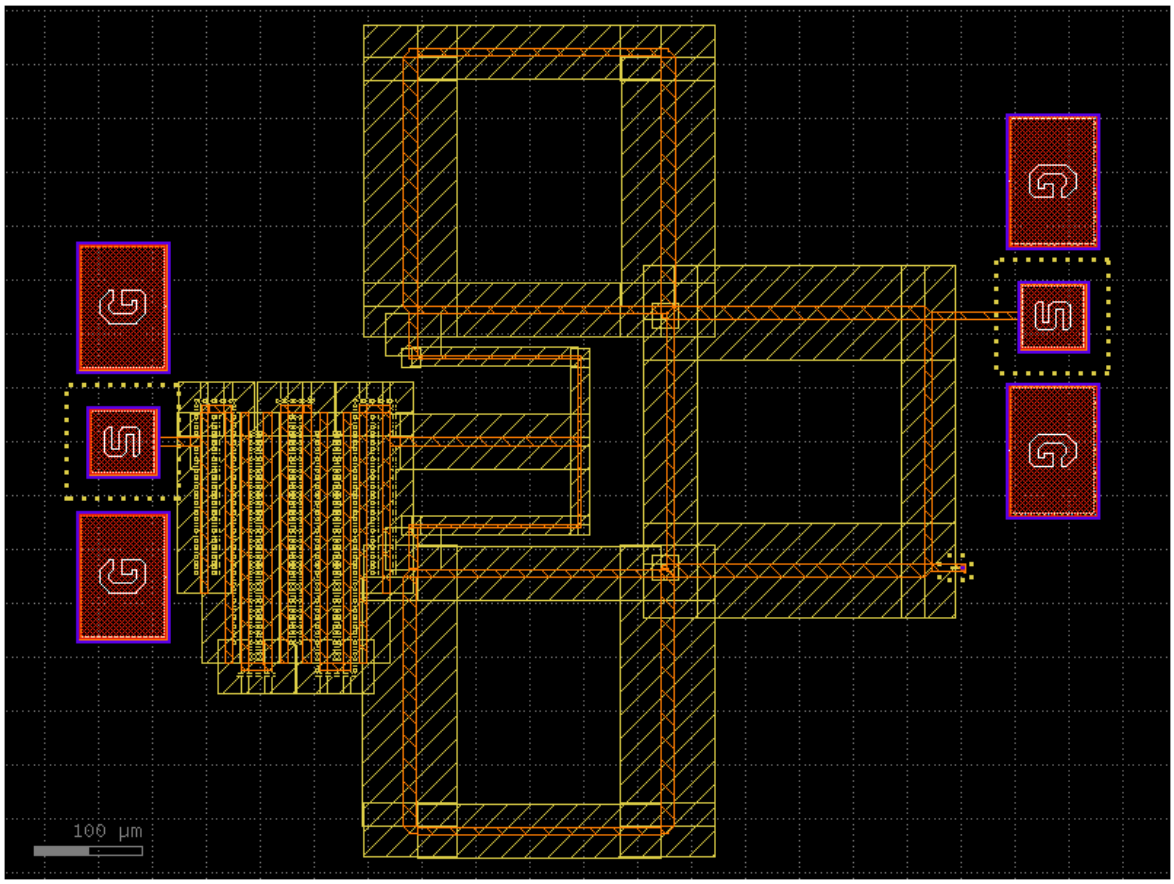
connection_blc_pad_2 = c.add_ref(ihp.cells.straight(
    length=round(CONNECTION_LEN_BLC_PAD + CONNECTION_LEN_BLC_EXTRA - width_Z0,
2),
    cross_section=signal_cross_section,
    width=width_Z0,
))
connection_blc_pad_2.connect("e1", connection_blc_pad_1.ports["e2"])

# probe pads right
probe_right = c.add_ref(ihp.cells.bondpad_array(
    config="GSG",
    pitch=PROBE_PITCH,
    width_ground=PROBE_GROUND_WIDTH_RIGHT,
    width_signal=PROBE_SIGNAL_SIZE,
    length_signal=PROBE_SIGNAL_SIZE,
    signal_cross_section="topmetal1_routing",
    ground_cross_section="metal1_routing",
    ground_connection="psub",
)).rotate(-90)
probe_right.connect("e1", connection_blc_pad_2.ports["e2"],
allow_width_mismatch=True)

c.add_ref(gf.components.rectangle(size=(NOFILL_GSG_SIZE, NOFILL_GSG_SIZE),
layer=ihp.tech.LAYER.Metal5nofill)).center = probe_right.center
c.add_ports(probe_right.ports, prefix="probe_right_")

c.plot()
```

```
Calculated width: 7.2 um for Z0: 50 ohm
```



RF Interconnects to the Power Detectors

The four six-port outputs (ports `e2` and `e3` of BLC_1 and BLC_2) are extended by a short straight TopMetal2 line each. Together with the `RFIN_PD` long feed line inside the power detector cell, they bridge the `RFIN_GAP` between the BLC and the detector input. The power detectors are later placed by connecting their `rfin` port to the far end of these lines.

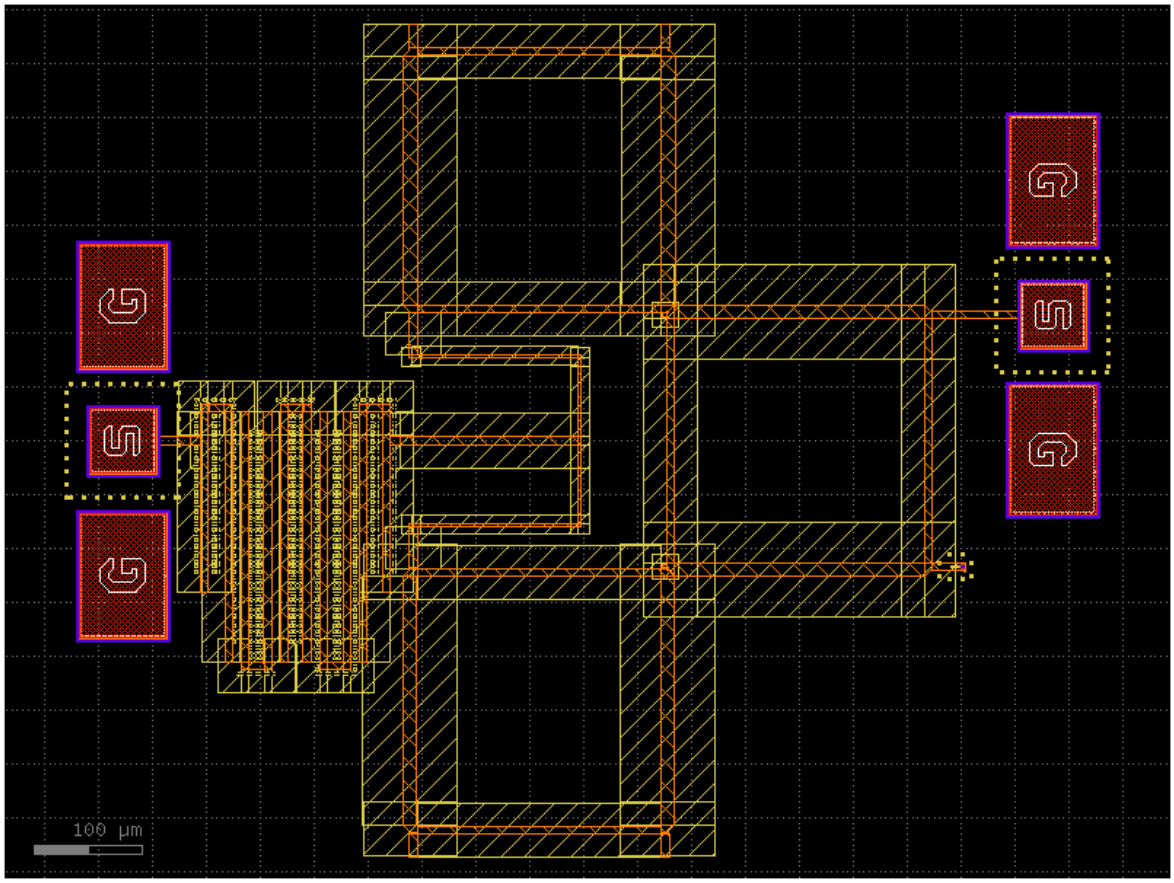
In [34]:
```
rfin_connection_pd1 = c.add_ref(ihp.cells.straight(
    length=RFIN_GAP - RFIN_PD,
    cross_section=signal_cross_section,
    width=blc_1_ref.ports["e2"].width,
))
rfin_connection_pd1.connect("e1", blc_1_ref.ports["e2"])

rfin_connection_pd2 = c.add_ref(ihp.cells.straight(
    length=RFIN_GAP - RFIN_PD,
    cross_section=signal_cross_section,
    width=blc_1_ref.ports["e3"].width,
))
rfin_connection_pd2.connect("e1", blc_1_ref.ports["e3"],
allow_width_mismatch=True)

rfin_connection_pd3 = c.add_ref(ihp.cells.straight(
    length=RFIN_GAP - RFIN_PD,
    cross_section=signal_cross_section,
    width=blc_2_ref.ports["e3"].width,
))
rfin_connection_pd3.connect("e1", blc_2_ref.ports["e3"],
allow_width_mismatch=True)

rfin_connection_pd4 = c.add_ref(ihp.cells.straight(
    length=RFIN_GAP - RFIN_PD,
    cross_section=signal_cross_section,
    width=blc_2_ref.ports["e2"].width,
))
rfin_connection_pd4.connect("e1", blc_2_ref.ports["e2"],
allow_width_mismatch=True)

c.plot()
```



Measurement Bondpads for Power Detectors

The RF network is now completed, but the six-port outputs feed into the above-designed power detectors, which demodulate the signal for direct waveform sampling and measurements. Since we are dealing with "low" frequency signals now, we can add bondpads instead of probepads, which are then bonded to a PCB for convenient measurement setup.

The top and bottom bondpad arrays are arranged as an SSGSGSS configuration and have the following pinout: VREF, VOUT, VSS, VDD, VSS, VOUT, VREF This means that per bondpad array, two power detectors are connected. The bondpads are placed just outside the power detectors, so their distance to the six-port follows from `RFIN_GAP`, the power detector height, and `PROBE_PD_GAP`, and scales with the design frequency. Metal5 rectangles are placed around each signal bondpad to minimize parasitic capacitance.

The first code cell instantiates the power detector once to know its height, moves the chip so that its lower left corner lies at the origin, and places the top SSGSGSS array centered above BLC_1, at the distance `RFIN_GAP`/2 + detector height + `PROBE_PD_GAP`. Metal5 no-fill regions around the VDD pad and on both sides of the array reduce the capacitance of the pads to the ground plane.

In [35]:
```
# the power detector height sets the distance of the bondpads to the six-port
pd = powdet_sbd()

# snap whole chip on grid
c.xmin = 0
c.ymin = 0
```

```
# probe pads top, positioned to clear PD extent
pd_height = pd.ysize
chip_center = c.center
probe_top = c.add_ref(ihp.cells.bondpad_array(
    config="SSGSGSS",
    signal_cross_section="topmetal1_routing",
    ground_cross_section="metal1_routing",
    ground_connection="psub",
))
probe_top.center = chip_center
probe_top.ymin = snap_to_grid(blc_1_ref.ymax + RFIN_GAP / 2 + pd_height +
PROBE_PD_GAP)
probe_top.xmin = snap_to_grid(probe_top.xmin)

# no fill VDD
c.add_ref(gf.components.rectangle(size=(NOFILL_VDD_SIZE, NOFILL_VDD_SIZE),
layer=ihp.tech.LAYER.Metal5nofill)).center = probe_top.center
c.add_ports(probe_top.ports, prefix="probe_top_")

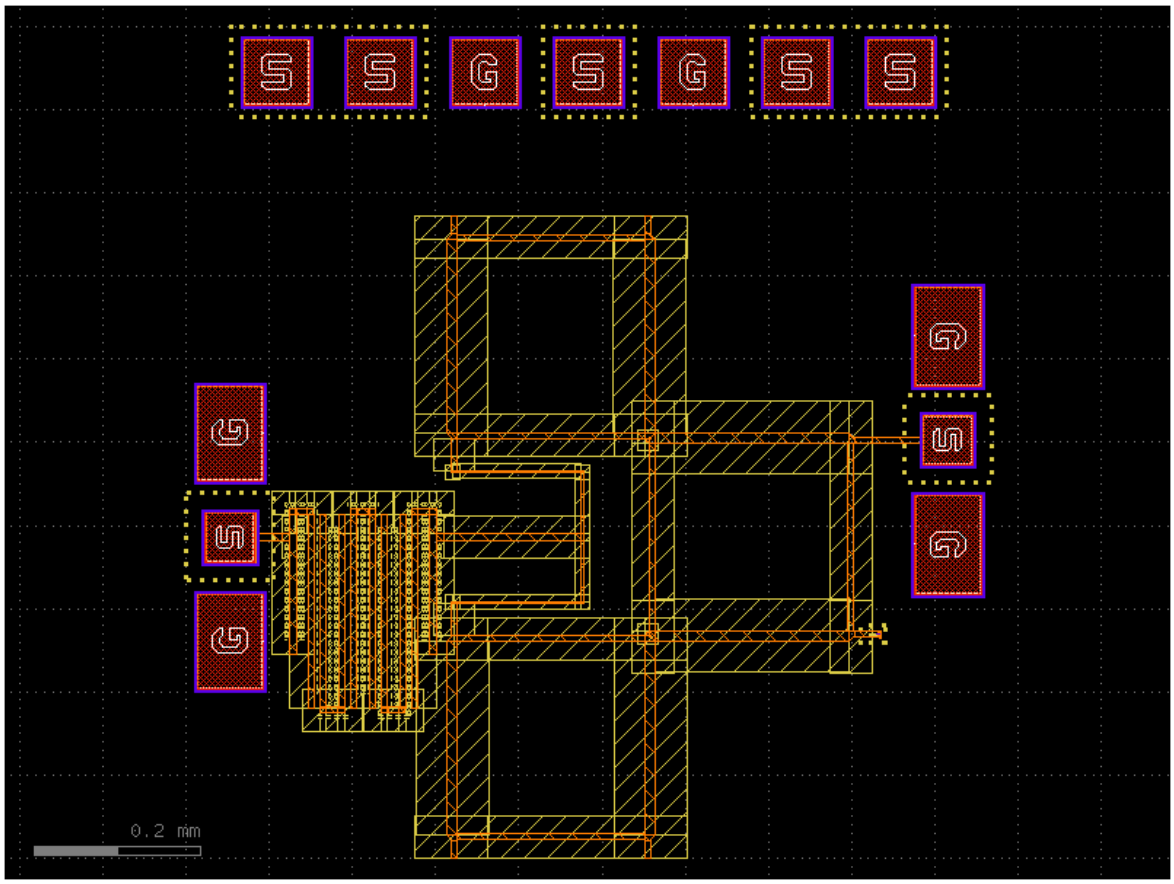
# no fill top
no_fill_top_left = c.add_ref(gf.components.rectangle(size=(NOFILL_SIDE_WIDTH,
NOFILL_VDD_SIZE), layer=ihp.tech.LAYER.Metal5nofill))
no_fill_top_left.xmin = probe_top.xmin - NOFILL_SIDE_OFFSET
no_fill_top_left.ymin = probe_top.ymin - NOFILL_SIDE_OFFSET

no_fill_top_right = c.add_ref(gf.components.rectangle(size=(NOFILL_SIDE_WIDTH,
NOFILL_VDD_SIZE), layer=ihp.tech.LAYER.Metal5nofill))
no_fill_top_right.xmax = probe_top.xmax + NOFILL_SIDE_OFFSET
no_fill_top_right.ymin = probe_top.ymin - NOFILL_SIDE_OFFSET

c.plot()
```

name	width	orientation	layer	center	port_type
vout	0.61	90.0	Metal4drawing (50/0)	(7.37, 40.84)	electrical
vref	2.0	90.0	Metal3drawing (30/0)	(5.47, 48.68)	electrical
rfin	7.2	270.0	TopMetal2drawing (134/0)	(0.0, -18.400000000000002)	electrical
vss	11.2	90.0	Metal5drawing (67/0)	(70.32000000000001, 79.8)	electrical
vdd	10.08	90.0	TopMetal1drawing (126/0)	(23.44, 79.8)	electrical

```
size_nofill_x: 116.68, size_nofill_y: 103.8
```



The bottom array is placed the same way below BLC_2, rotated by 180° so that its pads face the chip.

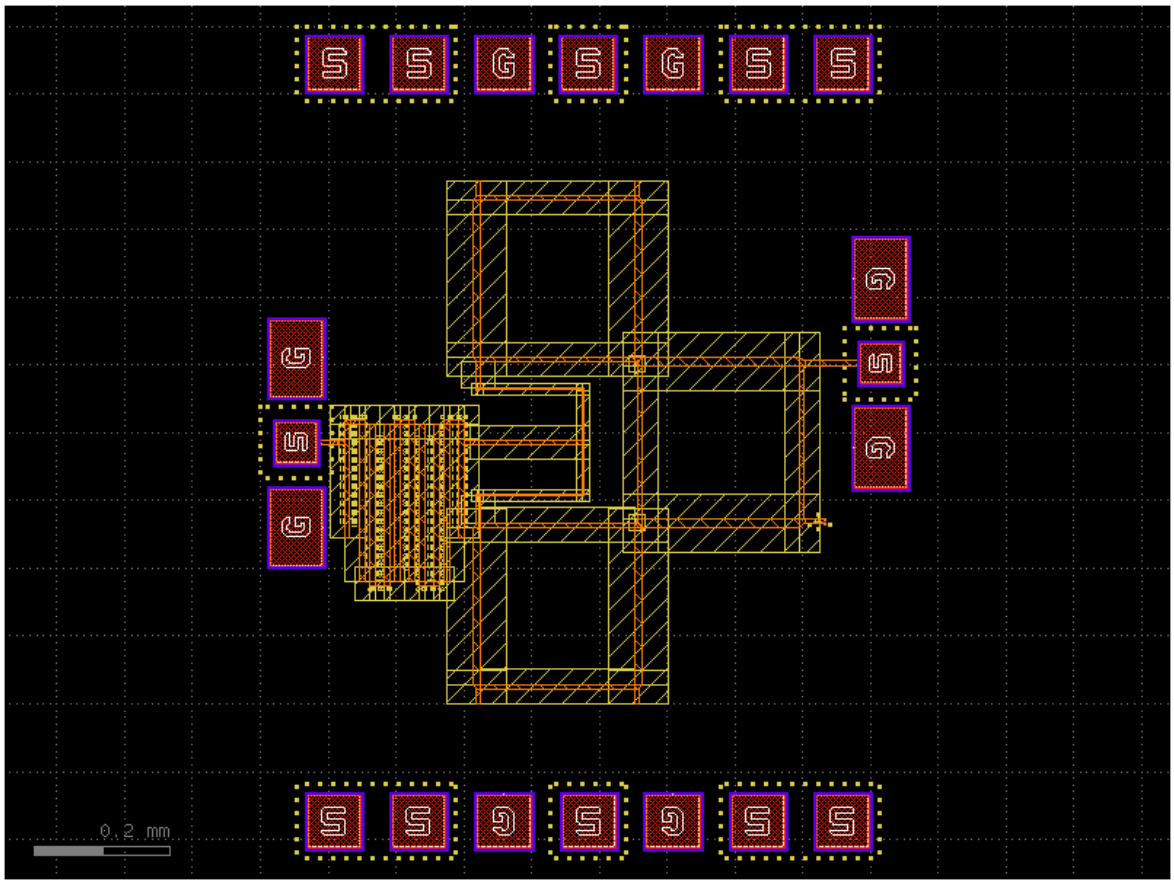
In [36]:
```
# probe pads bottom
probe_bottom = c.add_ref(ihp.cells.bondpad_array(
    config="SSGSGSS",
    signal_cross_section="topmetal1_routing",
    ground_cross_section="metal1_routing",
    ground_connection="psub",
)).rotate(180)
probe_bottom.center = chip_center
probe_bottom.ymax = snap_to_grid(blc_2_ref.ymin - RFIN_GAP / 2 - pd_height -
PROBE_PD_GAP)
probe_bottom.xmin = snap_to_grid(probe_bottom.xmin)

# no fill VDD
c.add_ref(gf.components.rectangle(size=(NOFILL_VDD_SIZE, NOFILL_VDD_SIZE),
layer=ihp.tech.LAYER.Metal5nofill)).center = probe_bottom.center
c.add_ports(probe_bottom.ports, prefix="probe_bottom_")

# no fill bottom
no_fill_bottom_left = c.add_ref(gf.components.rectangle(size=(NOFILL_SIDE_WIDTH,
NOFILL_VDD_SIZE), layer=ihp.tech.LAYER.Metal5nofill))
no_fill_bottom_left.xmin = probe_bottom.xmin - NOFILL_SIDE_OFFSET
no_fill_bottom_left.ymin = probe_bottom.ymin - NOFILL_SIDE_OFFSET

no_fill_bottom_right = c.add_ref(gf.components.rectangle(size=
(NOFILL_SIDE_WIDTH, NOFILL_VDD_SIZE), layer=ihp.tech.LAYER.Metal5nofill))
no_fill_bottom_right.xmax = probe_bottom.xmax + NOFILL_SIDE_OFFSET
no_fill_bottom_right.ymin = probe_bottom.ymin - NOFILL_SIDE_OFFSET

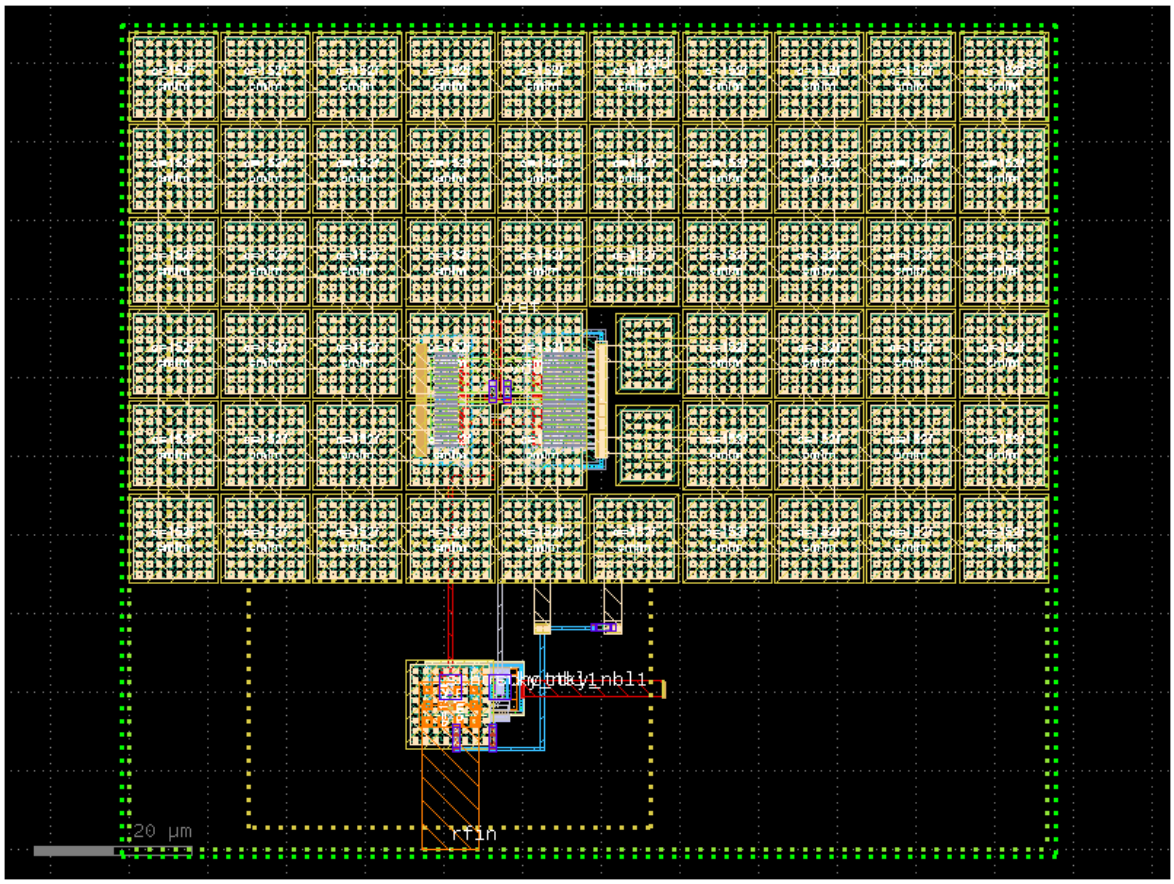
c.plot()
```



Instantiate Power Detector Module

The instance of the above-defined power detector module (powdet_sbd), created when placing the bondpads, is shown here. This detector will be used as the base template for all four detector channels on the chip.

In [37]:
```
pd.plot()
```



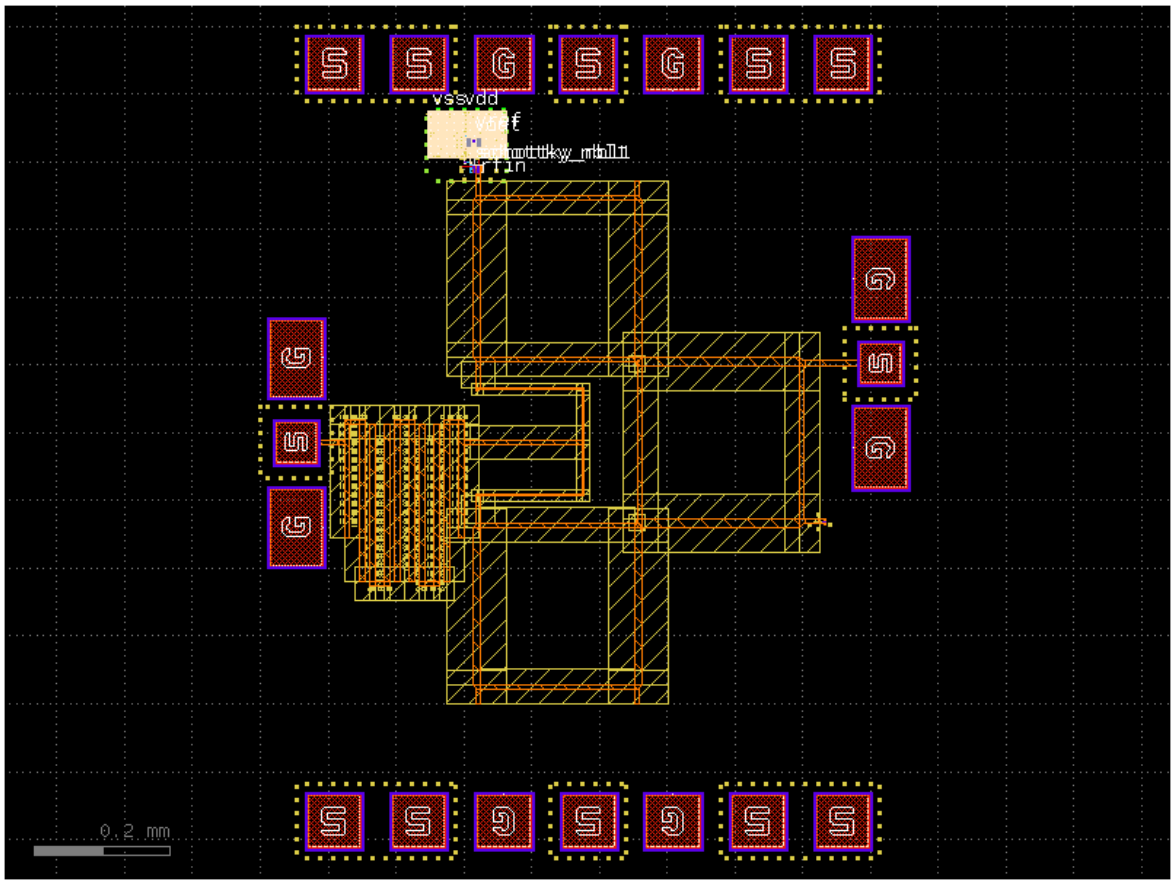
Place and Connect First Power Detector (PD1)

PD1 is instantiated using a mirrored detector variant and is placed by connectivity:

- Take the RF interconnect line at `blc_1_ref` port `e2` (`rfin_connection_pd1`)
- Attach PD1 `rfin` to the interconnect output

This anchors the PD1 location from the RF topology and keeps placement robust to geometry changes.

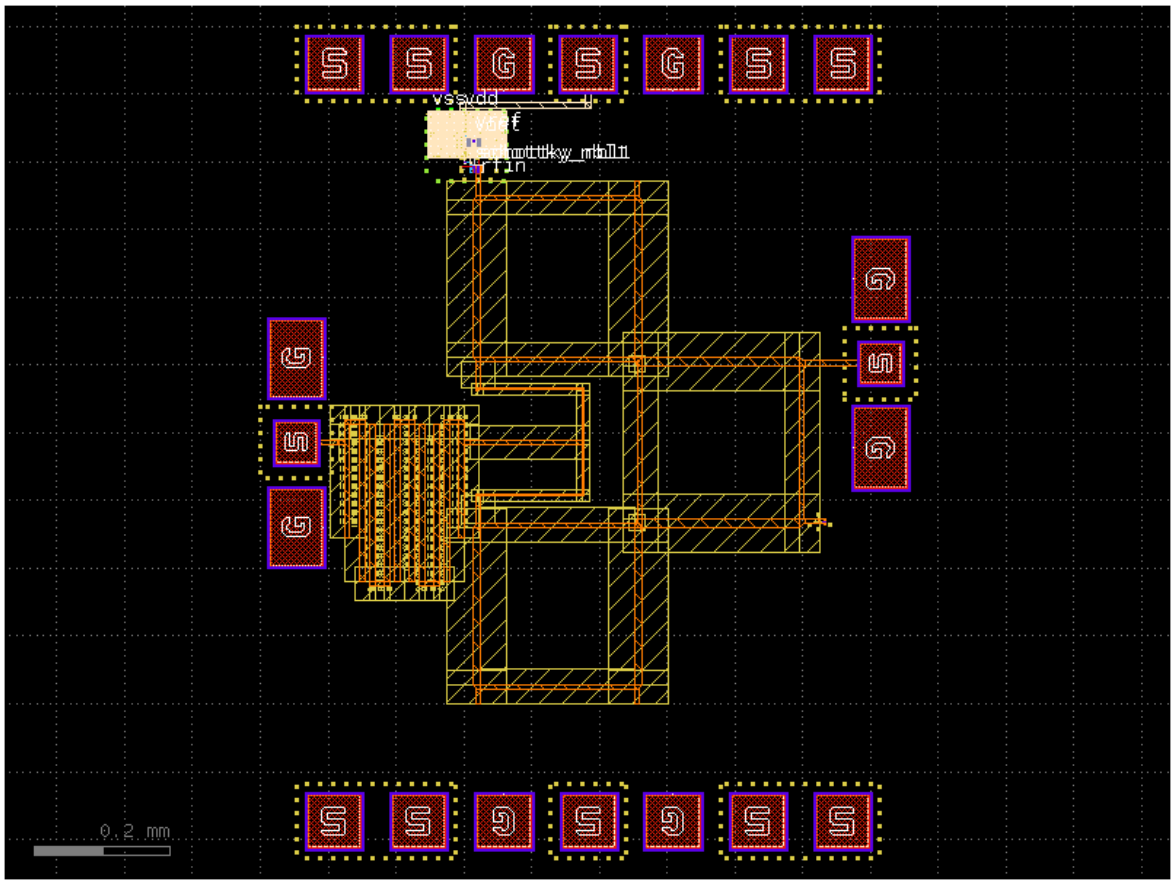
In [38]:
```
pd1_ref = c.add_ref(pd).mirror_x()
pd1_ref.connect("rfin", rfin_connection_pd1.ports["e2"],
allow_width_mismatch=True)
c.plot()
```



PD1 is now topology-placed via RF port connections. Next, route its supply and output nets to the top bondpads.

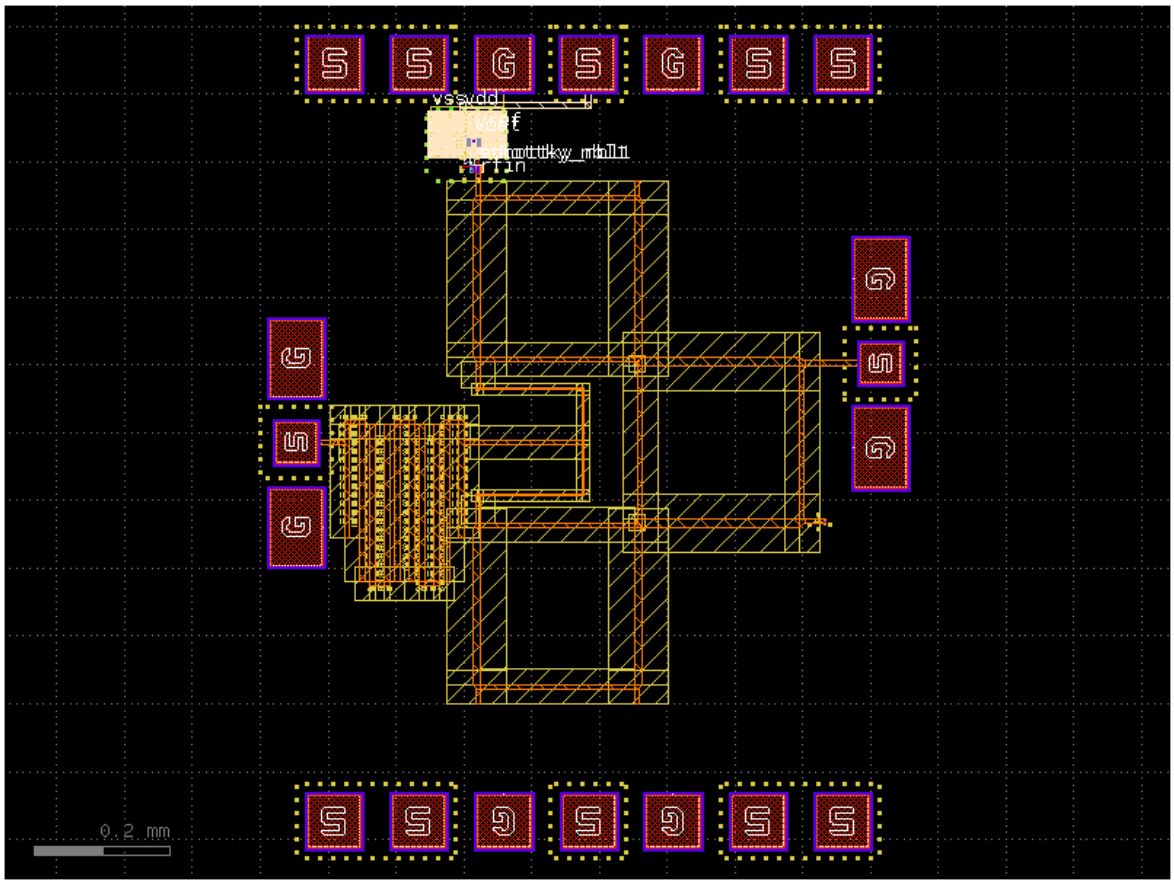
The first step connects PD1 `vdd` using the GDSFactory built-in electrical bundle router.

In [39]:
```
route_pd1_vdd = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd1_ref.ports["vdd"]],
    ports2=[probe_top.ports["e4"]],
    route_width=8,    # adjust for thicker connections
    layer=ihp.tech.LAYER.TopMetal1drawing,
    start_straight_length=10,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
c.plot()
```



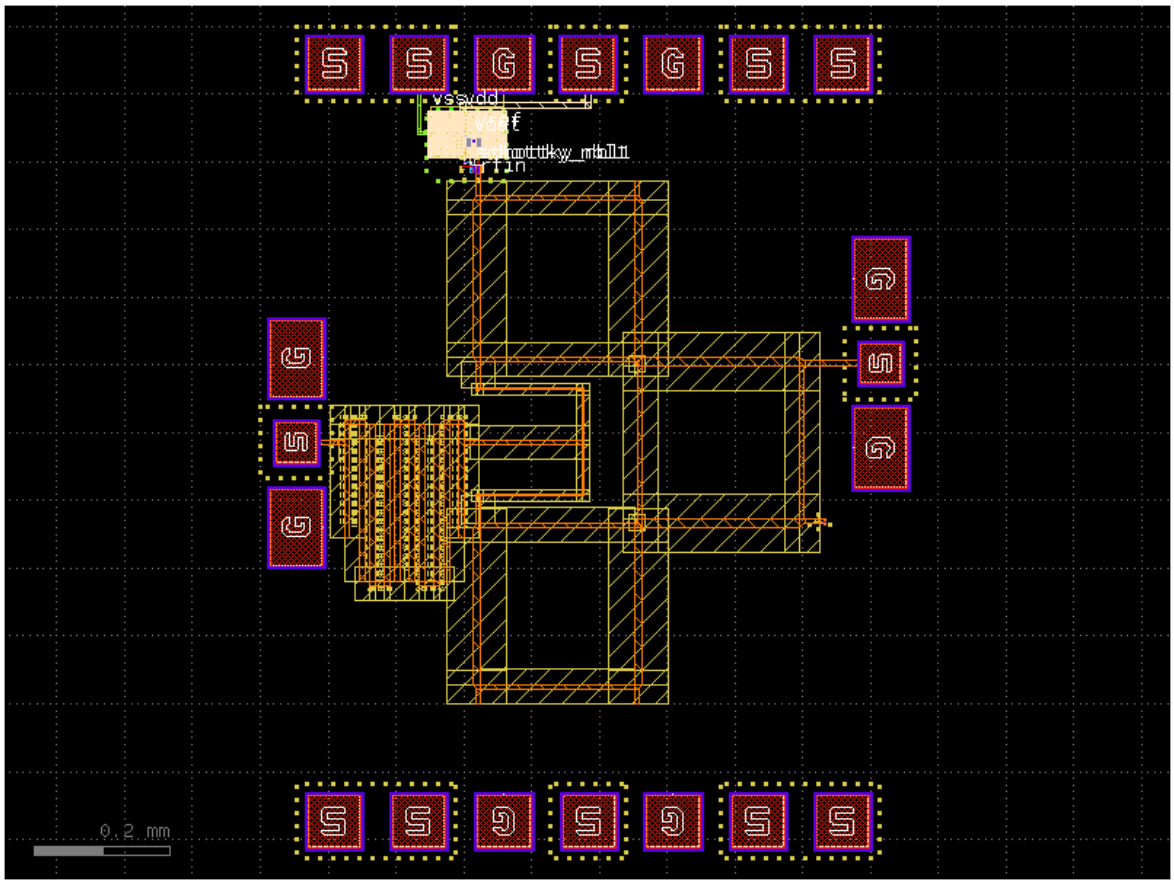
Next, PD1 `vss` is connected to the top bondpad using a Metal5 electrical route with a straight lead for clean routing.

In [40]:
```
route_pd1_vss = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd1_ref.ports["vss"]],
    ports2=[probe_top.ports["e3"]],
    route_width=2,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal5drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
c.plot()
```



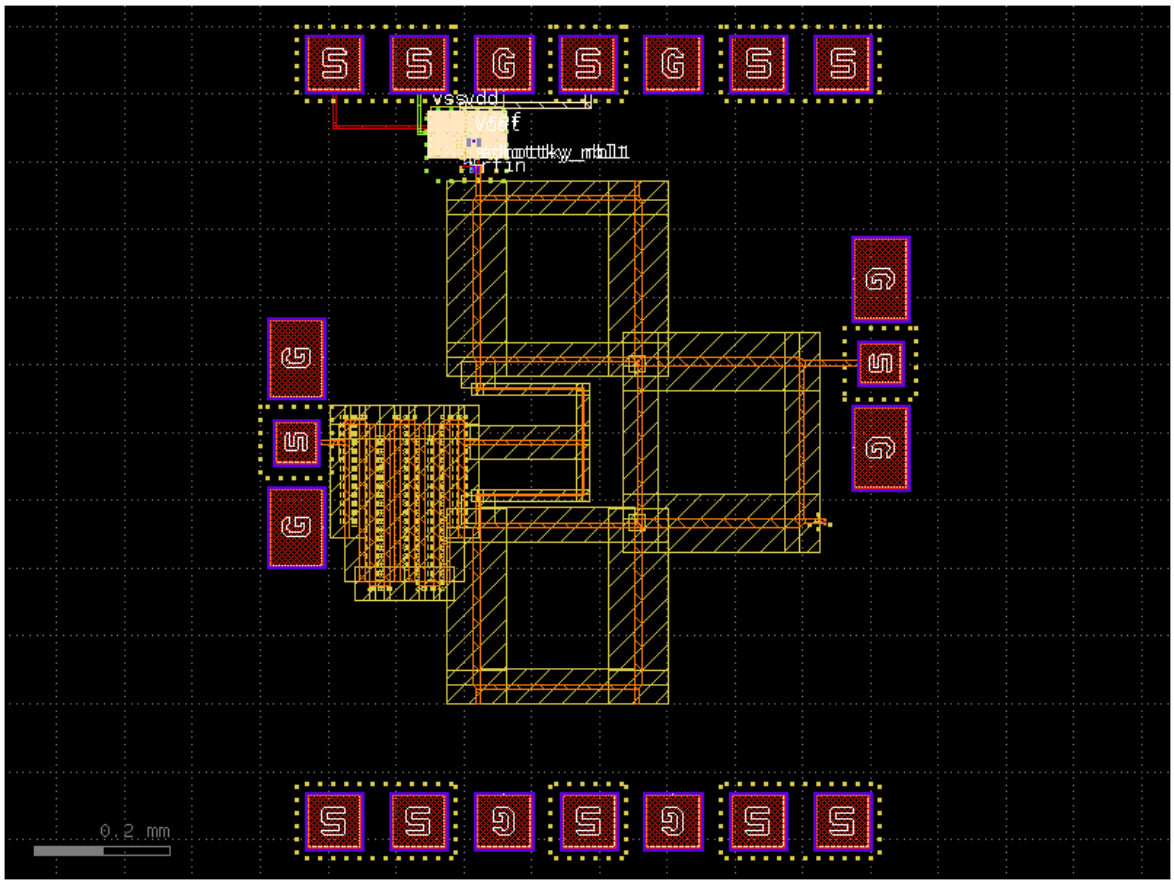
Afterwards, PD1 `vout` is connected to the top bondpad through a Metal4 route using a TopMetal1-to-Metal4 VIA stack.

In [41]:
```
via_M4_TM1 = ihp.cells.via_stack(
    top_layer="TopMetal1",
    bottom_layer="Metal4",
    vn_columns=10,
    vn_rows=3,
    vt1_columns=5,
    vt1_rows=1,
)
via_M4_TM1.ports["top"].orientation = 90
via_M4_TM1.ports["bottom"].orientation = 270
via_M4_TM1_ref = c.add_ref(via_M4_TM1)
via_M4_TM1_ref.connect("top", probe_top.ports["e2"], allow_width_mismatch=True,
allow_layer_mismatch=True)
via_M4_TM1_ref.move((0, via_M4_TM1_ref.ysize/2))
route_pd1_vout = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd1_ref.ports["vout"]],
    ports2=[via_M4_TM1_ref.ports["bottom"]],
    route_width=4,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal4drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
c.plot()
```



Finally, PD1 `vref` is connected to the top bondpad through a Metal3 route using a TopMetal1-to-Metal3 VIA stack.

In [42]:
```
via_M3_TM1 = ihp.cells.via_stack(
    top_layer="TopMetal1",
    bottom_layer="Metal3",
    vn_columns=10,
    vn_rows=3,
    vt1_columns=5,
    vt1_rows=1,
)
via_M3_TM1.ports["top"].orientation = 90
via_M3_TM1.ports["bottom"].orientation = 270
via_M3_TM1_ref = c.add_ref(via_M3_TM1)
via_M3_TM1_ref.connect("top", probe_top.ports["e1"], allow_width_mismatch=True,
allow_layer_mismatch=True)
via_M3_TM1_ref.move((0, via_M3_TM1_ref.ysize/2))
route_pd1_vref = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd1_ref.ports["vref"]],
    ports2=[via_M3_TM1_ref.ports["bottom"]],
    route_width=4,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal3drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
c.plot()
```



The RF and bias routing for PD1 is completed. Next, instantiate and route PD2 using the complementary orientation and adjacent BLC branch.

Place and Connect Second Power Detector (PD2)

PD2 is instantiated in its non-mirrored orientation and is placed by connectivity:

- Connect an RF interconnect from `blc_1_ref` port `e3` to PD2 `rfin`
- Route `vdd` and `vss` to top bondpads
- Route `vout` and `vref` through Metal4/Metal3 paths using via stacks to top bondpads

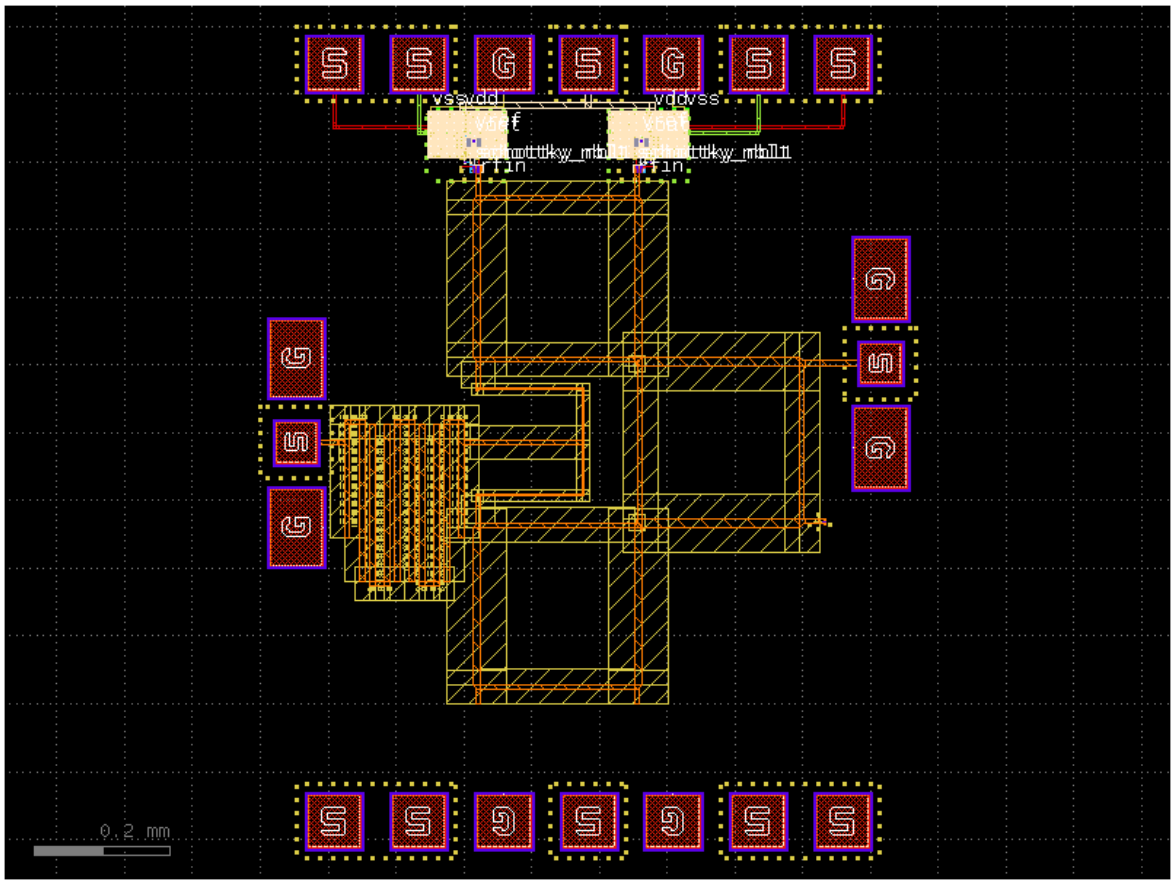
This keeps PD2 aligned with the surrounding network as dimensions scale.

In [43]:
```
pd2_ref = c.add_ref(pd)
pd2_ref.connect("rfin", rfin_connection_pd2.ports["e2"],
allow_width_mismatch=True)

# vdd connection of pd2 to probe top
route_pd2_vdd = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd2_ref.ports["vdd"]],
    ports2=[probe_top.ports["e4"]],
    route_width=8,    # adjust for thicker connections
    layer=ihp.tech.LAYER.TopMetal1drawing,
    start_straight_length=10,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
```

```
    separation=0,
)
# vss connection of pd2 to probe top
route_pd2_vss = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd2_ref.ports["vss"]],
    ports2=[probe_top.ports["e5"]],
    route_width=2,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal5drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
# vout connection of pd2 to probe top
via_M4_TM1_ref = c.add_ref(via_M4_TM1)
via_M4_TM1_ref.connect("top", probe_top.ports["e6"], allow_width_mismatch=True,
allow_layer_mismatch=True)
via_M4_TM1_ref.move((0, via_M4_TM1_ref.ysize/2))
route_pd2_vout = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd2_ref.ports["vout"]],
    ports2=[via_M4_TM1_ref.ports["bottom"]],
    route_width=4,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal4drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
# vref connection of pd2 to probe top
via_M3_TM1_ref = c.add_ref(via_M3_TM1)
via_M3_TM1_ref.connect("top", probe_top.ports["e7"], allow_width_mismatch=True,
allow_layer_mismatch=True)
via_M3_TM1_ref.move((0, via_M3_TM1_ref.ysize/2))
route_pd2_vref = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd2_ref.ports["vref"]],
    ports2=[via_M3_TM1_ref.ports["bottom"]],
    route_width=4,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal3drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

c.plot()
```



Place and Connect Third Power Detector (PD3)

PD3 is instantiated with both mirror operations (`mirror_y().mirror_x()`), then placed through net connectivity:

- Drive `rfin` from `blc_2_ref` port `e3` through the RF interconnect
- Route `vdd` , `vss` , `vout` , and `vref` to the bottom bondpads
- Use VIA-assisted transitions for `vout` (Metal4) and `vref` (Metal3)

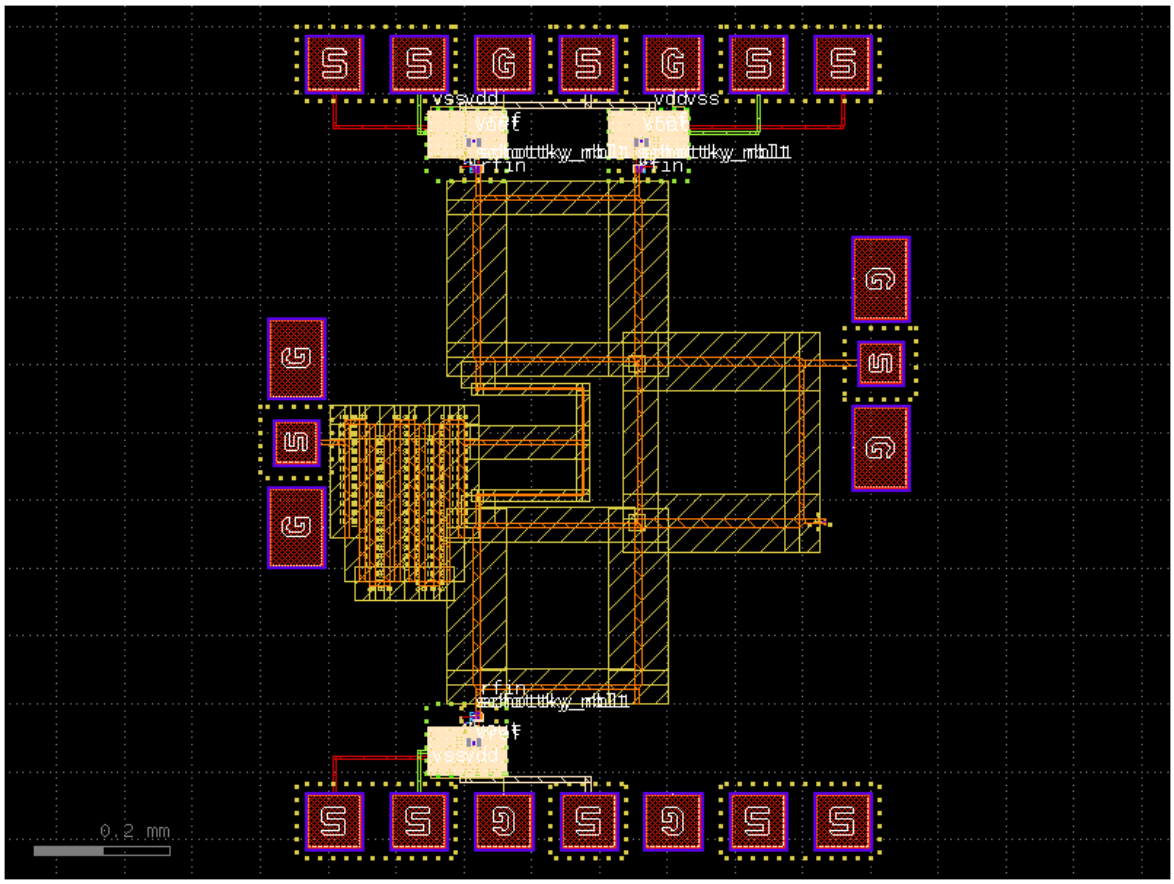
PD3 placement is derived from mirror transforms plus port connections, so it adapts automatically with layout changes.

In [44]:
```
pd3_ref = c.add_ref(pd).mirror_y().mirror_x()
pd3_ref.connect("rfin", rfin_connection_pd3.ports["e2"],
allow_width_mismatch=True)

# vdd connection of pd3 to probe bottom
route_pd3_vdd = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd3_ref.ports["vdd"]],
    ports2=[probe_bottom.ports["e4"]],
    route_width=8,    # adjust for thicker connections
    layer=ihp.tech.LAYER.TopMetal1drawing,
    start_straight_length=10,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
```

```
# vss connection of pd3 to probe bottom
route_pd3_vss = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd3_ref.ports["vss"]],
    ports2=[probe_bottom.ports["e5"]],
    route_width=2,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal5drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
# vout connection of pd3 to probe bottom
via_M4_TM1_ref = c.add_ref(via_M4_TM1)
via_M4_TM1_ref.connect("top", probe_bottom.ports["e6"],
allow_width_mismatch=True, allow_layer_mismatch=True)
via_M4_TM1_ref.move((0, -via_M4_TM1_ref.ysize/2))

route_pd3_vout = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd3_ref.ports["vout"]],
    ports2=[via_M4_TM1_ref.ports["bottom"]],
    route_width=4,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal4drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
# vref connection of pd3 to probe bottom
via_M3_TM1_ref = c.add_ref(via_M3_TM1)
via_M3_TM1_ref.connect("top", probe_bottom.ports["e7"],
allow_width_mismatch=True, allow_layer_mismatch=True)
via_M3_TM1_ref.move((0, -via_M3_TM1_ref.ysize/2))
route_pd3_vref = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd3_ref.ports["vref"]],
    ports2=[via_M3_TM1_ref.ports["bottom"]],
    route_width=4,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal3drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
c.plot()
```



Place and Connect Fourth Power Detector (PD4)

PD4 is instantiated with a single Y-axis mirror (`mirror_y()`), then placed by connecting ports:

- Connect `rfin` from `blc_2_ref` port `e2` through the RF interconnect
- Route `vdd` and `vss` to the bottom bondpads
- Route `vout` and `vref` through via-assisted Metal4/Metal3 paths to bottom pads

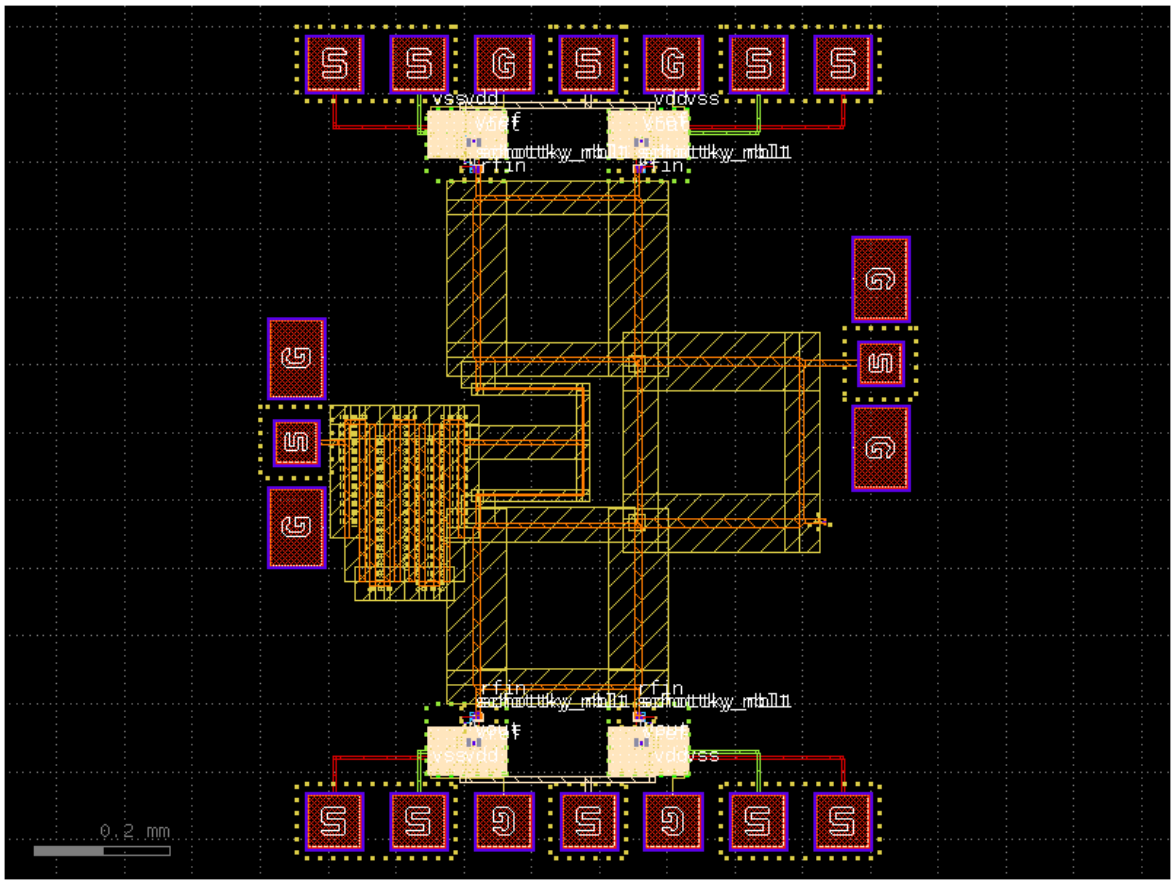
Like the other detectors, PD4 location is topology-driven and scales with the surrounding layout geometry.

In [45]:
```
pd4_ref = c.add_ref(pd).mirror_y()
pd4_ref.connect("rfin", rfin_connection_pd4.ports["e2"],
allow_width_mismatch=True)

# vdd connection of pd4 to probe bottom
route_pd4_vdd = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd4_ref.ports["vdd"]],
    ports2=[probe_bottom.ports["e4"]],
    route_width=8,    # adjust for thicker connections
    layer=ihp.tech.LAYER.TopMetal1drawing,
    start_straight_length=10,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
```

```
# vss connection of pd4 to probe bottom
route_pd4_vss = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd4_ref.ports["vss"]],
    ports2=[probe_bottom.ports["e3"]],
    route_width=2,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal5drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
# vout connection of pd4 to probe bottom
via_M4_TM1_ref = c.add_ref(via_M4_TM1)
via_M4_TM1_ref.connect("top", probe_bottom.ports["e2"],
allow_width_mismatch=True, allow_layer_mismatch=True)
via_M4_TM1_ref.move((0, -via_M4_TM1_ref.ysize/2))
route_pd4_vout = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd4_ref.ports["vout"]],
    ports2=[via_M4_TM1_ref.ports["bottom"]],
    route_width=4,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal4drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
# vref connection of pd4 to probe bottom
via_M3_TM1_ref = c.add_ref(via_M3_TM1)
via_M3_TM1_ref.connect("top", probe_bottom.ports["e1"],
allow_width_mismatch=True, allow_layer_mismatch=True)
via_M3_TM1_ref.move((0, -via_M3_TM1_ref.ysize/2))
route_pd4_vref = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd4_ref.ports["vref"]],
    ports2=[via_M3_TM1_ref.ports["bottom"]],
    route_width=4,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal3drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

c.plot()
```



Place Sealring

After the six-port is assembled, it is important to place a sealring around it. The sealring is necessary at the chip edge to stop cracks from spreading during the dicing process. It also works as a metal barrier to block moisture and chemicals from entering the internal layers. This ensures the long-term reliability and prevents corrosion of the semiconductor die.

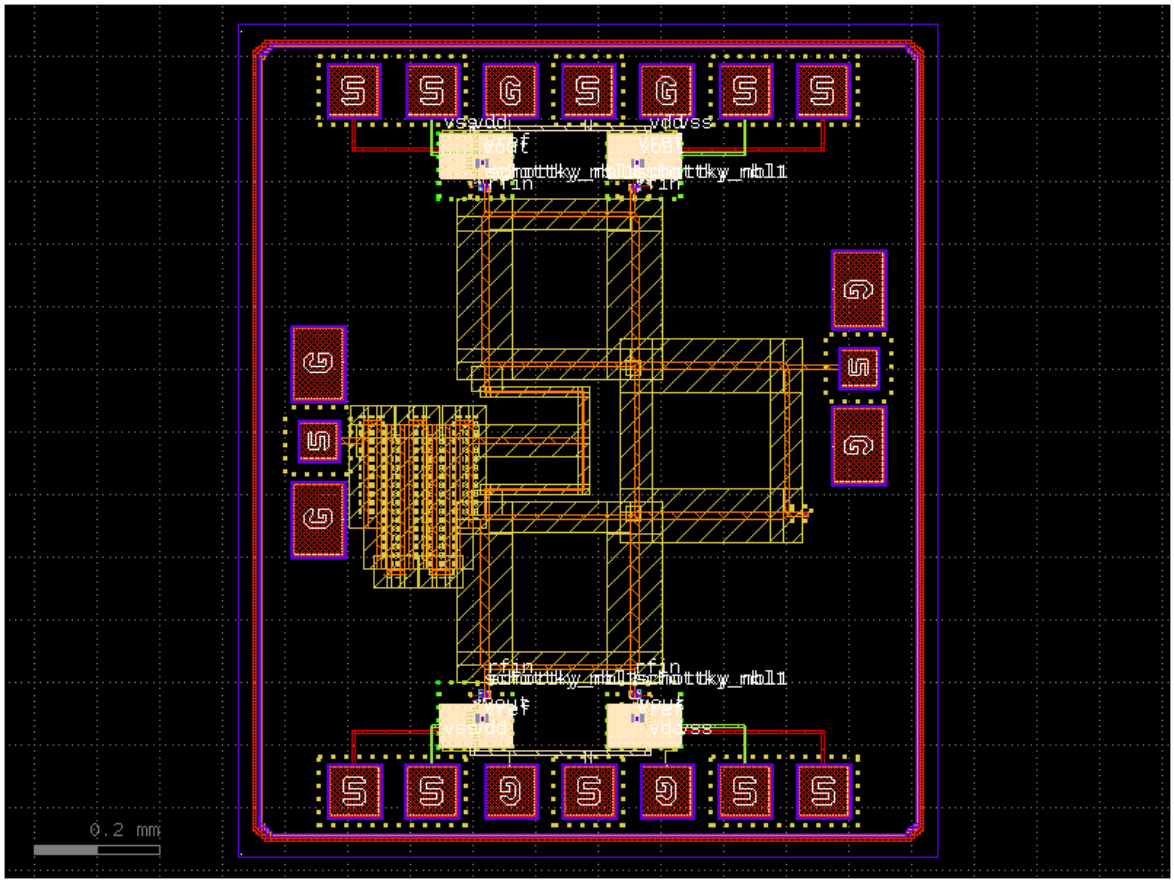
The code cell sizes the sealring from the extent of the chip so far, with a margin of `SEALRING_MARGIN` = 50 µm on the left and right side and 25 µm at the top and bottom, and centers it on the chip. The inner edge of the sealring, on the layer `EdgeSealboundary`, is the reference for the metal fill below.

In [46]:
```
sealring_margin = SEALRING_MARGIN  # margin around circuit for sealring
sealring_width = round(c.xsize + 2 * sealring_margin)
sealring_height = round(c.ysize + 2 * sealring_margin / 2)

sealring_center = (round(c.center[0], 2), round(c.center[1], 2))  # round it to
avoid off grid errors
c.add_ref(ihp.cells.sealring(width=sealring_width,
height=sealring_height)).center = sealring_center

c.plot()
```

```
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
Warning: Box.destroy: already destroyed!
```



Define Metal Fill Patterns

Define helper functions for creating metal fill cells:

- **`fill_cell()`** : Generic rectangular fill cell of a given size on the specified layer
- **`fill_gat_active()`** : GatPoly filler with a centered, inset Activ filler for gate and active region filling
- **`fill_ground()`** : 1×1 µm Metal5 ground plane fill cell
- **`slit_ground()`** : Metal5 slit pattern for the ground plane

These fill cells are used by gdsfactory's `fill()` function to uniformly distribute metal and maintain electrical properties.

In [47]:
```
@gf.cell
def fill_cell(layer=ihp.tech.LAYER.Metal5slit, size=(3, 3)) -> gf.Component:
    """Simple rectangular fill cell for a given layer and size."""
    c = gf.Component()
    c << gf.c.rectangle(layer=layer, size=size)
    return c


@gf.cell
def fill_gat_active(size=(3, 3), active_extension=(0.18, 0.18)) -> gf.Component:
    """Gate-poly + active filler cell with centred Activ region.

    `size` sets GatPoly:filler directly (bounded by GFil.a's 5.00um max width);
    Activ:filler is inset by `active_extension` so GatPoly:filler encloses it
(GFil.j).
    """
    c = gf.Component()
    gat_ref = c.add_ref(gf.c.rectangle(layer=ihp.tech.LAYER.GatPolyfiller,
size=size))
    activ_ref = c.add_ref(
        gf.c.rectangle(
            layer=ihp.tech.LAYER.Activfiller,
            size=(size[0] - 2 * active_extension[0], size[1] - 2 *
active_extension[1]),
        )
    )
    activ_ref.center = gat_ref.center
    return c


@gf.cell
def fill_ground() -> gf.Component:
    """1 x 1 um Metal5 ground fill tile."""
    c = gf.Component()
    c.add_polygon(
        layer=ihp.tech.LAYER.Metal5drawing,
        points=[
            (0, 0),
            (1, 0),
            (1, 1),
            (0, 1),
        ],
    )
    return c


@gf.cell
def slit_ground() -> gf.Component:
    """Metal5 slit opening for ground fill."""
    c = gf.Component()
    c.add_polygon(
        layer=ihp.tech.LAYER.Metal5slit,
        points=[
            (1.1, 1.1),
            (4, 1.1),
            (4, 4.1),
```

```
            (1.1, 4.1),
        ],
    )
    return c
```

Fill All Metal Layers

The metal fill keeps the density of every layer within the limits of the design rules, which the chemical-mechanical polishing (CMP) needs for flat layers. `c.fill()` places copies of a fill cell on a regular grid inside the seal boundary, minus the exclusion zones around existing structures:

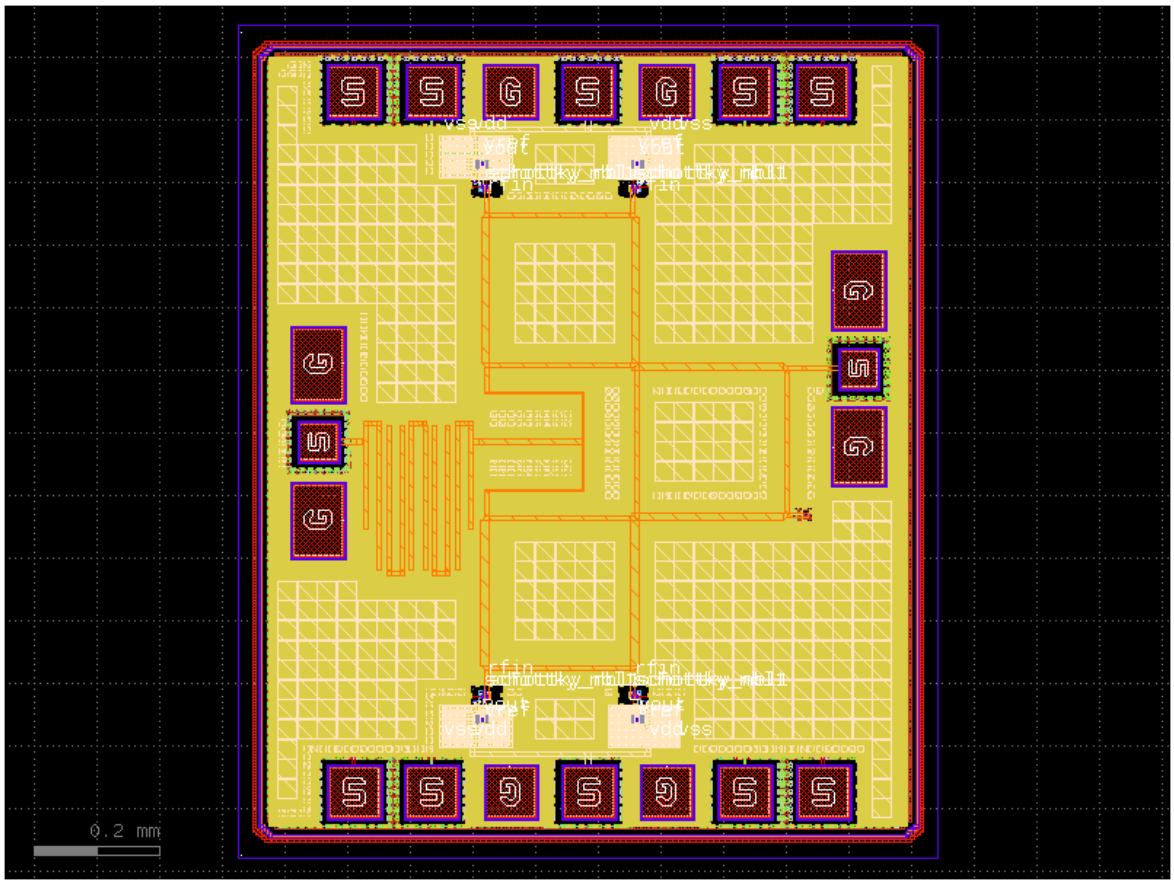
Layer	Fill cell	Spacing	Distance to the seal boundary	Kept away from
GatPoly and Activ	5 µm x 5 µm GatPoly filler with an Activ filler inset by 0.18 µm	0.85 µm	30 µm	devices (Activ, implants, wells, GatPoly, contacts) by 1.1 µm to 2 µm, pad openings by 12 µm
Metal1 and Metal2	5 µm x 5 µm	1.1 µm	40 µm	drawn metal and pad openings by 10 µm
Metal3 and Metal4	5 µm x 5 µm	1 µm	40 µm	drawn metal and pad openings by 10 µm
Metal5	solid ground plane of 1 µm x 1 µm tiles, then slits of 2.9 µm x 3 µm	0 µm, slits 1.1 µm x 1.2 µm	50 µm	pad openings and Metal5 no-fill regions, slits 5 µm away from TopMetal2 lines and 1 µm from MIM capacitors
TopMetal1	30 µm x 30 µm drawn squares, then 10 µm x 10 µm filler	1.65 µm, 3 µm	10 µm	drawn TopMetal1 by 10 µm, TopMetal2 by 20 µm

The Metal5 ground plane is the reference plane of all microstrip lines. The design rules require slits in wide metal, so slits are cut into it everywhere except under the TopMetal2 lines and the MIM capacitors. It stays 50 µm inside the seal boundary: any closer, it merges with the Metal5 ring of the sealring, which connects the whole sealring to VSS and adds antenna violations at the transistor gates tied to VSS.

In [48]:
```
# active/gat poly fill
gatpoly_activ_fill_space = 0.85
c.fill(
    fill_cell=fill_gat_active(size=(5, 5)),
    fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -30)],
    exclude_layers=[
        (ihp.tech.LAYER.Activdrawing, 1.1),
        (ihp.tech.LAYER.pSDdrawing, 1.1),
        (ihp.tech.LAYER.nSDblock, 1.1),
        (ihp.tech.LAYER.SalBlockdrawing, 1.1),
        (ihp.tech.LAYER.NWelldrawing, 1.1),
        (ihp.tech.LAYER.nBuLaydrawing, 1.1),
```

```
        (ihp.tech.LAYER.PWellblock, 1.5),
        (ihp.tech.LAYER.GatPolydrawing, 2),
        (ihp.tech.LAYER.Contdrawing, 1.2),
        (ihp.tech.LAYER.Passivdrawing, 12),
        (ihp.tech.LAYER.GatPolynofill, 1),
        (ihp.tech.LAYER.Activnofill, 1),
    ],
    x_space=gatpoly_activ_fill_space,
    y_space=gatpoly_activ_fill_space,
)
# metal1 fill
metal_fill_space = 1.1
c.fill(
    fill_cell=fill_cell(layer=ihp.tech.LAYER.Metal1filler, size=(5, 5)),
    fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -40)],
    exclude_layers=[
        (ihp.tech.LAYER.Metal1drawing, 10),
        (ihp.tech.LAYER.Passivdrawing, 10),
        (ihp.tech.LAYER.Metal1nofill, 1),
    ],
    x_space=metal_fill_space,
    y_space=metal_fill_space,
)
# metal2 fill
c.fill(
    fill_cell=fill_cell(layer=ihp.tech.LAYER.Metal2filler, size=(5, 5)),
    fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -40)],
    exclude_layers=[
        (ihp.tech.LAYER.Metal2drawing, 10),
        (ihp.tech.LAYER.Passivdrawing, 10),
        (ihp.tech.LAYER.Metal2nofill, 1),
    ],
    x_space=metal_fill_space,
    y_space=metal_fill_space,
)
# metal3 fill
metal_fill_space = 1
c.fill(
    fill_cell=fill_cell(layer=ihp.tech.LAYER.Metal3filler, size=(5, 5)),
    fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -40)],
    exclude_layers=[
        (ihp.tech.LAYER.Metal3drawing, 10),
        (ihp.tech.LAYER.Passivdrawing, 10),
        (ihp.tech.LAYER.Metal3nofill, 1),
    ],
    x_space=metal_fill_space,
    y_space=metal_fill_space,
)
# metal4 fill
c.fill(
    fill_cell=fill_cell(layer=ihp.tech.LAYER.Metal4filler, size=(5, 5)),
    fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -40)],
    exclude_layers=[
        (ihp.tech.LAYER.Metal4drawing, 10),
        (ihp.tech.LAYER.Passivdrawing, 10),
        (ihp.tech.LAYER.Metal4nofill, 1),
    ],
    x_space=metal_fill_space,
    y_space=metal_fill_space,
)
```

```
# metal5 fill (groundplate)
c.fill(
    fill_cell=fill_ground(),
    fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -50)],
    exclude_layers=[(ihp.tech.LAYER.Passivdrawing, 0),
(ihp.tech.LAYER.Metal5nofill, 0)],
    x_space=0,
    y_space=0,
)
c.fill(
    fill_cell=slit_ground(),
    fill_layers=[(ihp.tech.LAYER.Metal5drawing, -2)],
    exclude_layers=[
        (ihp.tech.LAYER.TopMetal2drawing, 5),
        (ihp.tech.LAYER.MIMdrawing, 1),
        (ihp.tech.LAYER.Metal5slit, 1),
    ],
    x_space=1.1,
    y_space=1.2,
)
# topmetal1 fill
c.fill(
    fill_cell=fill_cell(layer=ihp.tech.LAYER.TopMetal1drawing, size=(30, 30)),
    fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -10)],
    exclude_layers=[(ihp.tech.LAYER.TopMetal1drawing, 10),
(ihp.tech.LAYER.TopMetal2drawing, 20)],
    x_space=1.65,
    y_space=1.65,
)
c.fill(
    fill_cell=fill_cell(layer=ihp.tech.LAYER.TopMetal1filler, size=(10, 10)),
    fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -10)],
    exclude_layers=[(ihp.tech.LAYER.TopMetal1drawing, 10),
(ihp.tech.LAYER.TopMetal2drawing, 20)],
    x_space=3,
    y_space=3,
)
c.plot()
```



Add Logos and Decorative Elements

Import and place decorative logos for the chip. All positions are derived from the layout, so the logos follow the design frequency:

- **JKU Logo**: Johannes Kepler University branding on TopMetal2, in the lower right corner just above the bottom bondpads
- **IWS Logo**: Institute branding, aligned with the top right corner of the BLCs (only up to 200 GHz, where there is enough space)
- **D. Kellerer**: Designer name annotation (rotated -90°), between the top and left pads if there is enough space
- **Supervisors**: Acknowledgment text (rotated -90°), between the bottom and left pads if there is enough space

In [49]:
```
logo_dir = Path("/tmp/assets")

logo_gds_path = logo_dir / "jku_logo_m5.gds"
kellerer_gds_path = logo_dir / "D_Kellerer.gds"
iws_gds_path = logo_dir / "IWS.gds"
supervisors_gds_path = logo_dir / "supervisors.gds"

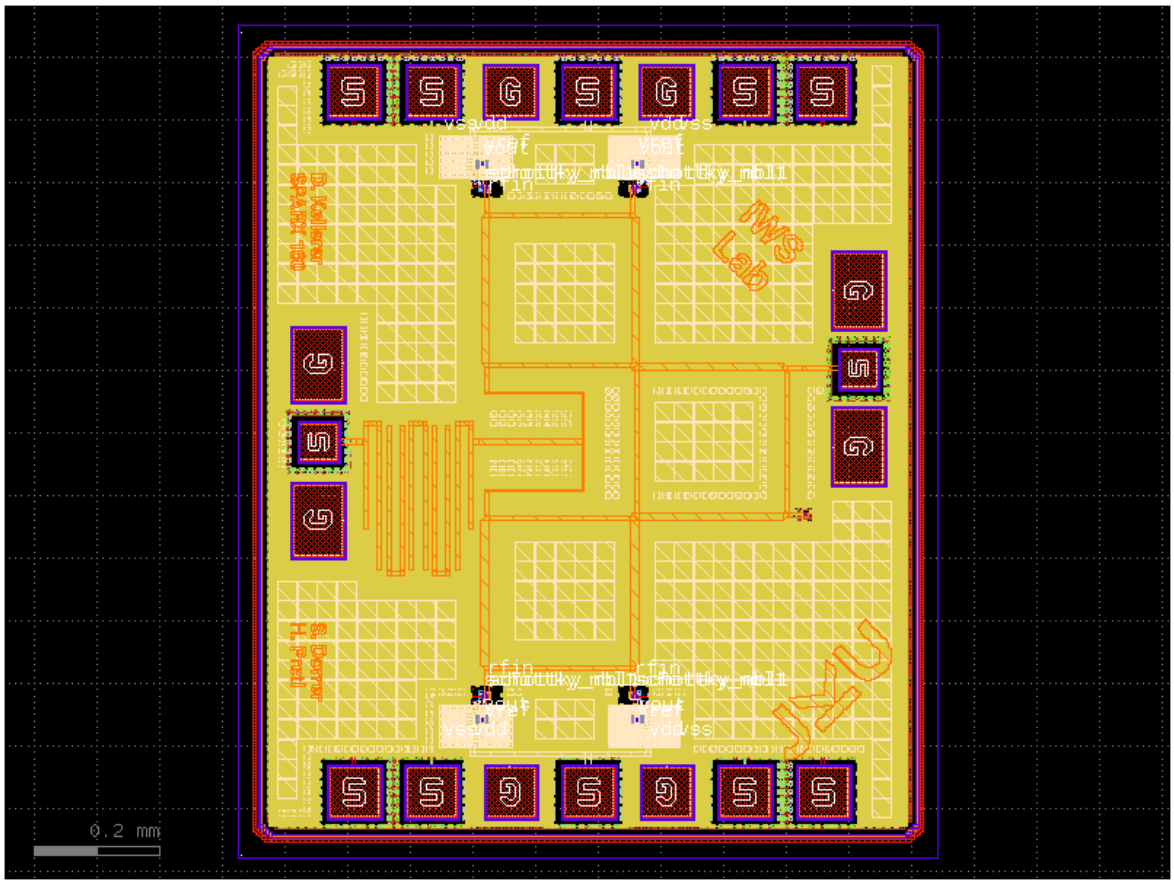
# JKU logo in the lower right corner, relative to right chip edge, just above
south pads
jku_logo_ref = c.add_ref(gf.import_gds(str(logo_gds_path), cellname="JKU_LOGO"))
chip_right = sealring_center[0] + sealring_width / 2
jku_logo_ref.xmax = chip_right - sealring_margin
jku_logo_ref.ymin = probe_bottom.ymax + 10
```

```
# place IWS logo only up to 200 GHz, to avoid overcrowding the layout for higher
frequencies where the design is more compact
if f <= 200e9:
    iws = c.add_ref(gf.import_gds(str(iws_gds_path), cellname="Logo_IWS"))
    iws.xmax = blc_3_ref.xmax  # align right edge of IWS logo with right edge of
BLC block
    iws.ymax = blc_1_ref.ymax  # align top edge of IWS logo with top edge of BLC
block

# place gds with D. Kellerer only if space is available between top probe pads
and left probe pads
kellerer = gf.import_gds(str(kellerer_gds_path), cellname="Name_D").rotate(-90)
if abs(probe_top.ymin - probe_left.ymax) > kellerer.ysize + 2 *
LOGO_VERTICAL_CLEARANCE:
    kellerer_ref = c.add_ref(kellerer)
    kellerer_ref.xmin = probe_left.xmin
    kellerer_ref.center = (kellerer_ref.center[0], probe_left.ymax +
(abs(probe_top.ymin - probe_left.ymax) / 2))
    kellerer_ref.xmin = snap_to_grid(kellerer_ref.xmin)
    kellerer_ref.ymin = snap_to_grid(kellerer_ref.ymin)

# place supervisor names only if space is available between bottom probe pads
and left probe pads
supervisors = gf.import_gds(str(supervisors_gds_path),
cellname="supervisors").rotate(-90)
if abs(probe_bottom.ymax - probe_left.ymin) > supervisors.ysize + 2 *
LOGO_VERTICAL_CLEARANCE:
    supervisors_ref = c.add_ref(supervisors)
    supervisors_ref.xmin = probe_left.xmin
    supervisors_ref.center = (supervisors_ref.center[0], probe_bottom.ymax +
(abs(probe_bottom.ymax - probe_left.ymin) / 2))
    supervisors_ref.xmin = snap_to_grid(supervisors_ref.xmin)
    supervisors_ref.ymin = snap_to_grid(supervisors_ref.ymin)

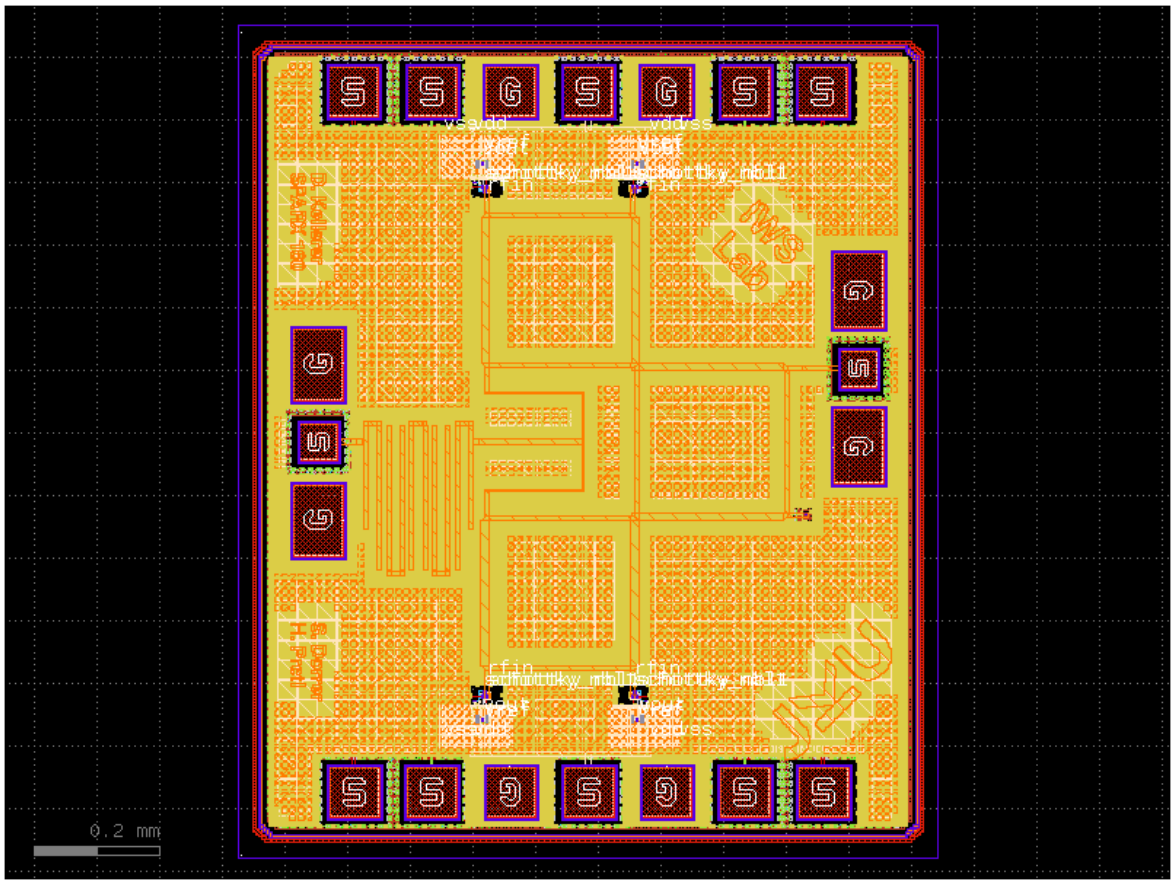
c.plot()
```



Final TopMetal2 Fill and Chip Visualization

The TopMetal2 fill pattern is applied to complete the metal density requirements. Use 9×9µm fill cells with 3µm spacing, excluding the 20µm perimeter around TopMetal2 drawing layers. Plot the final complete chip design to visualize the finished layout with all components, routing, logos, and metal fill in place.

In [50]:
```
c.fill(
        fill_cell=fill_cell(layer=ihp.tech.LAYER.TopMetal2filler, size=(9, 9)),
        fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -50)],
        exclude_layers=[(ihp.tech.LAYER.TopMetal2drawing, 20)],
        x_space=3,
        y_space=3,
    )
c.plot()
```



Pad Labels and Pins

Finally, the chip is moved to its final origin, and every probe pad and bondpad gets a text label and a pin rectangle on its own metal layer. The labels name the pad nets (`vlo` , `vrf` , `vdd` , `vss` , `vout1` – `vout4` , `vref1` – `vref4`) for LVS and for identifying the pads in the layout viewer.

`bondpad_array` places the ports at the outer edge of each pad, so `add_pad_pin()` first moves the port to the pad center, then adds a text label on the text datatype (25) and a pin rectangle on the pin datatype (2) of the metal layer of the pad.

In [51]:
```
def add_pad_pin(c: gf.Component, port_name: str, pad_name: str, pad_length:
float ) -> None:
    """Add a port label and pin rectangle centered on a probe pad.

    Ports on pad arrays sit at the pad's outer edge rather than its
    center (see bondpad_array), offset by half the pad size in the
    direction the port faces. This recenters the label/pin on the pad
    before adding it, based on the port's orientation.

    Args:
        c: Parent component (`c`) that owns the port.
        port_name: Name of the port on `c` to label.
        pad_name: Text to use for the port label (e.g. "VOUT"). The text
            and pin layers are derived from the port's own drawing layer.
    """
    port = c.ports[port_name].copy()
    orientation = port.orientation % 360
```

```
    if orientation == 0:
        port.center = (port.center[0] - pad_length / 2, port.center[1])
    elif orientation == 90:
        port.center = (port.center[0], port.center[1] - pad_length / 2)
    elif orientation == 180:
        port.center = (port.center[0] + pad_length / 2, port.center[1])
    elif orientation == 270:
        port.center = (port.center[0], port.center[1] + pad_length / 2)
    else:
        raise ValueError(f"Unsupported port orientation: {port.orientation}")

    layer_number, _ = gf.get_layer_tuple(port.layer)
    text_layer = (layer_number, 25)
    pin_layer = (layer_number, 2)

    gf.labels.add_port_labels(c, ports=[port], layer=text_layer, texts=
[pad_name])
    gf.add_pins.add_pin_rectangle(c, port=port, layer=pin_layer)


c.xmin = 0
c.ymin = 0
c.move((-25, -25))


# add port labels for probe pads on the left side of the chip
add_pad_pin(c = c, port_name="probe_left_e1", pad_name="vlo",
pad_length=PROBE_SIGNAL_SIZE)
add_pad_pin(c = c, port_name="probe_left_e2", pad_name="vss",
pad_length=PROBE_SIGNAL_SIZE)
add_pad_pin(c = c, port_name="probe_left_e3", pad_name="vss",
pad_length=PROBE_SIGNAL_SIZE)

# add port labels for probe pads on the right side of the chip
add_pad_pin(c = c, port_name="probe_right_e1", pad_name="vrf",
pad_length=PROBE_SIGNAL_SIZE)
add_pad_pin(c = c, port_name="probe_right_e2", pad_name="vss",
pad_length=PROBE_SIGNAL_SIZE)
add_pad_pin(c = c, port_name="probe_right_e3", pad_name="vss",
pad_length=PROBE_SIGNAL_SIZE)

# add port labels for probe pads on the top side of the chip
PROBE_SIGNAL_SIZE_TOP_BOTTOM = c.ports["probe_top_e1"].width  # size of the
probe signal pads in um
add_pad_pin(c = c, port_name="probe_top_e1", pad_name="vref1",
pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
add_pad_pin(c = c, port_name="probe_top_e2", pad_name="vout1",
pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
add_pad_pin(c = c, port_name="probe_top_e3", pad_name="vss",
pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
add_pad_pin(c = c, port_name="probe_top_e4", pad_name="vdd",
pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
add_pad_pin(c = c, port_name="probe_top_e5", pad_name="vss",
pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
add_pad_pin(c = c, port_name="probe_top_e6", pad_name="vout2",
pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
add_pad_pin(c = c, port_name="probe_top_e7", pad_name="vref2",
pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)

# add port labels for probe pads on the bottom side of the chip
```

```
add_pad_pin(c = c, port_name="probe_bottom_e1", pad_name="vref4",
pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
add_pad_pin(c = c, port_name="probe_bottom_e2", pad_name="vout4",
pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
add_pad_pin(c = c, port_name="probe_bottom_e3", pad_name="vss",
pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
add_pad_pin(c = c, port_name="probe_bottom_e4", pad_name="vdd",
pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
add_pad_pin(c = c, port_name="probe_bottom_e5", pad_name="vss",
pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
add_pad_pin(c = c, port_name="probe_bottom_e6", pad_name="vout3",
pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
add_pad_pin(c = c, port_name="probe_bottom_e7", pad_name="vref3",
pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
```

Export Final GDS

The finished chip is written to `sparx_top.gds` without GDSFactory metadata, and on Colab the file is downloaded to the local machine. At `f` = 160 GHz, it is geometrically identical to `layout/sparx160_top.gds` of the SPARX repository, which `scripts/six_port_gen.py` generates.

In [52]:
```
c.write_gds("sparx_top.gds", with_metadata=False)

# on Google Colab, download the GDS to the local machine
try:
    from google.colab import files
    files.download("sparx_top.gds")
except ImportError:
    pass
```

Design Rule Check (DRC)

Finally, the generated layout is checked with the KLayout DRC decks of the IHP-Open-PDK. The decks are Ruby DRC scripts, so the full KLayout program is required in addition to the `klayout` Python module installed with GDSFactory.

Installation of KLayout:

The KLayout package matching the Ubuntu version of the runtime is downloaded from the KLayout website and installed with `apt`.

In [53]:
```
import platform

KLAYOUT_VERSION = "0.30.12"
UBUNTU = platform.freedesktop_os_release()["VERSION_ID"].split(".")[0]

! wget -q https://www.klayout.org/downloads/Ubuntu-
{UBUNTU}/klayout_{KLAYOUT_VERSION}-1_amd64.deb -O /tmp/klayout.deb
! apt-get update -qq > /dev/null 2>&1; apt-get install -y -qq /tmp/klayout.deb
> /dev/null
assert shutil.which("klayout"), "KLayout was not installed, see the apt output
```

```
above"
! klayout -b -v
```

```
debconf: unable to initialize frontend: Dialog

debconf: (No usable dialog-like program is installed, so the dialog based frontend
cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line 79, <STDIN> li
ne 153.)

debconf: falling back to frontend: Readline

KLayout 0.30.12
```

Run DRC:

The DRC is run with the `run_drc.py` wrapper of the IHP-Open-PDK. Three levels are available:

- `precheck` : minimal rule set of the foundry precheck. Fastest option.
- `macro` : all rules except chip-level density, antenna, and the extra rules. This is the level of `make klayout-drc` in the SPARX repository. The 160 GHz layout is clean, and so are the layouts of the other twelve frequencies the generator produces from 60 GHz to 300 GHz.
- `regular` : all rules, including density and antenna. The antenna violations reported at this level were waived by IHP for the tapeout, see KNOWN_ISSUES.md.

> **Note:** On the two vCPUs of the free Colab tier, the DRC of the full chip takes about 6 minutes for `macro` , 3 minutes for `precheck` , and 17 minutes for `regular` . Select `precheck` for a quicker check.

In [54]:
```
import time

DRC_LEVEL = "precheck"  # precheck, macro, or regular
DRC_DIR   = f"drc_{DRC_LEVEL}"
DRC_FLAGS = {
    "precheck": "--precheck_drc --no_density --no_offgrid --no_angle --
disable_extra_rules --no_recommended",
    "macro":    "--no_density --disable_extra_rules",
    "regular":  "--antenna",
}[DRC_LEVEL]

# the rule deck is split into one part per physical CPU core, hyperthreads of
the same core do not help because every part reloads the layout
DRC_CORES =
len({open(f"/sys/devices/system/cpu/cpu{cpu}/topology/thread_siblings_list").rea
d() for cpu in os.sched_getaffinity(0)})

start = time.time()
# the full logs are written to DRC_DIR, only a rule counter, the progress per
rule table, and the result are shown here
! rm -rf {DRC_DIR} && python3 {PDK_ROOT}/ihp-
sg13g2/libs.tech/klayout/tech/drc/run_drc.py --path=sparx_top.gds --topcell=Six-
Port-CAC --run_dir={DRC_DIR} --mp={DRC_CORES} {DRC_FLAGS} 2>&1 \
    | awk '/Executing rule/ {{ if (++n % 50 == 0) print n " rules checked" }}
/completed in|Check Passed|Check Failed|Violated rules|Total DRC Run time|ERROR/
```

```
{{ print }} {{ fflush() }}'
print(f"DRC ({DRC_LEVEL}) finished in {(time.time() - start) / 60:.1f} min")
```

```
50 rules checked

2026-10-12 14:50:13 +0000: Memory Usage (1829296K) : KLayout DRC run for tables 'm
ain' completed in 86.75 seconds

12-Oct-2026 14:50:15 | INFO     | ✅ --- KLayout DRC Check Passed: No DRC violatio
ns detected in the layout. --- ✅

12-Oct-2026 14:50:15 | INFO     | Total DRC Run time: 90.10 seconds (including exec
ution, analysis, and reporting)

DRC (precheck) finished in 1.5 min
```

DRC Summary:

The violations are counted per rule from the KLayout result databases (`.lyrdb`), which can also be opened in the KLayout marker browser.

In [55]:
```
import glob
import xml.etree.ElementTree as ET
from collections import Counter

# every <item> in a result database is one violation, its <category> is the rule
name
lyrdbs = glob.glob(f"{DRC_DIR}/*.lyrdb")
assert lyrdbs, f"No DRC result in {DRC_DIR}, the DRC did not run, see the output
above"
violations = Counter()
for lyrdb in lyrdbs:
    for item in ET.parse(lyrdb).getroot().iterfind("./items/item"):
        violations[item.findtext("category").strip("'")] += 1

for rule, count in sorted(violations.items()):
    print(f"{rule:<24} {count:>6}")
print(f"DRC ({DRC_LEVEL}): {sum(violations.values())} violations in
{len(violations)} rules")
```

```
DRC (precheck): 0 violations in 0 rules
```

Circuit Simulation with VACASK

The passive RF structures of SPARX were simulated with the EM solver AWS Palace, and the power detector was designed in Xschem. The following cells take this flow to the circuit level with VACASK, an open-source analog simulator with harmonic balance (HB), periodic steady state (PSS), and the periodic small-signal analyses HBAC, HBNOISE, PAC, and PNOISE. Most open-source RFIC work uses Qucs-S and Xyce, which was long the only open-source SPICE-compatible simulator with HB, but offers neither PSS nor HBAC nor transient noise. VACASK provides all of them, a shooting PSS, the quasi-periodic HBAC on top of an HB solution, and device-level transient noise, and since September 2026 also HBNOISE around an HB solution and PAC and PNOISE around a shooting PSS, which is what makes the large-signal characterization of a pumped detector possible in an open-source flow:

1. The EM result of the six-port core is converted into a lumped-element netlist with `snp2le`, which carries the thermal noise of the core by Bosma's theorem.
2. The power detector is characterized: transfer curve (HB and PSS), noise figure (HBAC and HBNOISE, checked with PAC and PNOISE), and transient noise.
3. The receiver, the six-port core driving the four post-layout power detectors, is simulated with HB, HBAC, and a transient.
4. The noise figure of the receiver is simulated through the core model, with HBAC and HBNOISE, checked with PAC and PNOISE.

The testbenches are the ones of the SPARX repository, where each result is explained in detail in the testbench README. Their netlists were exported from the Xschem schematics and are fetched from the repository together with the post-processing scripts.

> **Important:** The simulations use the 160 GHz design of the SPARX repository. The EM result of the six-port core (`sparx160_core.s7p`), the post-layout extraction of the power detector, and the LO and RF frequencies of the testbenches do not follow the frequency `f` set above. For a layout generated at another frequency, the EM simulation of the new layout with AWS Palace and the extraction have to be redone in the repository flow first, which is beyond the scope of this notebook.

> **Note:** Every cell of this section has a `RUN_...` flag at its top, set it to `0` to skip the cell. The transient-noise simulation and the two receiver simulations are switched off by default because of their runtime, set their flags to `1` to run them. The noise-figure simulation reads the responsivity of the transfer-curve simulation, and the transient-noise simulation reads the result of the noise-figure simulation, so run them in this order.

Installation of VACASK:

The periodic analyses PAC and PNOISE reached VACASK at the end of September 2026, and no IIC-OSIC-TOOLS image ships them yet. A build of VACASK `1b48553`, the commit the results of the SPARX repository were produced with, is therefore stored in the repository under `sscs-ose-code-a-chip/build/vacask`, made in the Google Colab runtime image. It is unpacked and verified against its checksum, and the OpenVAF-Reloaded compiler is taken from the IIC-OSIC-TOOLS 2026.09 image on Docker Hub. With `BUILD_VACASK = 1`, the same VACASK commit is built from source instead, which takes about 12 minutes.

VACASK reads the device models of the IHP-Open-PDK as its own model libraries, and those are generated from the ngspice models of the PDK by the `sg13g2tovc` converter shipped with VACASK. The testbenches, their netlists, and the post-processing scripts come from the same sparse checkout of the SPARX repository as the VACASK build.

> **Runtime:** On the two vCPUs of the free Colab tier, this cell takes about **1 minute**.

In [56]:

```
import hashlib
import json
import re
import subprocess

RUN_VACASK_INSTALL = 1  # 1 = run this cell, 0 = skip it
BUILD_VACASK       = 0  # 1 = build VACASK from source, 0 = unpack the build
stored in the SPARX repository

# VACASK 1b48553 has pac and pnoise, which no IIC-OSIC-TOOLS image ships yet, so
its build is stored in the SPARX repository
VACASK_COMMIT  = "1b48553c5b204bc11fb6f6d900b18f079fafc90e"
VACASK_ARCHIVE = "sscs-ose-code-a-chip/build/vacask/vacask-1b48553-ubuntu24.04-
x86_64.tar.gz"
VACASK_SHA256  =
"7e92185685efe80f18c27f2ec642a6282a42d1f9d49c8c12d73d124faa6823af"
# OpenVAF-Reloaded of the IIC-OSIC-TOOLS 2026.09 image, one image layer,
compiles the Verilog-A models of the PDK
IIC_IMAGE      = "hpretl/iic-osic-tools"
OPENVAF_LAYER  =
"sha256:2a89bd6bb9998d8dd2929839f5d2adc466b52b1596742e93411ca7706d829127"
VACASK_DIR     = "/foss/tools/vacask"
OPENVAF_DIR    = "/foss/tools/openvaf"
SPARX_DIR      = "/tmp/sparx"  # sparse checkout of the SPARX repository
XSCHEM_DIR     = f"{SPARX_DIR}/testbenches/xschem"
SIM_DIR        = f"{XSCHEM_DIR}/simulations"
PLOT_DIR       = f"{XSCHEM_DIR}/plot_simulations"

if RUN_VACASK_INSTALL:
    start = time.time()

    # testbench netlists, fitted passive models, post-layout extraction, EM
results, post-processing scripts, and the VACASK build
    ! rm -rf {SPARX_DIR} && git clone -q --depth 1 --filter=blob:none --sparse
-b {SPARX_BRANCH} https://github.com/iic-jku/SG13CMOS_SPARX.git {SPARX_DIR} >
/tmp/sparx_checkout.log 2>&1 || cat /tmp/sparx_checkout.log
    ! git -C {SPARX_DIR} sparse-checkout set testbenches/xschem netlist scripts
verification/em/s-parameter sscs-ose-code-a-chip/build/vacask >>
/tmp/sparx_checkout.log 2>&1 || cat /tmp/sparx_checkout.log
    assert os.path.isdir(f"{XSCHEM_DIR}/netlists"), "the testbenches were not
fetched from the SPARX repository"
    ! mkdir -p {SIM_DIR} && cp {XSCHEM_DIR}/netlists/* {SIM_DIR}/

    if BUILD_VACASK:
        # the build recipe of IIC-OSIC-TOOLS: Boost 1.88 is built because Ubuntu
24.04 ships 1.83, OpenVAF-Reloaded compiles the built-in device models
        ! apt-get update -qq > /dev/null 2>&1; apt-get install -y -qq cmake
ninja-build bison flex libfftw3-dev libsuperlu-dev libopenblas-dev liblapack-dev
libsuitesparse-dev libtomlplusplus-dev libllvm18 > /dev/null
        ! mkdir -p {OPENVAF_DIR} && wget -qO-
https://github.com/OpenVAF/OpenVAF-
Reloaded/releases/download/v24.0.2mob/openvaf-r-v24.0.2mob-linux-x86_64.tar.gz |
tar xz -C {OPENVAF_DIR} --strip-components=1
        ! cd /tmp && wget -q
https://archives.boost.io/release/1.88.0/source/boost_1_88_0.tar.gz && tar xzf
boost_1_88_0.tar.gz
        ! cd /tmp/boost_1_88_0 && ./bootstrap.sh --with-toolset=gcc >
/tmp/boost_build.log 2>&1 && ./b2 -j{os.cpu_count()} --with-filesystem --with-
process --with-asio link=static toolset=gcc
define=BOOST_PROCESS_V2_DISABLE_PIDFD_OPEN cxxflags=-march=x86-64-v2 cflags=-
```

```
march=x86-64-v2 >> /tmp/boost_build.log 2>&1 || tail -20 /tmp/boost_build.log
        ! rm -rf /tmp/vacask && git clone -q --filter=blob:none --no-checkout
https://codeberg.org/arpadbuermen/VACASK.git /tmp/vacask && git -C /tmp/vacask
checkout -q {VACASK_COMMIT}
        ! cmake -G Ninja -S /tmp/vacask -B /tmp/vacask/build -
DCMAKE_BUILD_TYPE=Release -DCMAKE_CXX_FLAGS="-march=x86-64-v2 -
DBOOST_PROCESS_V2_DISABLE_PIDFD_OPEN" -DCMAKE_C_FLAGS=-march=x86-64-v2 -
DOPENVAF_OPTIONS="--target_cpu generic" -DOPENVAF_DIR={OPENVAF_DIR}/bin -
DBoost_DIR=/tmp/boost_1_88_0/stage/lib/cmake/Boost-1.88.0 >
/tmp/vacask_build.log 2>&1 || tail -20 /tmp/vacask_build.log
        ! cmake --build /tmp/vacask/build -j{os.cpu_count()} >>
/tmp/vacask_build.log 2>&1 && cmake --install /tmp/vacask/build --prefix
{VACASK_DIR} --strip >> /tmp/vacask_build.log 2>&1 || tail -20
/tmp/vacask_build.log
        ! rm -rf /tmp/boost_1_88_0 /tmp/boost_1_88_0.tar.gz /tmp/vacask
    else:
        # the build of the repository links against KLU, OpenBLAS, FFTW, and
LLVM 18, which a plain Ubuntu 24.04 runtime lacks
        archive = f"{SPARX_DIR}/{VACASK_ARCHIVE}"
        assert hashlib.sha256(open(archive, "rb").read()).hexdigest() ==
VACASK_SHA256, "the VACASK build does not match its checksum"
        ! mkdir -p /foss/tools && tar xzf {archive} -C /foss/tools
        # the OpenVAF layer is fetched from Docker Hub as `docker pull` would,
and unpacked to the path of the image
        token = json.load(urllib.request.urlopen(f"https://auth.docker.io/token?
service=registry.docker.io&scope=repository:{IIC_IMAGE}:pull"))["token"]
        request = urllib.request.Request(f"https://registry-
1.docker.io/v2/{IIC_IMAGE}/blobs/{OPENVAF_LAYER}", headers={"Authorization":
f"Bearer {token}"})
        layer = urllib.request.urlopen(request).read()
        assert "sha256:" + hashlib.sha256(layer).hexdigest() == OPENVAF_LAYER,
"the OpenVAF layer does not match its digest"
        open("/tmp/openvaf.tar.gz", "wb").write(layer)
        ! tar xzf /tmp/openvaf.tar.gz -C / && rm /tmp/openvaf.tar.gz
        ! apt-get update -qq > /dev/null 2>&1; apt-get install -y -qq libklu2
libopenblas0 libfftw3-double3 libllvm18 > /dev/null

    # the VACASK model libraries of the PDK are generated from its ngspice
models, and its Verilog-A models are compiled to OSDI
    ! PYTHONPATH={VACASK_DIR}/lib/vacask/python OPENVAF_DIR={OPENVAF_DIR}/bin
python3 -m sg13g2tovc --openvaf-options --target_cpu generic >
/tmp/sg13g2tovc.log 2>&1 || tail -20 /tmp/sg13g2tovc.log
    assert os.path.exists(f"{PDK_ROOT}/ihp-
sg13g2/libs.tech/vacask/osdi/psp103.osdi"), "the PDK models were not generated,
see /tmp/sg13g2tovc.log"
    ! cp {VACASK_DIR}/share/doc/vacask/demo/ihp-sg13g2/.vacaskrc.toml ~/
    print(f"VACASK installed in {(time.time() - start) / 60:.1f} min")

# the post-processing scripts read the VACASK output with the rawfile module
shipped with VACASK
os.environ["PATH"]       = f"{VACASK_DIR}/bin:{os.environ['PATH']}"
os.environ["PYTHONPATH"] = f"{VACASK_DIR}/lib/vacask/python"
version = subprocess.run(["vacask", "--version"], capture_output=True,
text=True)
version = version.stdout + version.stderr
assert "This is vacask" in version, f"VACASK does not start:\n{version}"
print(version.splitlines()[0])


def run_testbench(tb, variant=""):
```

```
    """Simulate a testbench with VACASK and post-process it, as `make sim-
xschem` does in the SPARX repository."""
    global RUN_START
    sim_dir = f"{SIM_DIR}/{variant}" if variant else SIM_DIR
    start = RUN_START = time.time()
    with open(f"{sim_dir}/{tb}.log", "w") as log:
        vacask = subprocess.run(["vacask", "-qp", "-sp", f"{tb}.spectre"],
cwd=sim_dir, stdin=subprocess.DEVNULL, stdout=log, stderr=subprocess.STDOUT)
    print(f"{tb}: VACASK finished in {(time.time() - start) / 60:.1f} min")
    # VACASK exits with 0 even when an analysis aborts or is missing from the
build, so its log is checked as well
    log = open(f"{sim_dir}/{tb}.log").read()
    if vacask.returncode or re.search("aborted|failed|not found", log,
re.IGNORECASE):
        raise RuntimeError(f"VACASK failed, see
{sim_dir}/{tb}.log:\n{log[-3000:]}")
    plot = subprocess.run(["python3", f"{PLOT_DIR}/plot_{tb}.py"],
cwd=SPARX_DIR, env={**os.environ, "POWDET_VARIANT": variant},
capture_output=True, text=True)
    print(plot.stdout)
    if plot.returncode:
        raise RuntimeError(f"the post-processing of {tb}
failed:\n{plot.stderr[-3000:]}")


def show_figure(name):
    """Display a figure of the post-processing, but only if the last
run_testbench() has written it."""
    path = f"{PLOT_DIR}/figures/{name}"
    # the repository holds a committed copy of every figure, which must not pass
for a new result
    assert os.path.getmtime(path) >= RUN_START, f"{name} was not written by this
simulation"
    display(Image(filename=path, width=1024))
```

```
debconf: unable to initialize frontend: Dialog

debconf: (No usable dialog-like program is installed, so the dialog based frontend
cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line 79, <STDIN> li
ne 15.)

debconf: falling back to frontend: Readline

VACASK installed in 0.6 min
This is vacask 0.3.4-88-g1b48553.
```

S-Parameters to Lumped-Element Netlist (snp2le):

A circuit simulator cannot use the S-parameters of the EM simulation directly in harmonic balance or transient analyses. snp2le fits the 7-port S-parameters of the six-port core (BPF, WPD, and three BLCs in one EM solve) with a rational model of order 24, enforces passivity, and writes it as a VACASK subcircuit of resistors, capacitors, and controlled sources. The two receiver testbenches below include this subcircuit.

This step from the EM results back to circuit simulation has been the missing link of open-source RF flows, where EM-verified passives used to be approximated by hand-tuned lumped elements. The core can be modeled either by one fit of the complete 7-port EM result, as here, or by a composition of block fits of order 13 for the bandpass filter, 10 for

the Wilkinson power divider, and 6 for each branch-line coupler. The composition converges better and simulates faster, but contains neither the coupling between adjacent couplers nor the interconnects, which is why the receiver uses the single fit.

Thermal noise by Bosma's theorem:

A lossy passive is a noise source, and at a uniform temperature its noise follows from its S-parameters alone. Every port k of a noisy N-port emits a noise wave c_k on top of the scattered signal, $\mathbf{b} = \mathbf{S}\,\mathbf{a} + \mathbf{c}$, with $\mathbf{a}$ and $\mathbf{b}$ the incident and outgoing power waves. Bosma's theorem gives the correlation matrix of these noise waves for a linear passive N-port in thermal equilibrium at the temperature T,

$$\mathbf{C}_\mathrm{S} = kT\,(\mathbf{I} - \mathbf{S}\mathbf{S}^\mathrm{H}),$$

with the Boltzmann constant k, the identity matrix $\mathbf{I}$, and the conjugate transpose $\mathbf{S}^\mathrm{H}$ of the S-matrix. The diagonal entry $(\mathbf{C}_\mathrm{S})_{kk} = kT\,(1 - \sum_j |S_{kj}|^2)$ is the noise power per hertz that port k sends into a matched load. In a reciprocal network such as this core, it is kT times the share of the power entering port k that the network dissipates. A lossless network is therefore noiseless, and a matched resistor emits the full kT. The off-diagonal entries correlate the noise of ports that see the same lossy elements. $\mathbf{C}_\mathrm{S}$ is a covariance and has to be positive semidefinite, which holds exactly when the largest singular value of $\mathbf{S}$ is at most one, so passivity and noise are one condition. For the EM result of the core at 160 GHz, the equation gives 0.69 kT at the LO port, whose path runs through the BPF, the WPD, and a coupler, 0.29 kT at the RF port, and 0.45 kT to 0.55 kT at the four outputs.

The resistors of a vector fit place its poles and do not model the loss of the structure, so snp2le emits them noiseless (`noisy=0`). With `--thermal-noise` , as in `make sparx-core` of the repository, it factors $\mathbf{I} - \mathbf{S}\mathbf{S}^\mathrm{H}$ of the fitted model into a noise generator, driven by one white source of density kT per port, and appends it to the subcircuit without touching the signal network. VACASK sees this noise in `noise` , `hbnoise` , `pnoise` , and transient noise. The documentation discusses the theorem in more detail.

The fit is made from `sparx160_core.s7p` , the EM result without de-embedding, as for the fit in the repository, and the new fit replaces `netlist/spectre/sparx_core_le.inc` in the checkout. The result line reports the RMS error of the fit against the EM data, the number of poles, the largest singular value of the S-matrix (`sigma_max` , at most 1 for a passive model), and whether the DC operating point of the netlist can be solved.

In [57]:
```
RUN_SNP2LE = 1  # 1 = run this cell, 0 = skip it (the receiver then uses the fit
stored in the repository)

if RUN_SNP2LE:
    %pip install -q --progress-bar off snp2le==0.1.12 scikit-rf==2.1.0
    start = time.time()
    ! cd {SPARX_DIR} && QT_QPA_PLATFORM=offscreen snp2le -b convert
verification/em/s-parameter/sparx160_core.s7p --mode universal --order 24 --
format vacask --thermal-noise -o netlist/spectre/sparx_core_le.inc
    print(f"snp2le finished in {(time.time() - start) / 60:.1f} min")
    ! head -12 {SPARX_DIR}/netlist/spectre/sparx_core_le.inc
```

```
Note: you may need to restart the kernel to use updated packages.
[ OK ] verification/em/s-parameter/sparx160_core.s7p -> netlist/spectre/sparx_core
_le.inc  (vacask, rms=3.86e-03  order=23 (12 poles)  sigma_max=1.000  dc=solvable
noise=thermal, 2.2 s)

       note: passivity enforced (n_samples=200, rms=3.86e-03)

snp2le finished in 0.0 min
// sparx_core_le - lumped-element equivalent (VACASK / Spectre), generated by snp2
le

// EQUIVALENT CIRCUIT FOR VECTOR FITTED S-MATRIX

// Created using scikit-rf vectorFitting.py

//

//

// thermal noise of the passive: kT(I - S S^H) at the ports, from a spectral facto
r of this fit

// sources Rnz_e1..Rnz_e7 (0.25 ohm, the only noisy elements), 161 states nz_x*, c
ontrolled sources Gnz_*

// device models needed (declared by your testbench): capacitor, cccs, resistor, v
ccs, vsource

// NOTE: ground is 'GND'. Your testbench must declare it as the ground (Xschem's s
pectre netlist does this with `ground GND`), otherwise the circuit floats and the
result is wrong.

subckt sparx_core_le (p1 p2 p3 p4 p5 p6 p7)

  V1 (p1 s1) vsource dc=0

  R1 (s1 GND) resistor r=50 noisy=0
```

snp2le Result:

The fit reaches an RMS error of 3.9e-3 with 12 poles, is passive (`sigma_max = 1.000`) and carries the thermal noise of the core (`noise=thermal`). It reproduces the fit committed in the repository. Resimulated with VACASK in the repository testbench `sparx_core_le_tb_acsp_vacask` , it matches the EM S-parameters at 160 GHz within 0.45 dB and 9.9 degrees across all 49 S-parameters. Both extremes fall on one output-to-output coupling at -35 dB, every other S-parameter is within 0.36 dB and 5 degrees, see section 1 of the testbench README.

Power Detector: Transfer Curve and Input Impedance (HB and PSS):

The power detector is a forward-biased Schottky barrier diode feeding a transimpedance amplifier, with a replica branch for a differential readout. At the tapeout in March 2026, the detector was checked with a two-tone HB sweep in VACASK, which confirmed square-law detection, an IF output proportional to the product of the RF and the LO amplitude, and nothing beyond. The PSS, HBAC, HBNOISE, PAC, PNOISE, and transient-noise analyses

added to VACASK later in 2026 characterize the detector as a receiver front-end, in this and the following two cells. It is driven by a single tone at 161 GHz through 50 Ohm, swept from -76 dBm to +6.9 dBm of available power with harmonic balance at 15 harmonics. HB solves for the periodic steady state directly in the frequency domain, where a transient would have to integrate thousands of carrier periods until the detected output settles. Nine shooting PSS runs, one every 10 dB from -76 dBm to +4 dBm, solve the same steady state in the time domain. The two algorithms share no code path, so their agreement shows that the transfer curve is real.

The figure shows, from top to bottom, the detected output voltage against the available input power with the square-law line and the 1 dB compression point, the responsivity (detected voltage over input power), which is flat in the square-law region, and the large-signal input impedance at 161 GHz.

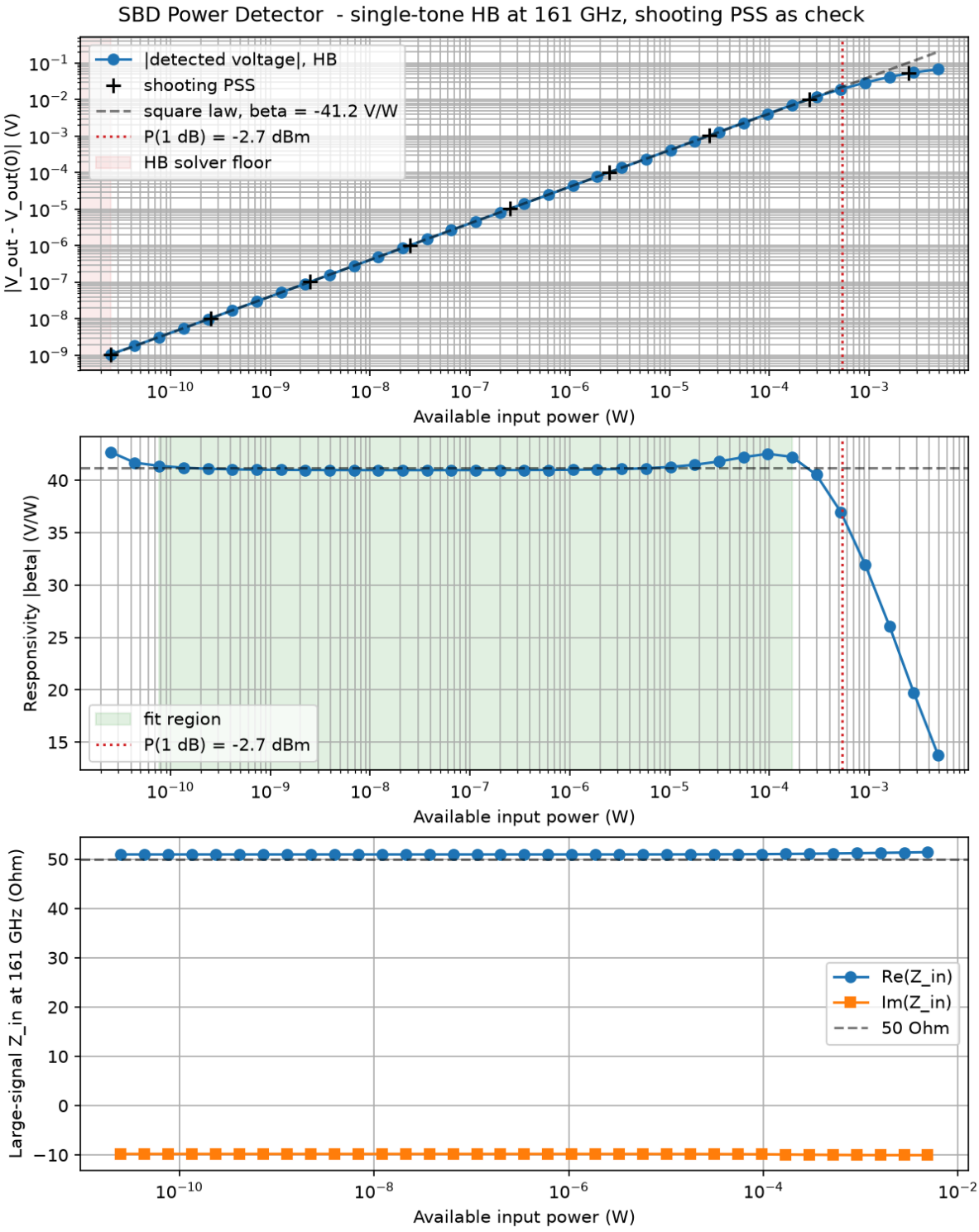
> **Runtime:** On the two vCPUs of the free Colab tier, this cell takes about **1 minute**.

In [58]:
```
RUN_PD_PSS = 1  # 1 = run this simulation, 0 = skip it

if RUN_PD_PSS:
    run_testbench("sparx_powdet_sbd_tb_pss_vacask")
    show_figure("sparx_powdet_sbd_pss_sweep.png")
```

```
sparx_powdet_sbd_tb_pss_vacask: VACASK finished in 0.4 min
Variant               : as fabricated
RF frequency          : 161.0 GHz
Source resistance     : 50 Ohm
Output offset V_dc(0) : -7.0327 mV  (fitted; OP analysis says -7.0332 mV, HB DC bi
n sits +548.2 nV from it)
Responsivity beta     : -41.22 V/W  (fit over -71.1 .. -7.7 dBm)
1 dB compression      : -2.74 dBm
HB solver floor       : -76.0 dBm (points below this scatter off the square-law li
ne)
Solver floor to P1dB  : 73.3 dB
Z_in at 161 GHz     : 50.9 -9.8j Ohm (small signal), 51.4 -10.0j Ohm at 6.9 dBm

Shooting PSS against HB, detected voltage at equal source amplitude:
  P_avail           HB          PSS    PSS/HB
   -76.0dBm   -0.00000mV   -0.00000mV   0.9884  (below the resolved range)
   -66.0dBm   -0.00001mV   -0.00001mV   0.9995  (below the resolved range)
   -56.0dBm   -0.00010mV   -0.00010mV   0.9996
   -46.0dBm   -0.00102mV   -0.00102mV   0.9996
   -36.0dBm   -0.01024mV   -0.01024mV   0.9996
   -26.0dBm   -0.10259mV   -0.10255mV   0.9996
   -16.0dBm   -1.04161mV   -1.04110mV   0.9995
    -6.0dBm  -10.25365mV  -10.29166mV   1.0037
     4.0dBm  -52.15749mV  -52.45610mV   1.0057
  largest deviation over the resolved range: 0.57 %
Wrote /tmp/sparx/testbenches/xschem/plot_simulations/data/sparx_powdet_sbd_beta.js
on
Wrote /tmp/sparx/testbenches/xschem/plot_simulations/data/sparx_powdet_sbd_pss.csv
Wrote /tmp/sparx/testbenches/xschem/plot_simulations/data/sparx_powdet_sbd_pss_che
ck.csv
```



Transfer Curve Result:

quantity	value
responsivity	41.2 V/W, flat from -71 dBm to -8 dBm
1 dB compression point	-2.7 dBm
input impedance at 161 GHz	50.9 - j9.8 Ohm, 51.4 - j10.0 Ohm at +6.9 dBm
shooting PSS against HB	within 0.6 % from -66 dBm to +4 dBm, 1.2 % at -76 dBm

The detected voltage follows the square law, proportional to the input power, up to the compression point, and the input match stays flat up to there. How much of the input the diode converts is stated by the coupling efficiency

$$\eta = \frac{\beta/R_2}{\beta_{I,\max}}, \qquad \beta_{I,\max} = \frac{1}{2nV_\mathrm{T}} = 18.29\ \mathrm{A/W},$$

the current responsivity β/R_2 of the detector, with the feedback resistor $R_2 = 911\ \Omega$ of the transimpedance amplifier, relative to that of an ideal square-law junction with ideality factor $n = 1.057$ and thermal voltage $V_\mathrm{T} = 25.86$ mV. It is only 0.25 % for the fabricated design. The Schottky unit cell has a series resistance of 1.36 kOhm against a junction resistance of 0.9 kOhm at its 30 uA bias, so the 52 Ohm termination, which gives the flat match, passes only about 2 % of the input power into the diode branch, and the series resistance and the junction capacitance of the cell take most of that at 161 GHz. This is the headline result of this block, see section 3.2 of the testbench README.

Power Detector: Noise Figure, NEP, and Minimum Detectable Power (HBAC, HBNOISE, PAC, and PNOISE):

With the LO applied, the detector works as a mixer: an RF signal offset from the LO appears at the output at the difference frequency, the IF. Its noise figure needs two analyses around the large-signal LO solution. HBAC gives the conversion gain from the upper and the lower sideband to the IF, and HBNOISE the output noise with the LO on. HBNOISE evaluates every noise source at the pumped operating point and folds the noise at all LO harmonics down to the IF, which a small-signal noise analysis at the DC operating point cannot do. That analysis, the small-signal noise at the dc operating point with the LO off, is kept as the reference, since the noise figure rested on it before VACASK had HBNOISE. A second pair of HBAC and HBNOISE sweeps the LO power at a 2 GHz IF.

The same conversion and pumped noise are computed a second time around the periodic steady state of the LO found by shooting (PSS), a large-signal solution that shares no code with harmonic balance: PAC is the counterpart of HBAC, and PNOISE of HBNOISE, at the fixed LO and against the LO drive.

With the transducer conversion gain $g = |V_\mathrm{IF}|^2/(2P_\mathrm{RF})$ in V²/W of the upper and the lower sideband from HBAC and the output noise density S_v at the IF, the double-sideband noise figure is

$$F_\mathrm{DSB} = 1 + \frac{S_v(f_\mathrm{IF})}{(g_\mathrm{U} + g_\mathrm{L})\, kT_0},$$

with the Boltzmann constant k and the reference temperature $T_0 = 290$ K. Both sidebands carry signal in a six-port, so the double-sideband definition applies. Without an LO, the detector is characterized by its noise-equivalent power, the input power that gives a signal-to-noise ratio of one in a 1 Hz bandwidth,

$$\mathrm{NEP}(f) = \frac{\sqrt{S_v(f)}}{\beta\,|H(f)|},$$

with the responsivity β of the previous cell and the normalized video response $|H(f)|$ from HBAC.

The figure shows the output noise density with its largest contributors (top left), the conversion of both sidebands to the IF (top right), and the double-sideband noise figure against the IF (bottom left) and against the LO power at a 2 GHz IF (bottom right). The open circles are the PAC and PNOISE results of the shooting cross-check.

> **Runtime:** On the two vCPUs of the free Colab tier, this cell takes about **1 minute**.

In [59]:
```
RUN_PD_NF = 1  # 1 = run this simulation, 0 = skip it

if RUN_PD_NF:
    run_testbench("sparx_powdet_sbd_tb_nf_vacask")
    show_figure("sparx_powdet_sbd_nf.png")
```

```
sparx_powdet_sbd_tb_nf_vacask: VACASK finished in 1.0 min
Variant               : as fabricated
LO                    : 159.0 GHz at -6.5 dBm available
Responsivity beta     : 41.22 V/W  (small signal, from the PSS testbench)
Conversion g at LO    : 0.6695 V^2/W at low IF from hbac, closed form 2 S'^2 P_lo
= 0.6502 V^2/W (S' = 38.01 V/W), hbac - closed form = +0.13 dB
hbnoise gain vs hbac  : within 0.0000 dB over the IF sweep, 0.0048 dB over the LO
sweep
IF output at -46.5 dBm RF: 173.6 uV at low IF, 61.2 uV at 2 GHz
Video bandwidth       : 1.24 GHz (-3 dB of the conversion, interpolated)
Sideband imbalance    : g_L/g_U = +0.02 dB at 2 GHz
Video band 1e+03 .. 5e+09 Hz :  1305.6 uV RMS, MDS  -15.0 dBm  (without PCell PNP:
-23.7 dBm)
Video band 1e+06 .. 5e+09 Hz :   905.3 uV RMS, MDS  -16.6 dBm  (without PCell PNP:
-23.9 dBm)

Output noise, pumped (hbnoise, LO on) against the small-signal noise at the dc ope
rating
point (noise, LO off), and the noise figure from each. The pumped column is the re
sult,
the other one is what the bench reported before hbnoise existed.
      f_IF  ASD pumped  ASD quiesc    p/q   NF_DSB  NF_DSB q   NF_SSB NF no PNP
beta(f)         NEP  NEP no PNP
      [Hz]    [V/rtHz]    [V/rtHz]            [dB]      [dB]     [dB]      [dB]
[V/W]    [W/rtHz]    [W/rtHz]
 1.000e+03   1.379e-05   1.131e-05   1.49   105.50    103.78   108.51     78.23
41.2   2.744e-07   1.493e-08
 9.996e+03   4.361e-06   3.577e-06   1.49    95.50     93.78    98.51     68.23
41.2   8.678e-08   4.724e-09
 9.993e+04   1.379e-06   1.131e-06   1.49    85.50     83.78    88.51     58.28
41.2   2.745e-08   1.501e-09
 9.989e+05   4.363e-07   3.578e-07   1.49    75.50     73.78    78.51     48.68
41.2   8.682e-09   4.958e-10
 9.986e+06   1.380e-07   1.132e-07   1.49    65.52     63.80    68.53     41.41
41.2   2.750e-09   2.119e-10
 9.982e+07   4.262e-08   3.388e-08   1.58    55.11     53.12    58.13     37.69
42.1   8.047e-10   1.353e-10
 5.002e+08   1.812e-08   1.360e-08   1.78    47.49     44.99    50.50     35.05
43.1   3.156e-10   9.762e-11
 9.979e+08   1.077e-08   7.988e-09   1.82    44.68     42.08    47.69     34.90
35.4   2.259e-10   9.387e-11
 1.991e+09   3.359e-09   2.606e-09   1.66    42.20     40.00    45.23     35.21
14.7   1.779e-10   9.872e-11
 5.000e+09   4.596e-10   3.985e-10   1.33    40.28     39.04    43.32     37.07
2.5   1.596e-10   1.258e-10

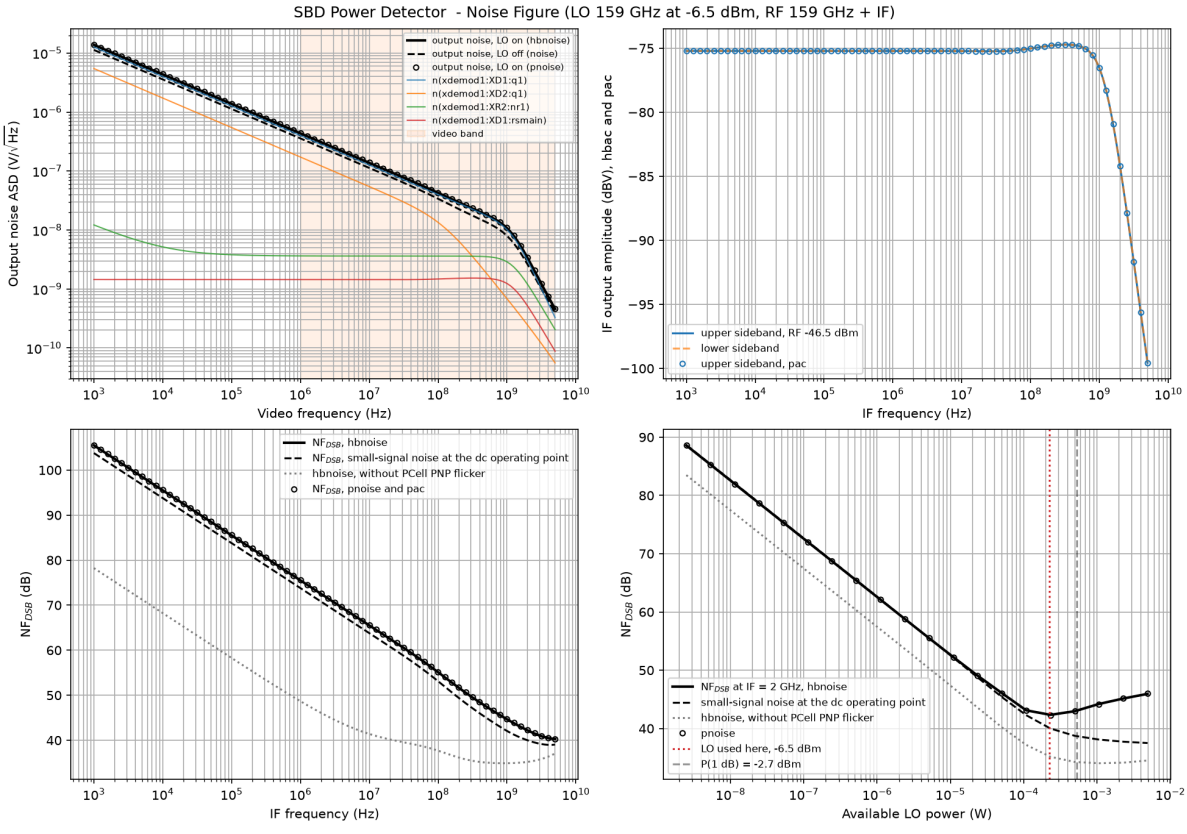
Noise figure against LO drive at 2 GHz IF:
  pumped noise   : best 42.3 dB at -6.3 dBm LO, 42.4 dB at the -6.5 dBm used here
  dc op. noise   : best 37.5 dB at 6.9 dBm LO, 40.1 dB at the -6.5 dBm used here
 P_LO [dBm]  S_int pumped  pumped/quiesc   NF_DSB  NF_DSB q
      -56.0     6.666e-18          0.98    88.58     88.66
      -52.7     6.666e-18          0.98    85.27     85.35
      -49.4     6.666e-18          0.98    81.96     82.04
      -46.1     6.666e-18          0.98    78.65     78.73
      -42.8     6.667e-18          0.98    75.34     75.42
      -39.5     6.669e-18          0.98    72.03     72.11
      -36.2     6.672e-18          0.98    68.72     68.79
      -32.8     6.680e-18          0.98    65.41     65.48
      -29.5     6.695e-18          0.99    62.10     62.16
      -26.2     6.729e-18          0.99    58.80     58.84
```

```
    -22.9     6.801e-18          1.00    55.51     55.50
    -19.6     6.956e-18          1.02    52.25     52.14
    -16.3     7.288e-18          1.07    49.06     48.76
    -13.0     7.940e-18          1.17    46.07     45.39
     -9.7     8.231e-18          1.21    43.12     42.28
     -6.3     1.166e-17          1.72    42.34     39.99
     -3.0     1.798e-17          2.65    42.96     38.74
      0.3     2.774e-17          4.09    44.21     38.10
      3.6     3.742e-17          5.51    45.14     37.73
      6.9     4.740e-17          6.98    45.94     37.50

Shooting cross-check, pac against hbac and pnoise against hbnoise:
  pac - hbac conversion  : +0.0050 .. +0.0068 dB, both sidebands over the IF sweep
  pnoise - hbnoise noise : +0.046 .. +0.095 dB over the IF sweep, +0.078 dB at 2 G
Hz
  pnoise gain vs pac     : within 4.0e-13 dB
  NF_DSB at 2 GHz        : 42.28 dB from pnoise, 42.20 dB from hbnoise
  against LO drive       : pnoise - HB conversion -0.041 .. +0.011 dB, NF -0.073
.. +0.099 dB, best NF 42.3 dB at -6.3 dBm

Top output-noise contributors over 1e+06 .. 5e+09 Hz, pumped, and their pumped-ove
r-quiescent ratio:
  n(xdemod1:XD1:q1)                      86.70 %   x1.84
  n(xdemod1:XD2:q1)                      10.99 %   x0.76
  n(xdemod1:XR2:nr1)                      1.31 %   x1.05
  n(xdemod1:XD1:rsmain)                   0.24 %   x0.91
  n(xdemod1:XR4:nr1)                      0.20 %   x1.03
  n(xdemod1:XD1:dmain)                    0.19 %   x1.99

Wrote /tmp/sparx/testbenches/xschem/plot_simulations/data/sparx_powdet_sbd_nf.json
Wrote /tmp/sparx/testbenches/xschem/plot_simulations/data/sparx_powdet_sbd_nf.csv
Wrote /tmp/sparx/testbenches/xschem/plot_simulations/data/sparx_powdet_sbd_nf_lo.c
sv
```



Noise Figure Result:

At a 2 GHz IF and -6.5 dBm of LO, the drive each detector sees in the receiver:

quantity	value
noise figure, double sideband, HBNOISE	42.2 dB
noise figure, double sideband, PNOISE with PAC (shooting cross-check)	42.3 dB
noise figure, double sideband, small-signal noise at the dc operating point	40.0 dB
best noise figure against the LO drive, HBNOISE	42.3 dB at -6.3 dBm
output noise with the LO on over the LO off	1.66 (1.5 at low IF, 1.8 near 1 GHz)
video bandwidth	1.24 GHz
NEP	8.7 nW/sqrt(Hz) at 1 MHz, 226 pW/sqrt(Hz) at 1 GHz
minimum detectable power, 1 MHz to 5 GHz	-16.6 dBm

- About 98 % of the pumped output noise power between 1 MHz and 5 GHz (97 % of the small-signal noise at the dc operating point) is flicker noise of the parasitic vertical PNP that the PDK model of the Schottky diode contains, so no device that was sized in this design sets the noise floor. Without it, the noise figure at 2 GHz would be 35.2 dB and the minimum detectable power -23.9 dBm.
- Because that noise is 1/f across the whole video band, detecting at DC, as a conventional six-port does, is expensive: moving the output to a 1 GHz IF improves the NEP 38-fold.
- The LO does not leave the noise alone. Over the IF, the small-signal noise at the dc operating point misses 1.7 dB to 2.6 dB of noise figure and is therefore a lower bound. Against the LO drive the gap grows: 2.3 dB at the minimum and 8.4 dB at +6.9 dBm, where the small-signal estimate keeps falling to 37.5 dB while HBNOISE and PNOISE rise from 42.3 dB to 45.9 dB. A mixer noise figure needs a large-signal noise analysis. The conversion side is on firm ground: the gain that HBNOISE reports agrees with HBAC within 0.005 dB.
- The shooting cross-check confirms the pumped result without harmonic balance: PAC agrees with HBAC within 0.007 dB and PNOISE with HBNOISE within 0.1 dB over the whole IF range, which gives 42.3 dB against 42.2 dB at 2 GHz. Against the LO drive the two agree within 0.1 dB as well.
- The dominant noise source is a model extrapolation, since the PNP flicker model is evaluated at about 20 pA of base current, far below any plausible characterization current. The silicon decides which figure is right, see section 3.3 of the testbench README.

Power Detector: Transient Noise:

Transient noise solves the circuit in the time domain with random noise sources and shares no code path with HBNOISE, so it is the independent check of the linearization around the pumped LO solution. Three 100 ns transients with the LO on at -6.5 dBm differ

only in the noise amplitude (`noisescale` 0.01, 0.0316, and 0.1). The output spectrum of each run is split into a term linear in the noise power and a remainder, and the linear term is compared against HBNOISE, PNOISE, and the small-signal noise at the dc operating point of the previous cell. The noise amplitude has to stay this small: at full amplitude, the transient noise of this stiff, strongly nonlinear circuit produces a numerical excess that is not physical.

The first figure shows the output noise of the three runs normalized by their noise amplitude, which collapse onto one curve if the circuit responds linearly, together with the linear term and the excess. The second figure compares the noise figure from four noise estimates: transient noise, HBNOISE, PNOISE, and the small-signal noise at the dc operating point.

> **Runtime:** On the two vCPUs of the free Colab tier, this cell takes about **6 minutes**.

In [60]:
```
RUN_PD_TN = 0  # 1 = run this simulation, 0 = skip it

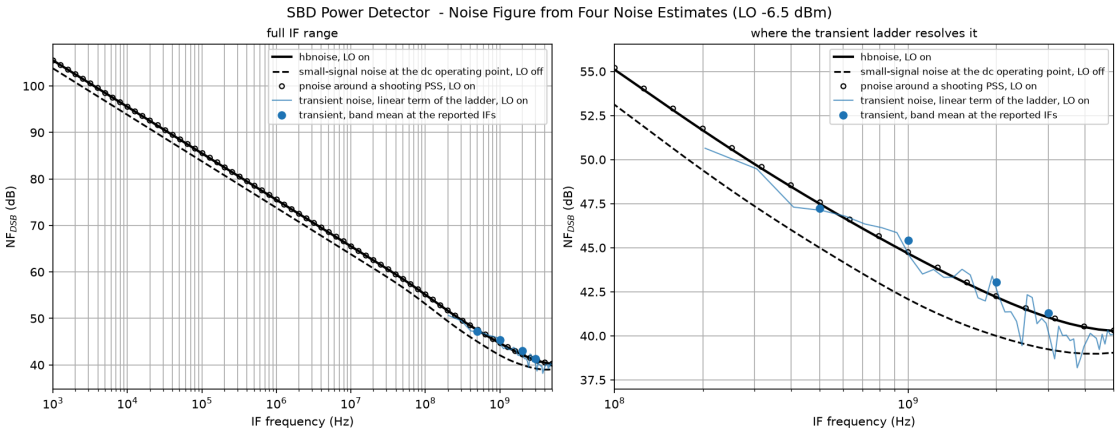
if RUN_PD_TN:
    if not os.path.exists(f"{SIM_DIR}/powdet_nf_hbnoise_usb.raw"):
        print("The transient-noise simulation needs the result of the noise-
figure simulation, run that cell first.")
    else:
        run_testbench("sparx_powdet_sbd_tb_tn_vacask")
        show_figure("sparx_powdet_sbd_tn.png")
        show_figure("sparx_powdet_sbd_nf_compare.png")
```

Transient Noise Result:

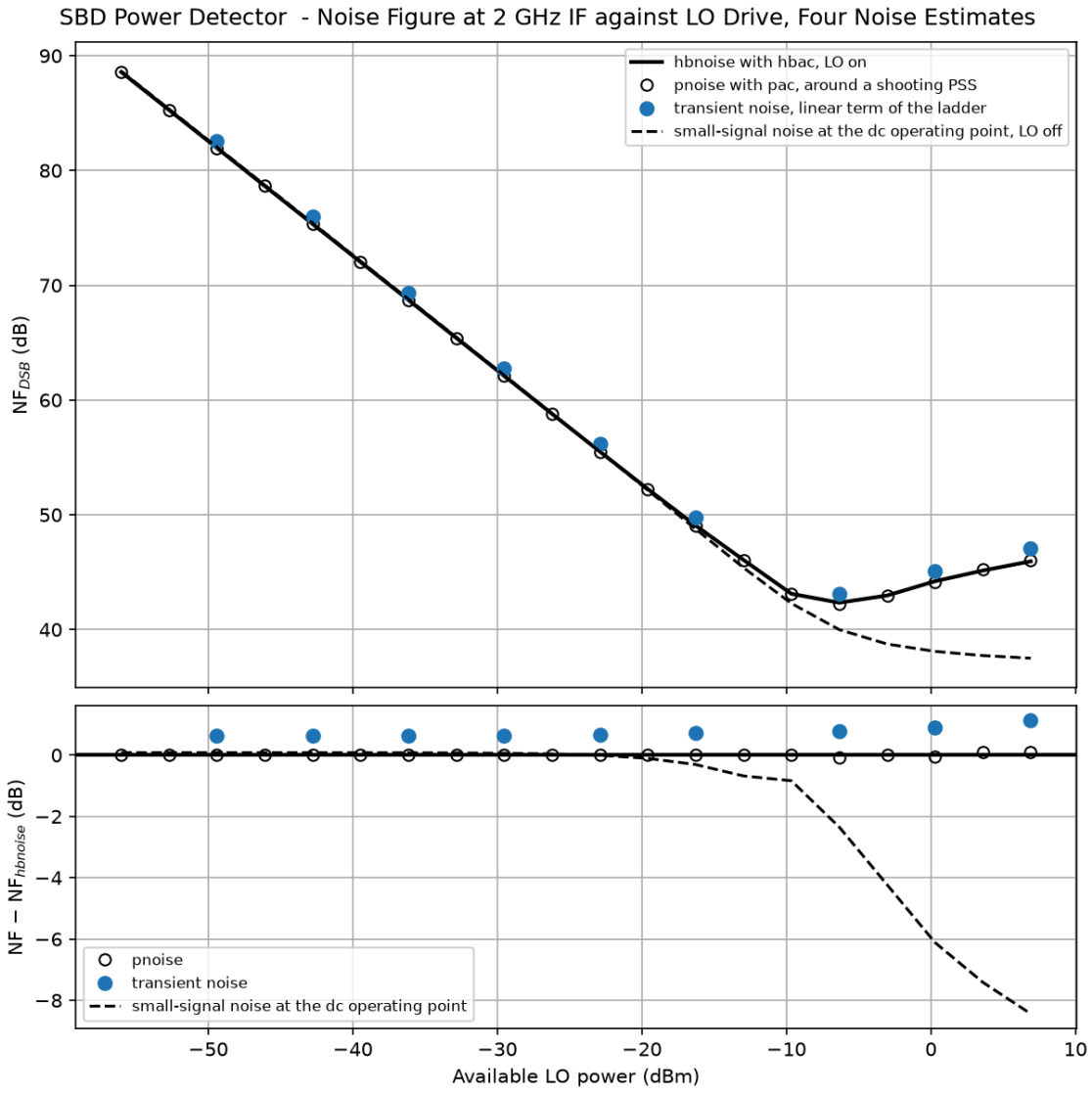
Double-sideband noise figure of one run of this notebook, with the HBAC conversion of the noise-figure simulation in the denominator:

IF	transient noise	HBNOISE	PNOISE	small-signal noise at the dc operating point
0.5 GHz	47.2 dB	47.5 dB	47.6 dB	45.0 dB
1 GHz	45.4 dB	44.7 dB	44.8 dB	42.1 dB
2 GHz	43.0 dB	42.2 dB	42.3 dB	40.0 dB
3 GHz	41.3 dB	41.1 dB	41.1 dB	39.2 dB

The three runs collapse onto one curve within 2 % when normalized by their noise amplitude, so the transient is in its linear regime. It lies within 0.9 dB of HBNOISE and 2.0 dB to 3.4 dB above the small-signal noise at the dc operating point, with the same frequency dependence. PNOISE around the shooting PSS sits within 0.1 dB of HBNOISE, so the offset of the transient belongs to the transient estimate and not to the periodic analyses. Transient noise is a random process, and a noise figure from a 100 ns record is good to about 1 dB: section 3.4 of the testbench README lists three noise seeds. The reference result of the repository:



The repository runs the same comparison against the LO drive at a 2 GHz IF, with a transient-noise ladder at nine LO levels (`sparx_powdet_sbd_tb_tn_lo_vacask` , about 24 minutes, not part of this notebook). PNOISE stays within 0.1 dB of HBNOISE over the whole LO sweep, and the transient lies 0.6 dB to 1.1 dB above them and follows their rise past compression. The small-signal noise at the dc operating point leaves the three above about -16 dBm and keeps falling, so it is 8.4 dB optimistic at +6.9 dBm, see section 3.5 of the testbench README:



Receiver: Six-Port Core with Post-Layout Power Detectors (HB, HBAC, and Transient):

The receiver testbench combines the snp2le fit of the six-port core with the four power detectors, each replaced by its post-layout extraction with `powdet_variant.py`, as `make sim-xschem VARIANT=m1_pex` does in the repository, and each output loaded with 5 pF off-chip. The LO at 159 GHz and +12 dBm and the RF at 161 GHz and -20 dBm are applied at the pads through 50 Ohm. One testbench runs all analyses:

- HB with the RF alone and with the LO alone gives the loss of the RF and LO paths from the pads to each detector, and the drive each detector sees.
- HBAC gives the IF response of the four outputs from 1 MHz to 5 GHz, and the IF output at 2 GHz against the LO power at the pad.
- A two-tone HB at the levels of the transient is an independent cross-check.
- A 10 ns transient gives the output waveforms at the 2 GHz IF. It runs this long because the detected DC needs about that time to settle.

The figure shows the last nanosecond of the four differential outputs, with the 2 GHz IF and a small carrier feed-through riding on it (top), the IF output at 2 GHz against the LO power (middle), and the IF response of the four outputs (bottom).

> **Runtime:** On the two vCPUs of the free Colab tier, this cell takes about **15 minutes**.

In [61]:
```
RUN_RX = 0  # 1 = run this simulation, 0 = skip it

if RUN_RX:
    # the schematic detector subcircuit is replaced by the Magic full-RC
extraction
    ! mkdir -p {SIM_DIR}/m1_pex && cp {SIM_DIR}/sparx_top_le_tb_rx_vacask.save
{SIM_DIR}/m1_pex/
    ! python3 {SPARX_DIR}/scripts/powdet_variant.py --variant m1_pex
{SIM_DIR}/sparx_top_le_tb_rx_vacask.spectre
{SIM_DIR}/m1_pex/sparx_top_le_tb_rx_vacask.spectre
    run_testbench("sparx_top_le_tb_rx_vacask", "m1_pex")
    show_figure("sparx_top_le_rx_m1_pex.png")
```

Receiver Result:

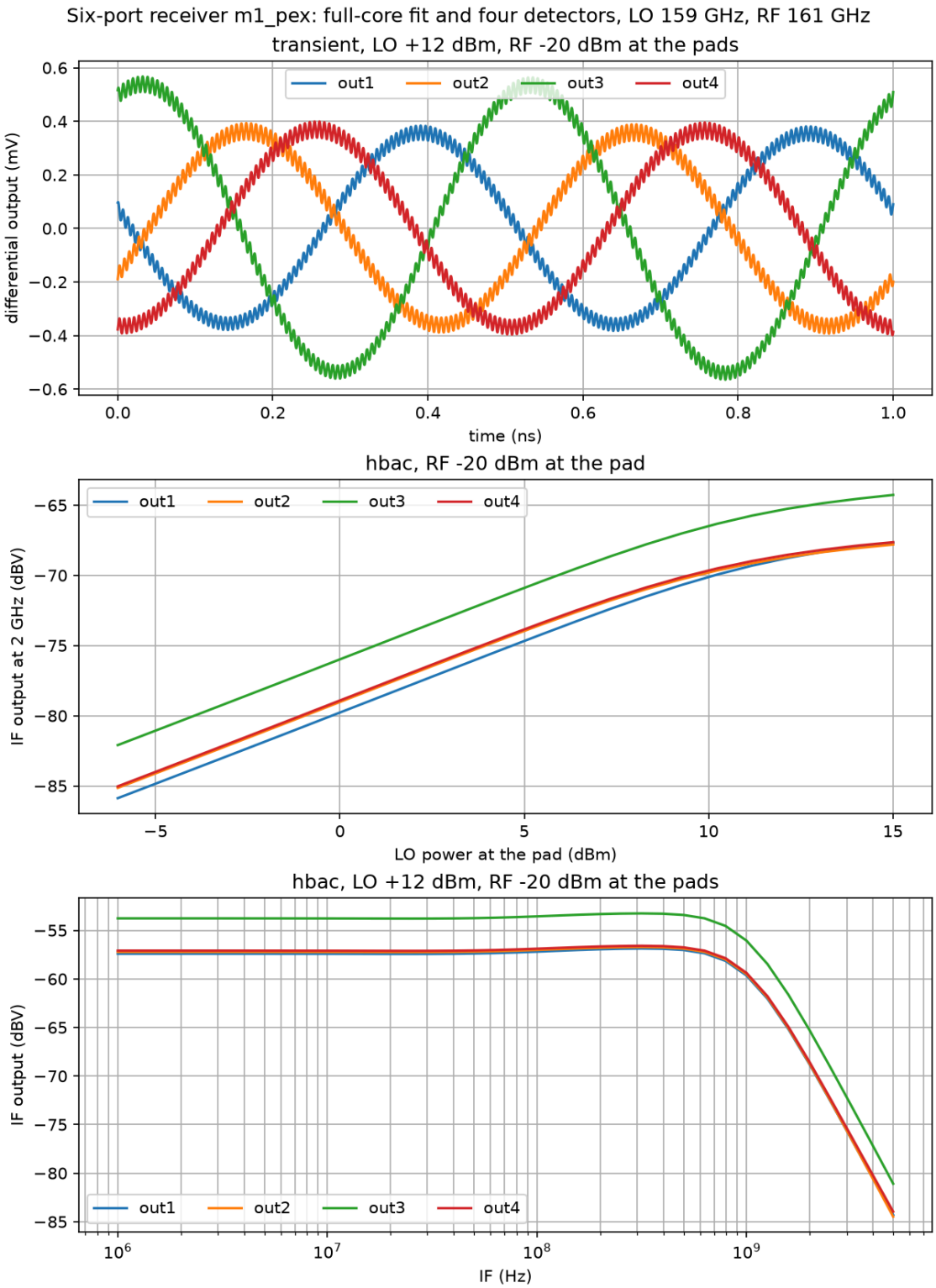
quantity	value
LO path loss, pad to detector	17.3 dB to 18.6 dB
LO power at the detectors	-6.6 dBm to -5.3 dBm
RF path loss, pad to detector	5.4 dB to 9.3 dB
I/Q amplitude imbalance	2.0 dB
I/Q phase difference	85.4 degrees in the transient, 85.6 degrees from HBAC
transient against HBAC	within 0.2 dB
two-tone HB against HBAC	within 0.5 dB

- The LO path through filter, divider, and coupler places each detector at the operating point the detector was characterized at above. The losses are into the post-layout

detectors, whose input impedance differs from 50 Ohm, so they differ from the 50 Ohm insertion loss of the EM solve by up to about 2 dB per output.

- The amplitude and phase imbalance of the differential I and Q outputs is absorbed by the six-port calibration. The EM data alone predict it within 0.1 dB and 0.1 degree: terminating the EM core on its four detector ports with the post-layout detector impedance, 40.3 - j25.0 Ohm at 161 GHz, and forming the product of the conjugated LO-path and the RF-path waves per output gives 2.1 dB and 85.3 degrees. The circuit simulation and the EM solve agree without any fitting. With 50 Ohm on every port the EM gives 2.5 dB and 84.5 degrees, so that difference is the mismatch of the detectors, which also makes V_{Q-} the largest of the four outputs.
- The noise figure of the receiver is simulated in the next cell, through the core fit and its thermal noise. Section 4.1 of the testbench README describes this testbench in detail.

The reference result of the repository:



Receiver: Noise Figure through the Noisy Core Model (HBAC, HBNOISE, PAC, and PNOISE):

The receiver noise figure used to be stated as a composition, the detector noise figure plus the RF path loss, because the fitted core had no noise of its own. With the thermal noise of the core in the snp2le fit, this testbench simulates it: the four detectors pumped through the fitted LO path, the RF reaching them through the fitted RF path, and the detectors loading the core with their own input impedance. The LO is at 159 GHz and +12 dBm at the pad, as in the receiver testbench, and the RF is a small signal at both sidebands of the LO. Six outputs are evaluated, the four differential detector outputs out1 to out4 (V_{I-}, V_{I+}, V_{Q-}, V_{Q+}) and the six-port outputs I = out2 - out1 and Q = out4 -

out3, which every noise analysis takes as a node pair. The double-sideband noise figure is that of the detector, referred to the RF pad, with the noise of the 50 Ohm RF source removed from the numerator. Per output, the testbench runs:

- the small-signal noise at the dc operating point, with the LO off,
- HBAC for both sidebands and HBNOISE around the HB solution of the LO at 15 harmonics,
- PAC for both sidebands and PNOISE around the shooting PSS of the same LO,
- HBAC and HBNOISE at a 2 GHz IF against the LO power at the pad, from -6 dBm to +18 dBm, and PNOISE of I and Q at five of these levels.

The shooting PSS of the fitted core needs a first stabilization step of 20 fs or more, and the testbench uses `stabstep=50f`. At the default of a thousandth of the LO period, the stabilization transient aborts. The detectors run as fabricated, since with their post-layout extraction the testbench takes about 2.5 hours of CPU time.

The figure shows the noise figure of I and Q over the IF with the PNOISE cross-check as open circles (top left), that of the four detector outputs with the composition of the detector noise figure and the RF path loss as crosses (top right), the IF conversion from HBAC with the PAC cross-check (bottom left), and the noise figure at a 2 GHz IF against the LO power at the pad (bottom right).

> **Runtime:** This cell takes 31 minutes on a laptop with an Intel Core Ultra 7 255U, and an estimated **45 minutes** on the two vCPUs of the free Colab tier.

In [62]:
```
RUN_RX_NF = 0  # 1 = run this simulation, 0 = skip it

if RUN_RX_NF:
    run_testbench("sparx_top_le_tb_nf_vacask")
    show_figure("sparx_top_le_nf.png")
```

Receiver Noise Figure Result:

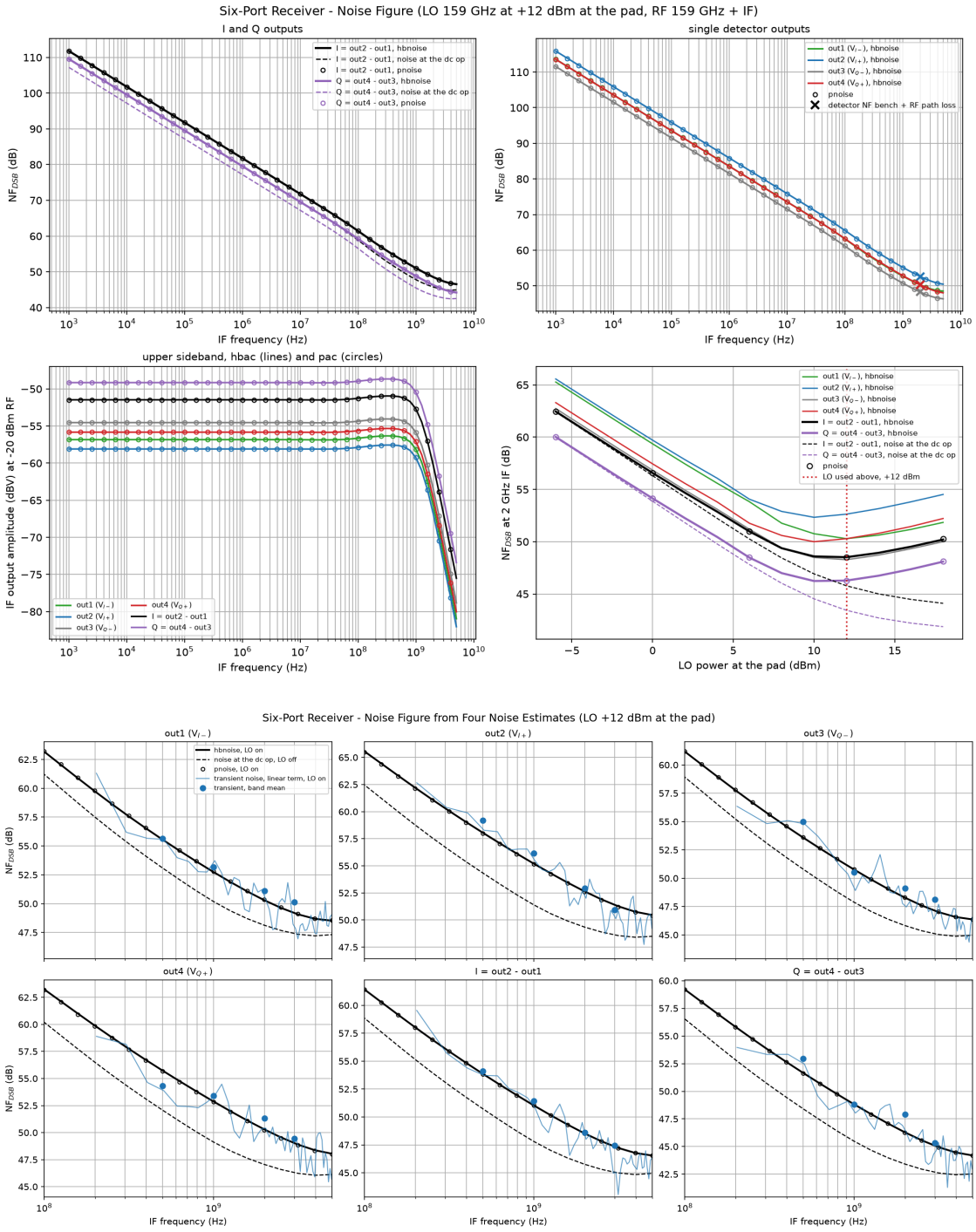
Double-sideband noise figure referred to the RF pad, at a 2 GHz IF and +12 dBm of LO at the pad:

output	HBNOISE	PNOISE	small-signal noise at the dc operating point	post-layout, HBNOISE
out1 (V_{I-})	50.3 dB	50.3 dB	48.1 dB	51.3 dB
out2 (V_{I+})	52.6 dB	52.6 dB	49.3 dB	51.6 dB
out3 (V_{Q-})	48.3 dB	48.3 dB	45.8 dB	47.8 dB
out4 (V_{Q+})	50.3 dB	50.2 dB	47.1 dB	51.4 dB
I	48.5 dB	48.5 dB	45.8 dB	48.5 dB

output	HBNOISE	PNOISE	small-signal noise at the dc operating point	post-layout, HBNOISE
Q	46.3 dB	46.3 dB	43.4 dB	46.5 dB

- **I and Q are better than either of their detectors**, by 1.8 dB and 2.0 dB. The two members of a pair sit about 180 degrees apart at the IF, so their signals add coherently in the difference while their noise adds in power.
- **The LO source noise cancels in I and Q.** It reaches both detectors of a pair through the same LO path, and the difference holds it 13.5 dB (I) and 17.0 dB (Q) below the sum of the two single outputs. The thermal noise of the 50 Ohm LO source is negligible either way, but the mechanism rejects the AM noise of a real LO the same way.
- **The composition holds.** Each detector output equals the noise figure of the detector simulation above at the LO drive that detector sees, plus its RF path loss, within 0.12 dB as fabricated and 0.28 dB post-layout, once both are referred to the power delivered into the detector.
- **The core's own thermal noise adds 0.0001 dB.** The noise generator of the snp2le fit carries 2e-5 to 3e-5 of the output noise, because 6 dB to 10 dB of passive loss sits in front of detectors with a noise figure of 42 dB. The detector sets the noise floor of this receiver.
- **PNOISE confirms HBNOISE, and the small-signal estimate is optimistic again.** PAC agrees with HBAC within 0.007 dB and PNOISE with HBNOISE within 0.1 dB on all six outputs, while the small-signal noise at the dc operating point is 2.2 dB to 3.3 dB optimistic, as on the single detector.
- **The noise figure against the LO drive has its minimum at the operating point**, +10 dBm to +12 dBm at the pad. Below it, the noise figure falls by 1 dB per dB of LO, the square-law regime.

The transient-noise ladder of the detector, run on the receiver in the repository (`sparx_top_le_tb_tn_vacask` , about 30 minutes, not part of this notebook), is the fourth estimate. It lies -1.4 dB to +1.6 dB around HBNOISE on all six outputs, the realization scatter of a 100 ns record, see section 4.2 of the testbench README. The reference results of the repository, the noise figure as fabricated and the comparison of the four noise estimates:



Conclusion

This notebook generated SPARX, a mmWave six-port receiver in the 130-nm IHP-Open-PDK, entirely in Python and with open-source tools only:

- **Layout:** The complete chip, from the Schottky-diode power detector over the six-port network of branch-line couplers, Wilkinson power divider, and hairpin bandpass filter to pads, sealring, and metal fill, is derived from a single design frequency `f`. Changing it regenerates the full layout, as the video demonstrates from 60 GHz to 300 GHz. The 160 GHz version was taped out in March 2026.
- **Verification:** The generated layout passes the KLayout DRC of the IHP-Open-PDK. At the `regular` level, only the antenna violations that IHP waived for the tapeout

remain.

- **EM to circuit:** snp2le turns the 7-port EM result of the six-port core into a passive lumped-element netlist, which reproduces the EM S-parameters at 160 GHz within 0.45 dB and 9.9 degrees, carries the thermal noise of the core by Bosma's theorem, and makes the passive network usable in harmonic balance, shooting, noise, and transient simulations.
- **Power detector:** HB and PSS give a responsivity of 41 V/W with a 1 dB compression point of -2.7 dBm. HBAC and HBNOISE give a double-sideband noise figure of 42 dB at a 2 GHz IF, dominated by the flicker noise of the parasitic PNP in the diode model. PAC and PNOISE around a shooting PSS confirm HBAC and HBNOISE within 0.1 dB, transient noise within about 1 dB, while the small-signal noise at the dc operating point is 2.3 dB optimistic at the operating point and 8.4 dB at +6.9 dBm of LO. A mixer noise figure needs a large-signal noise analysis.
- **Receiver:** With the post-layout detectors, the LO path places each detector at its characterized operating point, and the I/Q imbalance of 2.0 dB and 85.4 degrees agrees with the EM core loaded by the detectors, 2.1 dB and 85.3 degrees. The noise figure of the receiver, simulated through the noisy core model, is 48.5 dB at I and 46.3 dB at Q (46.5 dB post-layout), and the thermal noise of the core adds 0.0001 dB of it.

The frequency scaling rests on the closed-form models alone, without any tuning: the branch-line couplers and the Wilkinson power divider keep an input matching of -16 dB or better over the whole range from 60 GHz to 300 GHz, with a coupler amplitude imbalance of 1.2 dB to 1.8 dB.

Outlook: Silicon measurements of the 160 GHz chip are planned, followed by a revised design with 16 parallel Schottky cells per detector, which raises the responsivity from 41 V/W to 251 V/W, an optimized bandpass filter, and active RF blocks such as an LNA, an LO amplifier, and a VCO.

The complete design and verification flow, including the EM simulations with AWS Palace, LVS, and parasitic extraction, is available in the SPARX repository and runs in the IIC-OSIC-TOOLS container. The SPARX documentation describes it with all results.

Clean Up and Total Runtime

In [63]:
```
! rm -rf {IHP_DIR} {PDK_ROOT} {ASSET_DIR} /tmp/klayout.deb {SPARX_DIR}


print(f"Total runtime: {(time.time() - NOTEBOOK_START) / 60:.1f} min")
```

```
Total runtime: 6.4 min
```